Práctica 01 - Sistemas Inteligentes

Satisfacción de restricciones



Nombre: Daironas Sukis

NIE: Y1759578T

Grupo: 01

Asignatura: Sistemas Inteligentes, Ingeniería Informática, Universidad de Alicante

0. Índice

0. Índice 2
1. Sesión 1 - Introducción, entorno de trabajo y clase Variable 4
1.1 Contexto 4
1.2 Creación de la Clase Variable 4
1.3 Modificación del main.py 5
1.3.1 Función ConstruyeVariablesHorizontales 5
1.3.2 Función filtraDominioHorizontal 7
1.3.3 Función ConstruyeVariablesVertical 8
1.3.4 Función filtraDominioVertical 9
1.4 Modificación de la función main 9
1.5 Ejecución del crucigrama 10
2. Sesión 2 - Implementación del algoritmo backtracking 12
2.1 Contexto 12
2.2 Modificación del variable.py 12
2.3 Modificación del main.py 13
2.3.1 Función encuentraRestricciones 13
2.3.2 Función isFeasible 14
2.3.3 Función BackTracking 15
2.3.4 Función pintarSolucion 16
2.3.5 Modificación de la función main 16
2.4 Ejecución del algoritmo BackTracking 16
3. Sesión 3 y 4 - Implementación del algoritmo Forward Checking 18
3.1 Contexto 18
3.2 Modificación del variable.py 18
3.3 Modificación del main.py 19
3.3.1 Función esConsistenteFC 19
3.3.2 Función forward 20
3.3.3 Función restaurar 20
3.3.4 Función FC 21
3.3.5 Reorganización de la función main 22
3.3.6 Guardar y cargar crucigramas 23
3.4 Ejecución del algoritmo Forward Checking 25
3.4.1 Tablero 3x3 con solución 25
3.4.2 Tablero 5x4 sin solución 26
4. Sesión 5 y 6 - Implementación del algoritmo AC3 27
4.1 Contexto 27
4.2 Modificación del main.py 27
4.2.1 Función esConsistenteCruce 27
4.2.2 Función AC3 28
4.2.3 Medición de tiempos 29
4.2.4 Modificación de la función resuelve 30
4.2.5 Botón AC3 y variable tiempo AC3 31
4.3 Ejecución del crucigrama 32
4.3.1 Creación de un diccionario 32
4.3.2 Tablero 6x6 33
4.3.3 Tablero 7x7 38
4.3.4 Tablero 9x9 41
4.3.5 Análisis de los resultados obtenidos 45
5. Sesión 7 - Implementación del algoritmo BackJumping 50
5.1 Contexto 50
5.2 Modificación de variable.py 51
5.3 Reorganización del proyecto 51
5.4 Modificación de algorithms.py 52
5.4.1 Función conflictosCon 52

5.4.2 Función backJumpingRec....52
5.4.3 Función backJumping....54
5.5 Modificación de main.py....54
5.5.1 Botón BJ....54
5.6 Ejecución del algoritmo BackJumping....54
5.6.1 Tablero 6x6....55
5.6.2 Tablero 7x7....57
5.6.3 Tablero 9x9....58
5.6.4 Análisis de los resultados obtenidos....59
6. Sesión 8 - Documentación y pruebas....63
6.1 Contexto....63
6.2 Clase Variable....63
6.3 Cómo se han tratado las casillas con carácteres fijos....64
6.4 Especificación formal....64
6.5 Traza de Forward Checking....67
6.6 Traza de AC3....75
6.7 Heurística de ordenación de variables....80
6.7.1 Función ordenarVariables....80
6.7.2 Modificación main.py....81
6.7.3 Ejecución de heurísticas en tablero 9x9....81
6.7.4 Ejecución de heurísticas en tablero 7x7....85
6.7.5 Resultados de las heurísticas....90
6.7.6 Conclusiones sobre la heurística....92
6.8 Casuísticas....93
6.8.1 Caso 1: sin solución detectado por AC3....93
6.8.2 Caso 2: AC3 deja valores, pero no hay solución....93
6.8.3 Caso 3: con solución....94
6.9 Estudio de tiempo con otro diccionario....95
7. Dificultades encontradas....101
8. Conclusiones....101
9. Uso de IA....103
10. Referencias bibliográficas....104

1. Sesión 1 - Introducción, entorno de trabajo y clase Variable

1.1 Contexto

Se ha escogido la versión con la plantilla que usa tkinter, y para esta primera sesión se ha añadido el archivo **variable.py**, donde se guardará la implementación de la clase variable. El archivo **main.py** ha sido modificado como podremos ver a continuación.

1.2 Creación de la Clase Variable

Los atributos añadidos a la clase Variable han sido:

```
class Variable:
    def __init__(self, nombre, tipo, fila, col, longitud, dominio):
        self.nombre = nombre
        self.tipo = tipo        # h/v
        self.fila = fila
        self.col = col
        self.longitud = longitud
        self.dominio = dominio
        self.restricciones = []
```

- **nombre**: para identificar la Variable
- **tipo**: puede ser horizontal o vertical
- **fila**, **col**: para indicar en qué posición empieza la Variable
- **longitud**
- **dominio**: las palabras que pueden entrar en esa Variable, serán las que coincidan exactamente con su longitud, y en caso de que hubiese un carácter en esa variable, que coincida la posición del carácter con el de la palabra
- **restricciones**: esta no se necesita en esta primera sesión, pero el enunciado nos pide este atributo

El método **__str__** implementado, devuelve un string en formato específico de la clase Variable.

```
    def __str__(self):
        return f"Nombre {self.nombre} Posición {self.fila} {self.col} Tipo: {self.tipo} Dominio:
{self.dominio}"
```

1.3 Modificación del main.py

Una vez implementada la clase Variable, necesitamos una función que recorra la tabla en busca de las Variables disponibles. Para ello se ha implementado dos funciones principales, una que recorre por filas la tabla, en búsqueda de Variables horizontales, y otra que recorre por columnas las tablas, en búsqueda de variables verticales.

1.3.1 Función ConstruyeVariablesHorizontales

```
def construyeVariablesHorizontal(tablero, almacen):
    variables = []
    contador = 0

    for fila in range(tablero.getAlto()):
        longitudVariable = 0
        col_inicio = 0
        for col in range(tablero.getAncho()):
            if tablero.getCelda(fila,col) != LLENA:
                if longitudVariable == 0:
                    col_inicio = col
                longitudVariable += 1

            elif longitudVariable >= 2:                # Evitamos crear variables de un solo
hueco
                # Creamos la variable
                dominio = filtraDominioHorizontal(tablero, almacen, fila, col_inicio,
longitudVariable)
                nuevaVariable = Variable(contador, "H", fila, col_inicio, longitudVariable,
dominio)
                variables.append(nuevaVariable)
                contador += 1
                longitudVariable = 0
            else:
                longitudVariable = 0

        if longitudVariable >= 2:
            dominio = filtraDominioHorizontal(tablero, almacen, fila, col_inicio,
longitudVariable)
```

```
            nuevaVariable = Variable(contador, "H", fila, col_inicio, longitudVariable, dominio)
            variables.append(nuevaVariable)
            contador += 1
            longitudVariable = 0

    return variables, contador
```

Esta función recibe como parámetros de entrada el tablero a recorrer, y el almacén, que es un array de palabras que se van a usar en el crucigrama. También inicializamos un Array de variables vacío que servirá para almacenar todas las variables horizontales que vamos a encontrar.

Una vez inicializado los atributos necesarios, se procede a recorrer el tablero mediante dos bucles for anidados, e inicializamos las Variables *"longitudVariable"*, para calcular cuántos carácteres va a tener, y *"col_inicio"*, para guardar en qué columna empieza dicha Variable.

Recorriendo celda por celda, necesitamos comprobar los siguientes casos para identificar una Variable a guardar:

- **Caso 1: Si la celda no está llena**

Es decir si está vacía o contiene una letra. En este caso sabemos que posiblemente pertenezca a una Variable (a menos que solo ocupe una celda), y si justo en esa iteración *longitudVariable* es igual a cero, pues sabemos que es el comienzo de una variable, por lo que asignamos col_inicio al número de columna actual e incrementamos en 1 el valor de *longitudVariable* actual.

- **Caso 2: Si longitudVariable es mayor o igual que 2**

En esta condición ya sabemos que la celda está llena, y comprobamos que la longitud de la variable es mayor o igual que dos, para evitar que se creen variables de una sola celda.

Con la ayuda de la función auxiliar *filtraDominioHorizontal*, obtenemos el dominio de esa variable. Posteriormente se procede a crear dicha variable y se añade a la lista de variables, incrementando también el contador para seguir el orden del id, y se restablece la longitud para la siguiente variable.

- **Caso 3: longitudVariable vale 1 o menos**

Esta es la condición en la que sabemos que la celda está llena y ademas *longitudVariable* es menor o igual a 1, de esta manera no guardamos la variable y reiniciamos el contador *longitudVariable*.

Al finalizar el bucle for que recorre la fila entera, nos aseguramos de que no se haya quedado a medias una Variable porque justo la última iteración si que era una celda que podia pertenecer. Por lo que comprobamos si longitudVariable es mayor o igual que 2, y creamos de nuevo la variable.

Una vez comprobadas todas las celdas, devolvemos la lista de variables y el contador

1.3.2 Función filtraDominioHorizontal

```
def filtraDominioHorizontal(tablero, almacen, fila, col_inicio, longitud):
    dominio = almacen.get(longitud, set()).copy()

    for i in range(longitud):
        celda = tablero.getCelda(fila, col_inicio + i)
        if celda != VACIA and celda != LLENA:
            nuevoFiltrado = []
            for palabra in dominio:
                if palabra[i] == celda:
                    nuevoFiltrado.append(palabra)
            dominio = nuevoFiltrado

    return set(dominio)
```

Esta función se encarga de obtener con el método ***.get()***, todas aquellas palabras guardadas en el almacén, que tenga una cantidad de carácteres establecida por la variable *“longitud”*. El bucle for anidado, se encarga de filtrar la lista de palabras, de manera que si encuentra una celda con un carácter, se asegura de que el dominio sólo esté compuesto por palabras que contengan dicho carácter en su posición específica.

Al finalizar el filtrado, se convierte el resultado a un conjunto (set(dominio)), para garantizar que el dominio de la variable sea siempre de este tipo, independientemente de si el hueco contenía o no alguna letra fija

1.3.3 Función ConstruyeVariablesVertical

Esta función funciona de la misma manera que la explicada anteriormente, lo único que lo diferencia es que el recorrido lo hace de manera vertical, y se le pasa en contador como parámetro, para seguir el orden de los nombres de las Variables.

```
def construyeVariablesVertical(tablero, almacen, contador):             # Aqui pasamos el contador como parametro para evitar que el nombre de las variables se solapen con los de la funcion "construyeVariablesHorizontal"
    variables = []

    for col in range(tablero.getAncho()):
        longitudVariable = 0
        fila_inicio = 0
        for fila in range(tablero.getAlto()):
            if tablero.getCelda(fila,col) != LLENA:
                if longitudVariable == 0:
                    fila_inicio = fila
                longitudVariable += 1

            elif longitudVariable >= 2:                # Evitamos crear variables de un solo hueco
                # Creamos la variable
                dominio = filtraDominioVertical(tablero, almacen, fila_inicio, col, longitudVariable)

                nuevaVariable = Variable(contador, "V", fila_inicio, col, longitudVariable, dominio)
                variables.append(nuevaVariable)
                contador += 1
                longitudVariable = 0
            else:
                longitudVariable = 0

        if longitudVariable >= 2:
            dominio = filtraDominioVertical(tablero, almacen, fila_inicio, col, longitudVariable)
            nuevaVariable = Variable(contador, "V", fila_inicio, col, longitudVariable, dominio)
            variables.append(nuevaVariable)
            contador += 1
            longitudVariable = 0
```

```
    return variables
```

1.3.4 Función filtraDominioVertical

Funciona también de la misma manera, solo que el recorrido en vertical

```
def filtraDominioVertical(tablero, almacen, fila_inicio, col, longitud):
    dominio = almacen.get(longitud, set()).copy()

    for i in range(longitud):
        celda = tablero.getCelda(fila_inicio + i, col)
        if celda != VACIA and celda != LLENA:
            nuevoFiltrado = []
            for palabra in dominio:
                if palabra[i] == celda:
                    nuevoFiltrado.append(palabra)
            dominio = nuevoFiltrado

    return set(dominio)
```

1.4 Modificación de la función main

Se ha añadido código para poder ver visualmente el botón VAR, y hacerlo funcional a la hora de pulsarlo.

```
    posBotBK=altoVentana//5-altoBoton//2
    posBotFC=(altoVentana//5)*2-altoBoton//2
    posBotAC3=(altoVentana//5)*3-altoBoton//2
    posBotVariable=(altoVentana//5)*4-altoBoton//2

    botBK=pygame.Rect(anchoVentana-95, posBotBK, 70, altoBoton)
    botFC=pygame.Rect(anchoVentana-95,posBotFC , 70, altoBoton)
    botAC3=pygame.Rect(anchoVentana-95, posBotAC3, 70, altoBoton)
    botVariable=pygame.Rect(anchoVentana-95, posBotVariable, 70, altoBoton)
```

```
        pintarBoton(screen, fuenteBot, botBK, "BK")
```

```
        pintarBoton(screen, fuenteBot, botFC, "FC")
        pintarBoton(screen, fuenteBot, botAC3, "AC3")
        pintarBoton(screen, fuenteBot, botVariable, "VAR")
```

También se ha modificado la condición del if, para detectar cuando se pulsa el botón de Variable, y así poder obtener la lista de variables para imprimirlas en consola.

```
                elif pulsaBoton(pos, botVariable):
                    variablesHorizontales, contador = construyeVariablesHorizontal(tablero,
almacen)
                    variablesVerticales = construyeVariablesVertical(tablero, almacen, contador)
                    todasVariables = variablesHorizontales + variablesVerticales
                    for v in todasVariables:
                        print(v)
```

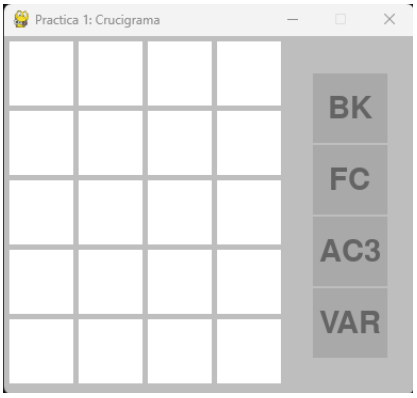
1.5 Ejecución del crucigrama

Para ejecutar el crucigrama y poder comprobar el trabajo realizado debemos de abrir una consola, y situados al nivel de los archivos del código fuente ejecutar:

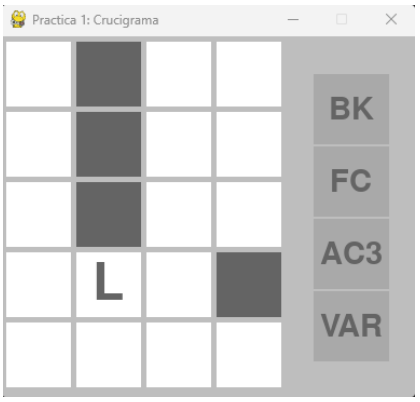
```
python main.py --filas M –columnas N --dic fichero.txt
```

```
PS C:\Users\Usuario\Desktop\SistemasInteligentes\Practica01> py main.py --filas 5 --columnas 4 --fich d0.txt
pygame 2.6.1 (SDL 2.28.4, Python 3.13.15)
Hello from the pygame community. https://www.pygame.org/contribute.html
Uso: python main.py --filas M --columnas N --dic archivo.txt
```

Una vez ejecutado se abrirá la interfaz gráfica del crucigrama, donde podremos manipularlo a nuestro antojo pulsando el click izquierdo para ocupar una celda, y el click derecho para poder insertar un carácter.



Modificando el crucigrama para probar un ejemplo quedaría así:



Y al pulsar el botón VAR, en la consola se imprimiría la siguiente lista de Variables:

```
Nombre 0 Posición 0 2 Tipo: H Dominio: {'DE', 'NO', 'AR', 'ES', 'SO', 'AS', 'SI', 'AL', 'LA'}
Nombre 1 Posición 1 2 Tipo: H Dominio: {'DE', 'NO', 'AR', 'ES', 'SO', 'AS', 'SI', 'AL', 'LA'}
Nombre 2 Posición 2 2 Tipo: H Dominio: {'DE', 'NO', 'AR', 'ES', 'SO', 'AS', 'SI', 'AL', 'LA'}
Nombre 3 Posición 3 0 Tipo: H Dominio: ['OLA']
Nombre 4 Posición 4 0 Tipo: H Dominio: {'LISO', 'MORA', 'PERO', 'ROSA', 'OSOS', 'OLOR'}
Nombre 5 Posición 0 0 Tipo: V Dominio: {'CIRCO', 'ASILO', 'RETOS', 'FOCOS', 'SETOS', 'ESOPO', 'OSERA', 'TOTEM'}
Nombre 6 Posición 3 1 Tipo: V Dominio: ['LA']
Nombre 7 Posición 0 2 Tipo: V Dominio: {'CIRCO', 'ASILO', 'RETOS', 'FOCOS', 'SETOS', 'ESOPO', 'OSERA', 'TOTEM'}
Nombre 8 Posición 0 3 Tipo: V Dominio: {'CON', 'SOL', 'OLA', 'OSO', 'ARA', 'RON'}
```

2. Sesión 2 - Implementación del algoritmo backtracking

2.1 Contexto

En esta sesión se pide implementar el algoritmo de *backtracking*, para ello primero necesitamos implementar una función, *encuentraRestricciones* en el que se hallen las restricciones de las variables, es decir, comprobar si dada una variable se cruza con alguna otra, restringiendo el uso de distintas posibles palabras. Posteriormente, se implementará la función *isFeasible*, con el objetivo de comprobar si una palabra es candidata a pertenecer a dicha variable. La función de *BackTracking*, que usará la función *isFeasible*, consistirá en el propio algoritmo para probar todas las combinaciones necesarias con el objetivo de encontrar la solución del crucigrama. Una vez encontrada la solución, se usará la función *pintarSolucion* para rellenar visualmente el crucigrama resuelto.

2.2 Modificación del variable.py

Para esta sesión vamos a necesitar un nuevo atributo de la clase *Variable* que represente el estado de la variable. Si la variable está libre y no tiene ninguna palabra asignada, entonces el valor de la variable *valor* será ***None***. Si la variable está ocupada, contendrá el *String* de la palabra ocupada. Este atributo no era necesario en la *Sesión 01* debido a que no existía aún un algoritmo que pudiese resolver el crucigrama, y tuviese asignar a las variables un valor, sin embargo en esta *Sesión 02* es necesario guardar valores y comprobar si palabra está siendo utilizada por otra variable.

```
class Variable:
    def __init__(self, nombre, tipo, fila, col, longitud, dominio):
        self.nombre = nombre
        self.tipo = tipo        # h/v
        self.fila = fila
        self.col = col
        self.longitud = longitud
        self.dominio = dominio
        self.restricciones = []
        self.valor = None
```

2.3 Modificación del main.py

2.3.1 Función encuentraRestricciones

Esta función se encarga de encontrar todas las restricciones de una variable.

```
def encuentraRestricciones(listaVariables):
    variablesHorizontales = []
    variablesVerticales = []

    for v in listaVariables:
        if v.tipo == 'H':
            variablesHorizontales.append(v)
        else:
            variablesVerticales.append(v)

    for var_h in variablesHorizontales:
        for var_v in variablesVerticales:
                # CONDICIÓN 1: la variable vertical está posicionada entre la columna inicial
y la columna final de la variable horizontal
                # CONDICIÓN 2: la variable horizontal está posicionada entre la fila inicial y
la fila final
                # Si se cumplen las 2 condiciones, las variables se cruzan
                if var_h.col <= var_v.col <= var_h.col + var_h.longitud - 1 and var_v.fila <=
var_h.fila <= var_v.fila + var_v.longitud - 1:
                    # En que posición de la variable se cruzan
                    pos_en_h = var_v.col - var_h.col
                    pos_en_v = var_h.fila - var_v.fila

                    var_h.restricciones.append((var_v, pos_en_h, pos_en_v))
                    var_v.restricciones.append((var_h, pos_en_v, pos_en_h))
```

Primero separamos las variables en dos listas, horizontales y verticales, y recorremos el primer bucle for para guardar en cada una de estas listas la variable correspondiente. El siguiente bucle for anidado nos permite recorrer todos los casos para cada una de las variables sin que se repitan las restricciones.

Una vez que tenemos la variable horizontal y la variable vertical, tenemos que comprobar si se cruzan entre ellas, y para eso tenemos que comprobar 2 cosas.

1. Como la variable vertical siempre va a estar posicionada en la misma columna, tenemos que comprobar que la posición inicial de la variable horizontal esté en la

misma columna o en una columna de la izquierda, y que la posición final esté en la misma columna o en una de las columnas de la derecha.

2. Como la variable horizontal va a estar en la misma fila siempre, tenemos que comprobar que la posición inicial de la variable vertical esté en la misma fila o en una de las filas de arriba, y que la posición final esté en la misma fila o en una de las filas de abajo.

Con estas dos condiciones cumplidas, podemos asegurar que las variables se cruzan. Al cumplirla, guardamos para la variable horizontal, y para la variable vertical, en que número de casilla choca cada una, y la añadimos a la lista de restricciones como una tupla que guarda con la variable que se choca, el número de celda en que se choca, y el número de celda que se choca el vecino.

2.3.2 Función isFeasible

Esta función determina si una palabra candidata puede asignarse a una variable concreta sin generar conflictos. Primero comprueba que la palabra no esté siendo utilizada en otra variable distinta del crucigrama. Posteriormente recorre todas las restricciones de la variable, y para cada variable vecina que tenga, comprueba si tiene asignada una palabra, y si es así, comprueba que en la celda que se cruzan coincida justo con el carácter. Si alguna de estas comprobaciones no tienen éxito, simplemente la función devuelve *False*, dado a entender que dicha palabra no se puede meter en dicha variable.

```
def isFeasible(variable, palabra, listaVariables):

    # Si la palabra pasada por parámetro esta siendo utilizada en otra
variable, FALSE
    for var in listaVariables:
        if var != variable:
            if var.valor == palabra:
                return False

    for restriction in variable.restricciones:
        # El vecino si tiene una palabra puesta
        if restriction[0].valor is not None:
```

```
            if palabra[restriction[1]] !=
restriction[0].valor[restriction[2]]:
                return False
    return True
```

2.3.3 Función BackTracking

Se trata de una función recursiva en el que tiene como parámetro la variable k, para contar en qué índice de *listaVariables* se encuentra. El caso base de la función recursiva es que el índice ya ha recorrido toda la *listaVariables*, y se ha encontrado solución con BackTracking.

Para el caso general, comprobamos que para todo dominio, es decir, para todas las posibilidades de palabra que entran en el crucigrama, de la variable que estamos comprobando, si es factible. Si lo es, asignamos a la variable su respectivo *valor*, y realizamos la llamada recursiva con el índice actualizado. Si el backTracking no ha encontrado una solución con esa palabra, esa palabra se descarta y se sigue con la siguiente palabra.

Si la llamada recursiva no encuentra una solución completa a partir de esa asignación, se deshace la designación y se prueba con el siguiente valor del dominio. Si se agotan todos los valores del dominio sin éxito, la función devolverá False, indicando a la variable anterior en la recursión de que pruebe otro valor distinto.

```
def backTracking(k, listaVariables):
    if k == len(listaVariables):
        return True
    else:
        actualVariable = listaVariables[k]
        for dom in actualVariable.dominio:
            if isFeasible(actualVariable, dom, listaVariables):
                actualVariable.valor = dom
                if backTracking(k+1, listaVariables):
                    return True
                actualVariable.valor = None
    return False
```

2.3.4 Función pintarSolucion

Esta función recorre toda la lista de variables, imprimiendo en la tabla cada carácter correspondiente al atributo valor, y en su respectiva posición.

```
def pintarSolucion(listaVariables, tablero):
    for var in listaVariables:
        for i in range(var.longitud):
            if var.tipo == 'H':
                tablero.setCelda(var.fila, var.col+i, var.valor[i])
            else:
                tablero.setCelda(var.fila+i, var.col, var.valor[i])
```

2.3.5 Modificación de la función main

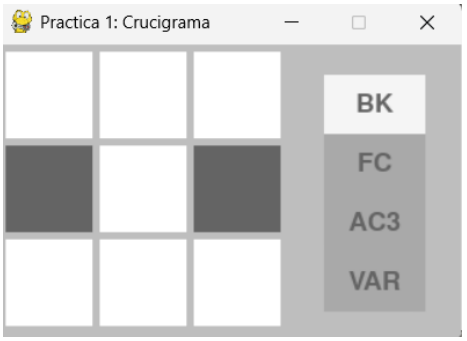
Se ha realizado una modificación en el bloque correspondiente al botón BK para que al pulsarlo, se cree la todas las variables del crucigrama, y se ejecute el algoritmo de backtracking.

```
                    print('BK')
                    variablesHorizontales, contador = construyeVariablesHorizontal(tablero,
almacen)
                    variablesVerticales = construyeVariablesVertical(tablero, almacen, contador)
                    todasVariables = variablesHorizontales + variablesVerticales
                    encuentraRestricciones(todasVariables)
                    if backTracking(0, todasVariables):
                        pintarSolucion(todasVariables, tablero)
                    else:
                        MessageBox.showwarning("Alerta", "No hay solución")
```

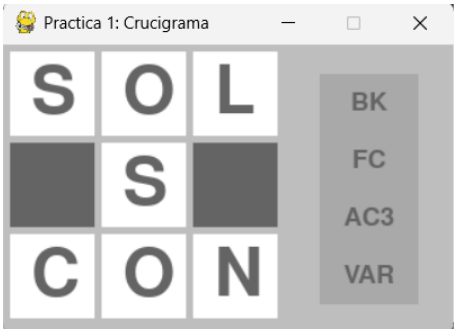
2.4 Ejecución del algoritmo BackTracking

Con un tablero 3x3, y el fichero d0.txt que viene con la práctica, si tapamos algunas celdas.

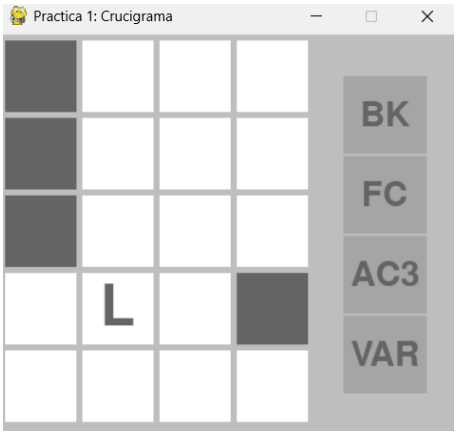




Ejecutando el algoritmo de BackTracking tenemos el siguiente resultado.



Con un tablero de 5x4, y el fichero d0.txt, con el carácter *L* en esa posición, el BackTracking no encuentra ninguna solución y se queda así.



3. Sesión 3 y 4 - Implementación del algoritmo Forward Checking

3.1 Contexto

En esta sesión se pide implementar el algoritmo de *Forward Checking (FC)*, una mejora sobre el backtracking visto en la sesión anterior. Mientras que backtracking solo comprueba si una asignación es consistente con las variables ya asignadas, Forward Checking va un poco más allá, ya que cada vez que asigna un valor a una variable, se revisan los dominios de las variables futuras, y se eliminan temporalmente las palabras que no serían compatibles con dicha asignación. Así si conseguimos que el dominio de una variable futura se quede vacía, se descarta la asignación actual sin la necesidad de seguir explorando ramas que ya sabemos que van a fallar.

3.2 Modificación del variable.py

Cuando tengamos que podar una palabra del dominio de la variable, tendremos que eliminarla del dominio, pero si llegamos a necesitar deshacer esa poda, necesitamos un nuevo atributo en la clase, una lista que almacene la tupla **(palabra_podada, variable_responsable)**, en este caso lo hemos llamado **podado**.

```
class Variable:
    def __init__(self, nombre, tipo, fila, col, longitud, dominio):
        self.nombre = nombre
        self.tipo = tipo        # h/v
        self.fila = fila
        self.col = col
        self.longitud = longitud
        self.dominio = dominio
        self.restricciones = [] # lista de tuplas (vecino, pos_propia, pos_vecino)
        self.valor = None
        self.podado = []    # lista de tuplas (palabra_podada, variable_responsable)
```

Es necesario guardar que variable ha sido la responsable en podar, ya que de esta manera sabremos qué podas deshacer para cada variable concreta cuando esta se desasigne, sin

afectar a las podas que hayan hecho otras variables que siguen asignadas en ese momento.

3.3 Modificación del main.py

3.3.1 Función esConsistenteFC

```
def esConsistenteFC(var_i ,valor_i, var_j, valor_j):
    if valor_i == valor_j:
        return False
    for restriction in var_i.restricciones:
        if restriction[0] == var_j:
            if valor_i[restriction[1]] == valor_j[restriction[2]]:
                return True
            else:
                return False
    return True
```

Esta función comprueba si dos valores concretos, de dos variables concretas, son compatibles entre sí, comprobando las dos restricciones que hay entre dos variables del crucigrama.

Primero comprueba que las dos palabras no sean iguales, ya que en el crucigrama no puede haber palabras repetidas, y posteriormente recorremos las restricciones de *var_i*, buscando la que corresponde con *var_j*, y comprueba que el carácter donde se cruzan coincidan con el valor de cada una. Como dos variables solo pueden cruzarse como máximo una vez, por eso retornamos nada más realizar esa comprobación. Si no encuentra como restricción a la *var_j* buscada, significa que las variables no se cruzan, por lo que no pueden entrar en conflicto por el cruce.

Es importante recalcar que esta función es distinta de *isFeasible*, ya que este comprueba si una palabra es válida para una variable teniendo en cuenta todas sus restricciones y que no esté repetida en el crucigrama. En cambio *esConsistenteFC* solo comprueba que son compatibles dos variables específicas, con dos valores concretos, usada por *forward* para decidir qué palabras podar de las variables futuras. Como también comprueba que son distintas, forward elimina la palabra asignada de los dominios de las variables futuras, y por eso FC ya no necesita usar *isFeasible*.

3.3.2 Función forward

```
def forward(k, listaVariables):
    dominio_vacio = False
    var_i = listaVariables[k]
    valor_i = var_i.valor

    for j in range(k+1, len(listaVariables)):
        var_j = listaVariables[j]

        for a in var_j.dominio.copy():
            if not esConsistenteFC(var_i, valor_i, var_j, a):
                var_j.dominio.remove(a)
                var_j.podado.append((a, var_i))

        if len(var_j.dominio) == 0:
            dominio_vacio = True
            break
    return not dominio_vacio
```

Una vez que se haya asignado un valor a la variable actual *var_i*, esta función recorre todas las variables futuras, desde *k+1 (la siguiente variable de la actual)*, hasta el final de la lista. Para cada una de las variables que vamos a recorrer, se va a recorrer su dominio, comprobando si cada palabra sigue siendo consistente con la asignación recién hecha. Recorremos la copia del dominio en el bucle, ya que estamos modificando el propio dominio y así evitamos problemas mientras se modifica.

Si en la comprobación encontramos que una palabra deja de ser consistente, se elimina del dominio de esa variable futura y se guarda en la lista de podado una tupla *(palabra_podada, variable_responsable)*, para poder deshacer esta poda más adelante si hace falta.

Después de haber revisado el dominio completo de una variable futura, se comprueba si se ha quedado vacío, si es así se marca *dominio_vacio* = *True*, y cortamos el bucle ya que no tiene sentido seguir comprobando las demás variables, ya que no va a llevar a ninguna solución.

Al final devolvemos ***not dominio_vacio***, si se ha vaciado algún dominio devolverá *False*, si no devolverá *True*.

3.3.3 Función restaurar

```
def restaurar(k, listaVariables):
```

```
    var_i = listaVariables[k]


    for j in range(k+1, len(listaVariables)):
        var_j = listaVariables[j]


        for elem_podado in var_j.podado.copy():
            palabra, responsable = elem_podado
            if responsable == var_i:
                var_j.podado.remove(elem_podado)
                var_j.dominio.add(palabra)
```

Esta función deshace lo que hizo forward para una asignación concreta. Recorre las mismas variables futuras *(desde k+1 hasta el final de la lista)*, y para cada una de estas revisa la lista de podados, buscando las tuplas cuyo responsable haya sido la variable *var_i* que se está desasignando.

Si encuentra alguna, elimina la tupla de la lista de podados, y devuelve la palabra al dominio.

3.3.4 Función FC

```
def FC(k, listaVariables):
    if k == len(listaVariables):
        return True
    actualVar = listaVariables[k]
    for a in listaVariables[k].dominio.copy():
        actualVar.valor = a
        if forward(k,listaVariables):
            if FC(k+1, listaVariables):
                return True
        actualVar.valor = None
        restaurar(k, listaVariables)
    return False
```

La estructura de esta función es muy parecida a BackTracking, pero con dos diferencias principales.

1. No usa *isFeasible* antes de asignar un valor. No hace falta comprobar la palabra con las variables ya asignadas, ya que cada una de ellas al asignarse eliminó con forward de los dominios de las variables futuras las palabras incompatibles, y por

tanto por el cruce como por estar repetidas. Por lo tanto cualquier palabra que quede en el dominio de la variable actual es válida.

2. Después de asignar la variable, llamamos a forward para podar las variables futuras. Solo si *forward* tiene éxito,sin dejar ningún dominio de las variables vacío, se continuará con la llamada recursiva *FC(k+1, listaVariables).*

Tanto si forward falla como si la llamada recursiva no encuentra solución, se desasigna la variable actual (*actualVar.valor = None*), y se llama a restaurar para devolver el dominio a esas variables futuras que se habían podado por culpa de esta asignación.

3.3.5 Reorganización de la función main

Con la incorporación de Forward Checking, el bloque de eventos del main empezaba a ocupar bastante código, y encima código repetitivo entre los botones *BK y FC*, ya que ambos siguen exactamente el mismo flujo: construir las variables, calcular las restricciones, ejecutar un algoritmo de búsqueda, y pintar la solución o avisar si no la hay. La única diferencia entre ambos botones es qué algoritmo se ejecuta. Para evitar esta repetición se han creado las siguientes dos funciones.

```
def obtenerVariables(tablero, almacen):
    variablesHorizontales, contador = construyeVariablesHorizontal(tablero, almacen)
    variablesVerticales = construyeVariablesVertical(tablero, almacen, contador)
    todasVariables = variablesHorizontales + variablesVerticales
    encuentraRestricciones(todasVariables)
    return todasVariables
```

```
def resuelve(algoritmo, tablero, todasVariables):
    if algoritmo(0, todasVariables):
        pintarSolucion(todasVariables, tablero)
    else:
        MessageBox.showwarning("Alerta", "No hay solución")
```

De esta manera en *obtenerVariables* agrupamos la construcción de las variables y el cálculo de las restricciones de dichas variables, mientras que en *resuelve* agrupamos la ejecución del algoritmo, el pintado de la solución, y el mensaje de error.

La función resuelve puede recibir backTracking o FC como parámetro del algoritmo ya que ambas funciones comparten la misma forma de parámetro de entrada, y devuelven True o False.

Además, en el main se ha añadido la variable *todasVariables*, que permite conservar las variables ya construidas si primero se pulsa el botón VAR y, a continuación, se resuelve el crucigrama con *BK* o *FC,* evitando así construirlas dos veces. De esta manera, evitamos trabajar con variables desactualizadas.

La variable *todasVariables* se pone a *None* cada vez que el estado del tablero cambia, es decir, si se edita una celda o cargamos un crucigrama distinto, para que la siguiente vez que se pulse un botón que necesite las variables, estas se reconstruyan a partir del tablero actual.

Tras ejecutar *BK* o *FC*, *todasVariables* también se pone a None, ya que las variables quedan en un estado ya resuelto y no tendría sentido reutilizarlas para una nueva resolución.

```
                    if todasVariables is None:
                        todasVariables = obtenerVariables(tablero, almacen)
                    resuelve(backTracking, tablero, todasVariables)
                    todasVariables = None

                elif pulsaBoton(pos, botFC):
                    if todasVariables is None:
                        todasVariables = obtenerVariables(tablero, almacen)
                    resuelve(FC, tablero, todasVariables)
                    todasVariables = None
```

3.3.6 Guardar y cargar crucigramas

Como una herramienta extra, se han añadido dos botones nuevos, *SAVE* y *LOAD*, que permiten guardar el estado actual del tablero en un fichero de texto, y volver a cargarlo más adelante, sin tener que redibujar el crucigrama cada vez que se quiera probar.

Guardar

```
def guardarCrucigrama(tablero, file):
    f = open(file, 'w', encoding="utf-8")
```

```
    for fila in range(tablero.getAlto()):
        celdas = []
        for col in range(tablero.getAncho()):
            celda = tablero.getCelda(fila, col)
            if celda == VACIA:
                celdas.append('0')
            elif celda == LLENA:
                celdas.append('1')
            else:
                celdas.append(celda)
        f.write(' '.join(celdas) + '\n')
    f.close()
```

A resumidas cuentas, esta función recorre el tablero celda por celda escribiendo un 0 para las celdas vacías, un 1 para las celdas negras, y el propio carácter para las celdas con una letra fija. Cada fila del fichero representa una fila del tablero, con los valores separados por espacios.

Al pulsar el botón *SAVE*,se pide al usuario un nombre del fichero sin la extensión .txt.

```
elif pulsaBoton(pos, botGuardarCrucigrama):
                    nombre = askstring("Guardar", "Nombre del fichero (sin .txt)")
                    if nombre:
                        guardarCrucigrama(tablero, nombre + ".txt")
                    print("Crucigrama guardado")
```

Cargar

Con el objetivo de leer estos ficheros, se ha añadido un método a la clase Tablero:

```
    def cargar(self, fichero):
        f = open(fichero, "r", encoding="utf-8")
        self.tablero = []

        fila = -1
        for linea in f:
            celdas = linea.split()
            if len(celdas) > 0:
                fila += 1
                self.tablero.append([])
```

```
                for car in celdas:
                    if car == '0':
                        self.tablero[fila].append('-')
                    elif car == '1':
                        self.tablero[fila].append('*')
                    else:
                        self.tablero[fila].append(car.upper())
        f.close()

        self.ancho = len(self.tablero[0])
        self.alto = len(self.tablero)
```

Se encarga de leer el fichero de línea en línea para traducir cada uno de los valores a las celdas, ignorando las líneas vacías.

Al pulsar el botón *LOAD*, se abre un explorador de archivos para elegir el fichero:

```
elif pulsaBoton(pos, botCargarCrucigrama):
                    ruta = askopenfilename(filetypes=[("Texto", "*.txt")])
                    if ruta:
                        tablero.cargar(ruta)
                        if tablero.getAlto() != filas or tablero.getAncho() != cols:
                            MessageBox.showwarning("Alerta", "El crucigrama guardado no tiene el
tamaño del tablero actual")
                            tablero = Tablero(filas, cols)
                    todasVariables = None
```

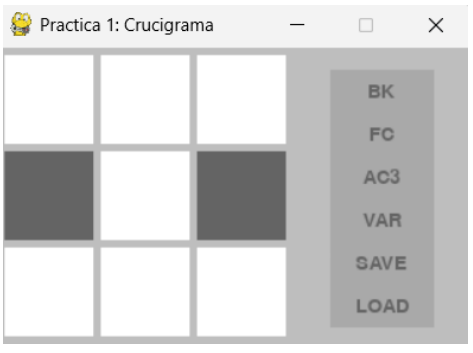
Solo se cargan crucigramas del mismo tamaño que el tablero actual, ya que aceptar un tamaño distinto obligaría a redimensionar la ventana y recalcular la posición de los botones, además de causar errores al intentar acceder a celdas fuera de rango.

3.4 Ejecución del algoritmo Forward Checking

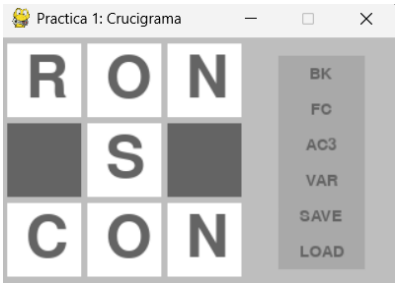
3.4.1 Tablero 3x3 con solución

Con un tablero de 3x3 y el fichero d0.txt, tapando las celdas (1,0) y (1,2):



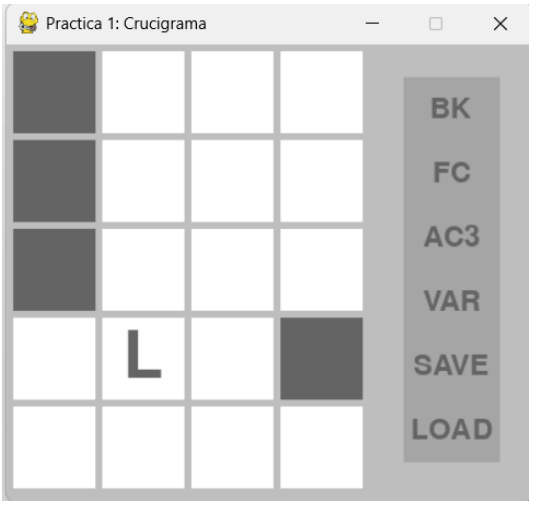


Ejecutando el algoritmo de Forward Checking tenemos el siguiente resultado.



3.4.2 Tablero 5x4 sin solución

Con un tablero de 5x4 y el fichero d0.txt, tapando las celdas (0,0), (1,0), (2,0) y (3,3), e introduciendo el carácter L en la posición (3,1):





4. Sesión 5 y 6 - Implementación del algoritmo AC3

4.1 Contexto

El algoritmo AC3 no se encarga de resolver el crucigrama, si no de eliminar los dominios de las palabras que no van a poder formar de ninguna solución. En el momento que pulsemos el botón de AC3 se nos mostrará por la consola los dominios antes y después de la ejecución del algoritmo. Una vez reducidos dichos dominios usaremos BackTracking o Forward Checking para que trabajen con los dominios ya reducidos.

Además también se ha añadido una medición de tiempos, para poder comparar los distintos tiempos de resolución del crucigrama de los algoritmos, usando o sin usar AC3.

4.2 Modificación del main.py

4.2.1 Función esConsistenteCruce

```
def esConsistenteCruce(var_k, palabra_k, var_m, palabra_m):
    for restriction in var_k.restricciones:
        if restriction[0] == var_m:
            if palabra_k[restriction[1]] == palabra_m[restriction[2]]:
                return True
            else:
                return False
    return True
```

Esta función comprueba si dos valores concretos, de dos variables concretas, son compatibles en el punto en el que se cruzan, y para ello recorremos las restricciones de *var_k* buscando la que corresponde con *var_m*, y comprobamos que el carácter donde se cruzan coinciden en el valor de cada una. Dos variables solo van a poder cruzarse entre sí una única vez, y por ello retornamos nada más hacer la comprobación.

Esta es la misma comprobación que hacemos en *esConsistenteFC*, pero la única diferencia es que aquí no comprobamos si las dos palabras son iguales, ya que *AC3* no asigna palabras, sino que solo revisa los arcos del grafo de restricciones, que en el crucigrama son los cruces y comprueba que cada palabra de una variable encaje con otra en la letra compartida. Esta regla de no repetir palabras no es una restricción del cruce, sino del

crucigrama entero y también afecta a las palabras que no se cruzan entre ellos, y por ello es backTracking el encargado de evitar que se repitan, con su función *isFeasible* y ForwardChecking con *esConsistenteFC*.

4.2.2 Función AC3

```
def AC3(listaVariables):
    cola = []
    for var_i in listaVariables:
        for restriction in var_i.restricciones:
            cola.append((var_i, restriction[0]))

    while len(cola) > 0:
        var_k, var_m = cola.pop(0)
        cambio = False

        for palabra_k in var_k.dominio.copy():
            hay_apoyo = False
            for palabra_m in var_m.dominio:
                if esConsistenteCruce(var_k, palabra_k, var_m, palabra_m):
                    hay_apoyo = True
                    break
            if not hay_apoyo:
                var_k.dominio.remove(palabra_k)
                cambio = True

        if len(var_k.dominio) == 0:
            return False

        if cambio:
            for restriction in var_k.restricciones:
                var_i = restriction[0]
                if var_i != var_m and (var_i, var_k) not in cola:
                    cola.append((var_i, var_k))
    return True
```

Inicializamos una cola y recorremos todas las variables, y para cada una de las variables recorremos sus restricciones y añadimos a la cola la tupla *(variable, var_vecino)*. Como cada cruce está guardado en las dos variables que se cruzan, la cola contendrá cada arco en los dos sentidos

Seguidamente mientras la cola no esté vacía, desencolamos una tupla (Vk, Vm) y, para cada palabra del dominio Vk recorremos el dominio Vm buscando al menos una palabra que

sea consistente con ella, es decir, que tengan la misma letra en la casilla que se cruzan. A esa palabra la denominamos apoyo, y en cuanto encuentre un apoyo dejamos de buscar, ya que con uno ese suficiente. Si no existe ninguna palabra en el Vm que sea consistente, esa palabra no va a poder formar parte nunca de una solución, por lo que la borramos del dominio de Vk.

Una vez revisadas todas las palabras de Vk, comprobamos si su dominio se ha quedado vacío, si es así, ninguna palabra de Vk puede encajar con Vm, por lo que el problema no tiene solución y la función devuelve *False.*

Si ha habido algún cambio en el dominio de Vk, es decir, si se ha borrado alguna palabra, hay que comprobar si agún vecino de Vk tenía su apoyo en la palabra borrada, por lo que encolamos de nuevo los arcos (Vi, Vk) de todos los vecinos de Vi de Vk, con dos excepciones:

- No encolamos el arco (Vm, Vk), ya que las palabras borradas de Vk no eran compatibles con ninguna palabra de Vm, así que ninguna palabra de Vm dependía de ellas.
- No encolamos un arco que ya esté en la cola, para no revisarlo dos veces innecesariamente.

Finalmente, cuando la cola se queda vacía, todos los arcos son consistentes y la función devuelve *True*. De esta manera los dominios de las variables se quedan reducidos pudiendo trabajar con ellos si se ejecuta BK o FC.

4.2.3 Medición de tiempos

Para poder comparar el rendimiento de los distintos algoritmos, aplicando o sin aplicar AC3, se ha añadido la medición de tiempo con el módulo *time* de la biblioteca estándar de *Python*.

```
import time
```

A partir de este módulo, usamos la función ***time.perf_counter()***, que devuelve el valor de un reloj, que lo ejecutaremos al inicio del algoritmo y al final, y calcularemos la diferencia para sacar el tiempo que ha tardado. Hay que destacar que ***perf_counter()*** tiene una resolución mucho mayor que ***time.time()***, y que por tanto nos permite ver bien la diferencia de tiempo en algoritmos que se diferencian en milésimas de segundos.

Además, el tiempo se mide únicamente durante de la ejecución del algoritmo, sin contar los print, la construcción de variables y sus restricciones y el dibujado de la solución en el tablero.

```
def resuelve(algoritmo, tablero, todasVariables, tiempoAC3=None):
    inicio = time.perf_counter()
    resuelto = algoritmo(0, todasVariables)
    fin = time.perf_counter()
    tiempo = fin - inicio
```

4.2.4 Modificación de la función resuelve

La función resuelve, que ha sido creada en la sesión anterior, se ha modificado para medir el tiempo de ejecución del algoritmo e indicar si se ha aplicado AC3 previamente.

```
def resuelve(algoritmo, tablero, todasVariables, tiempoAC3=None):
    inicio = time.perf_counter()
    resuelto = algoritmo(0, todasVariables)
    fin = time.perf_counter()
    tiempo = fin - inicio
    nombre = algoritmo.__name__

    if tiempoAC3 is None:
        print(f"{nombre}: {tiempo * 1e6:.0f} µs")
    else:
        print(f"AC3+{nombre}: {(tiempo + tiempoAC3) * 1e6:.0f} µs (AC3: {tiempoAC3 * 1e6:.0f} µs
+ {nombre}: {tiempo * 1e6:.0f} µs)")

    if resuelto:
        pintarSolucion(todasVariables, tablero)
    else:
        MessageBox.showwarning("Alerta", "No hay solución")
```

Respecto a la versión anterior hay tres cambios:

- **Nuevo parámetro *tiempoAC3***, que indica cuánto ha tardado *AC3* si este ha sido ejecutado previamente, que por defecto vale *None (sí AC3 no se ha ejecutado).*
- **Medición del tiempo** mediante las llamadas a *time.perf_counter().*
- **Nombre del algoritmo** pasado por parámetro.

De esta manera se ejecuta el algoritmo y se mide su tiempo y se muestra en consola de las

siguientes maneras:

- Si no se ha ejecutado AC3 (tiempoAC3 es None), se muestra solo el tiempo del algoritmo:

```
FC: 1213 µs
```

- Si se ha ejecutado AC3 antes, se muestra el tiempo total, y entre paréntesis lo que ha tardado cada parte:

```
 AC3+FC: 602 µs  (AC3: 486 µs + FC: 116 µs)
```

Con esta información vamos a poder analizar si AC3 compensa, ya que aunque vaya a reducir el tiempo de búsqueda, este tiene un coste propio y hay que compararlo con el del algoritmo sin usar AC3.

4.2.5 Botón AC3 y variable tiempo AC3

```
elif pulsaBoton(pos, botAC3):
                    if todasVariables is None:
                        todasVariables = obtenerVariables(tablero, almacen)
                    print("DOMINIOS ANTES DE AC3:")
                    for v in todasVariables:
                        print(v)
                    inicio = time.perf_counter()
                    resultado = AC3(todasVariables)
                    fin = time.perf_counter()

                    if tiempoAC3 is None:
                        tiempoAC3 = 0
                    tiempoAC3 = tiempoAC3 + (fin - inicio)
                    print(f"AC3: {(fin - inicio) * 1e6:.0f} µs")
                    if resultado:
                        print("DOMINIOS DESPUES DEL AC3")
                        for v in todasVariables:
                            print(v)
                    else:
                        MessageBox.showwarning("Alerta", "No hay solución")
```

Al pulsar el botón AC3 se muestran por consola los dominios de las variables antes y después de ejecutar el algoritmo, de manera que si AC3 devuelve *False* algún dominio se

ha quedado vacío y avisa de que no hay solución.

Como el botón trabaja sobre *todasVariables* y no la reinicia, si pulsamos *BK* o *FC,* estos van a recibir los dominios ya reducidos por *AC3*.

Además como AC3 y el algoritmo de búsqueda se activan con clicks distintos, guardamos el tiempo que tarda *AC3* en la variable *tiempoAC3*, que se le pasa posteriormente a la función *Resuelve* para mostrar el tiempo total.

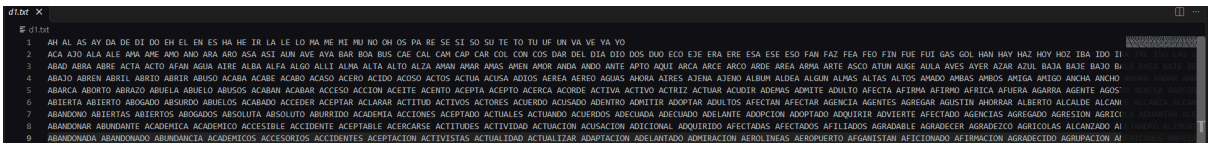
4.3 Ejecución del crucigrama

Para analizar el comportamiento de los distintos algoritmos, realizaremos cuatro ejecuciones del mismo algoritmo en tres crucigramas distintos, y cada una de las ejecuciones será usando las semillas 1,2,3 y 5. Como los dominios se guardan en sets, y Python cambia en cada ejecución el orden en el que los recorre, porque aleatoriza el hash de las cadenas, las semillas nos servirá para establecer ese mismo orden en una ejecución, y de esta manera conseguimos analizar para todos los algoritmos, el tiempo de resolución con un conjunto ordenado de la misma manera. Para establecer una semilla en Python usaremos el siguiente comando antes de ejecutar el programa:

```
$env:PYTHONHASHSEED=1
```

4.3.1 Creación de un diccionario

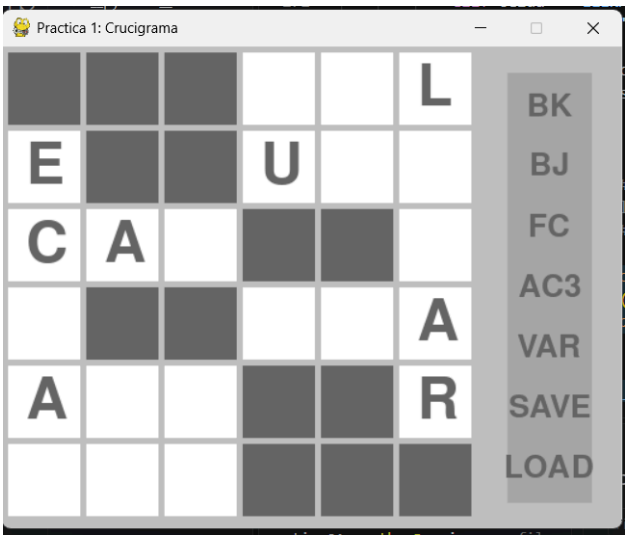
Debido a que vamos a monitorizar los tiempos de ejecución de los distintos algoritmos, es necesario empezar a utilizar tamaños de crucigramas más grandes, y por ello se ha creado un nuevo diccionario con una cantidad de **6026 palabras**, con palabras desde 2 hasta 10 letras.



Con esto conseguimos que el algoritmo pueda ejecutarse en un tiempo que sea coherente analizar y así poder discutir sobre los resultados.

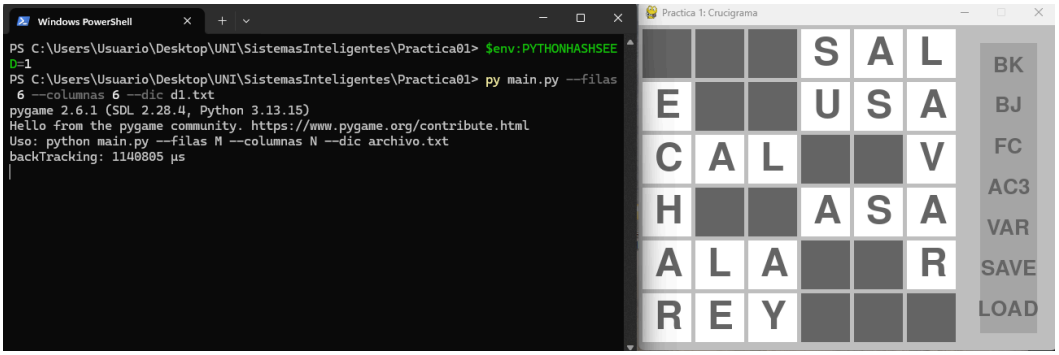
4.3.2 Tablero 6x6

Para el tablero de 6x6, el crucigrama a resolver será el siguiente:

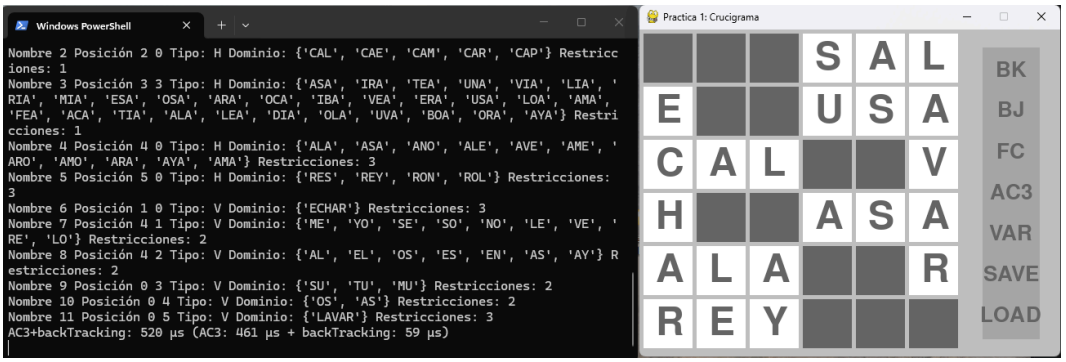


SEMILLA 1

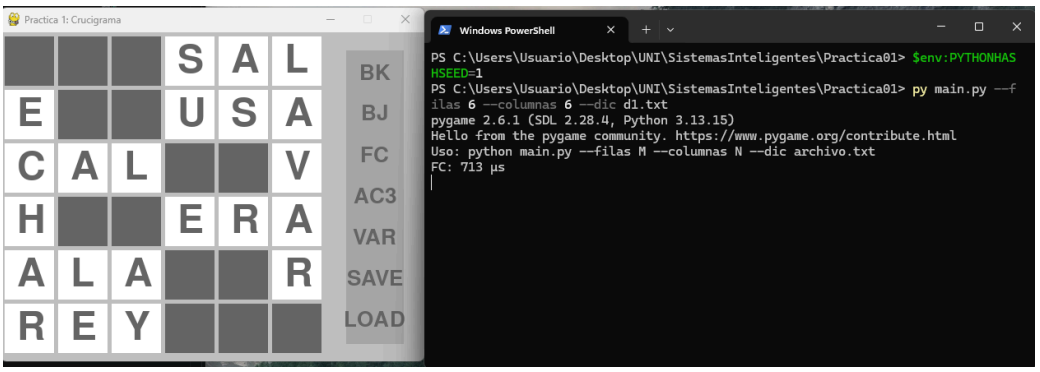
- **BackTracking - 1140805 µs**



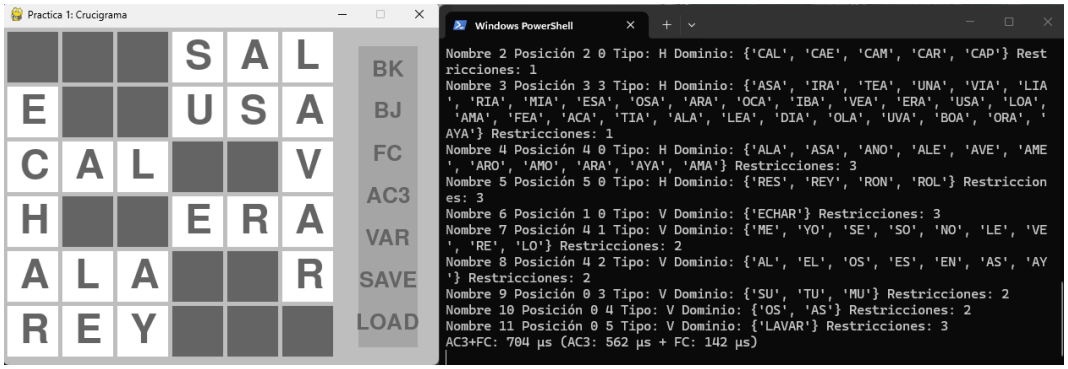
- **AC3 + BackTracking - 520 µs (461 µs + 59 µs)**



- **ForwardChecking - 713 µs**

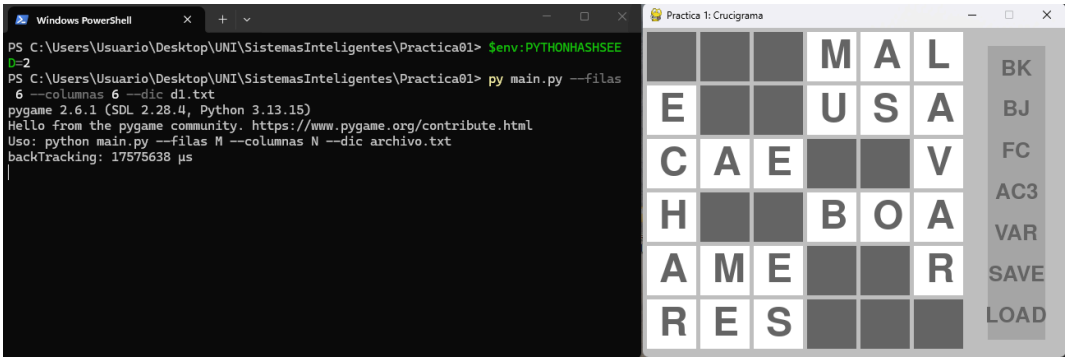


- **AC3 + ForwardChecking - 704 µs(562 µs + 142 µs)**

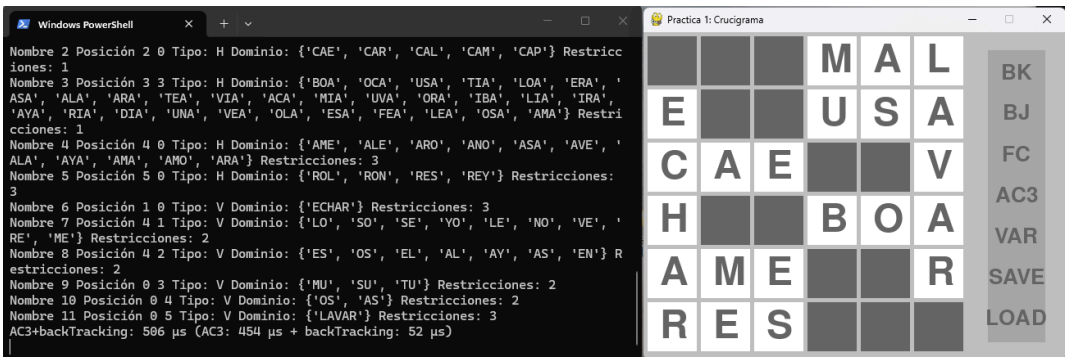


SEMILLA 2

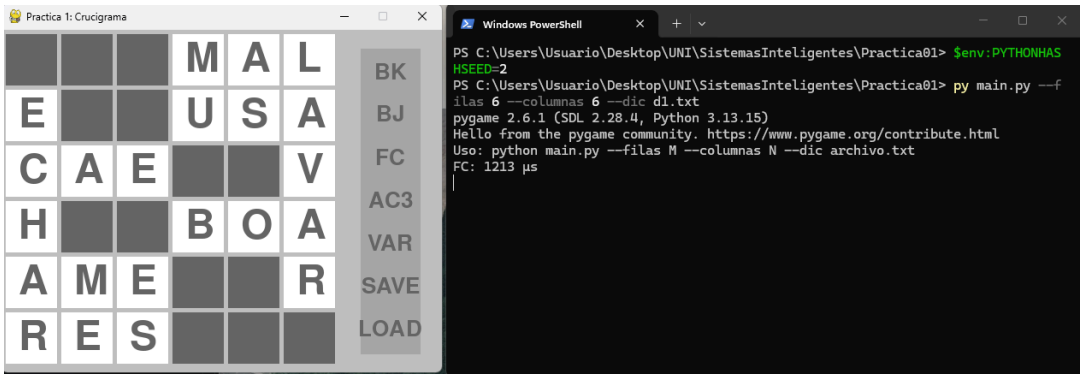
- **BackTracking - 17575638 µs**



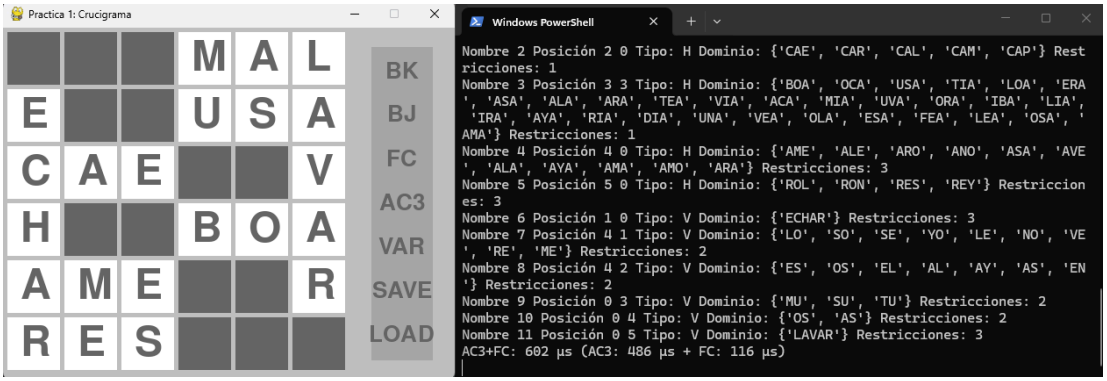
- **AC3 + BackTracking - 506 µs (454 µs + 52 µs)**



- **ForwardChecking - 1213 µs**

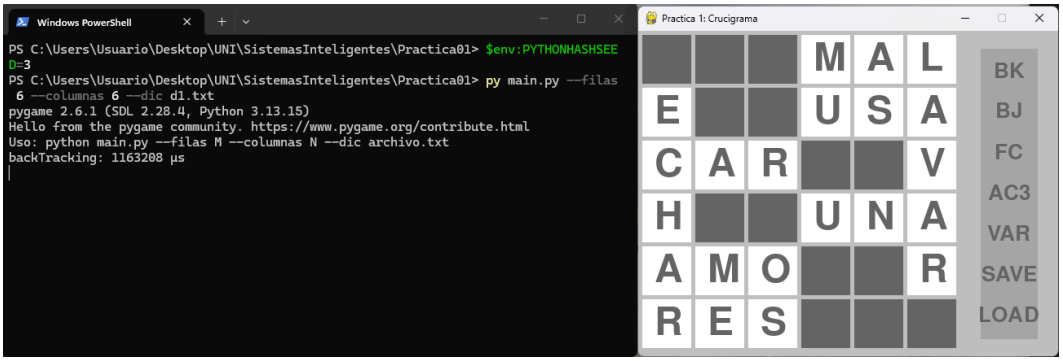


- **AC3 + ForwardChecking - 602 µs(486 µs + 116 µs)**

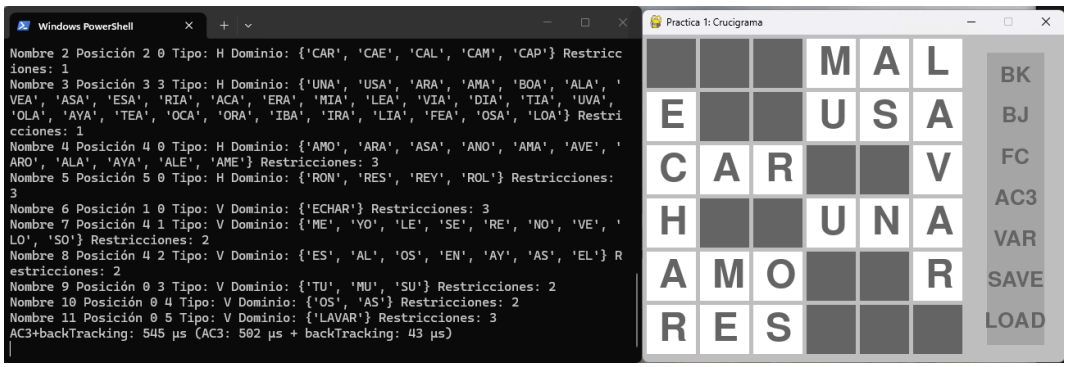


SEMILLA 3

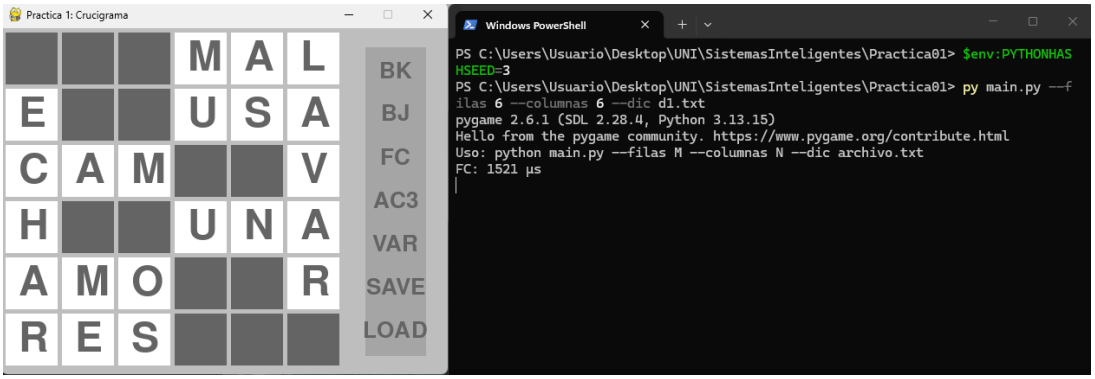
- **BackTracking - 1163208 µs**



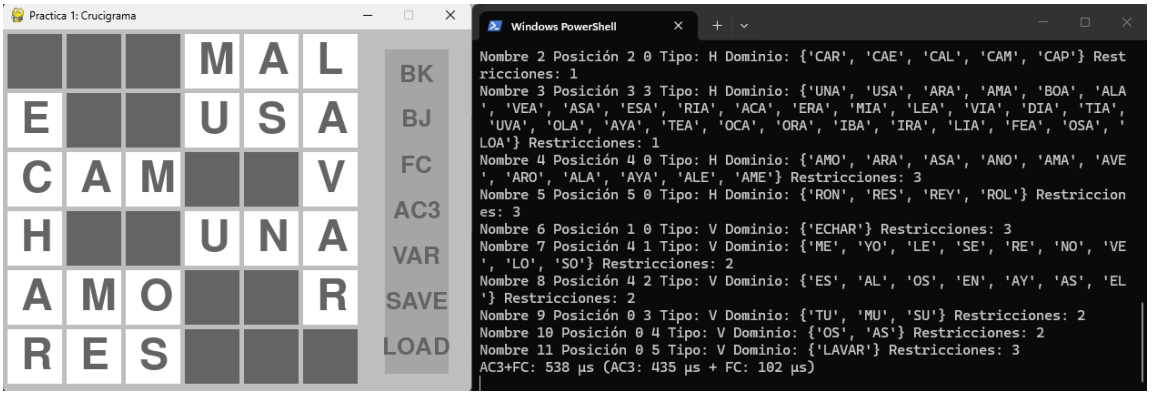
- **AC3 + BackTracking - 545 µs (502 µs + 43 µs)**



- **ForwardChecking - 1521 µs**

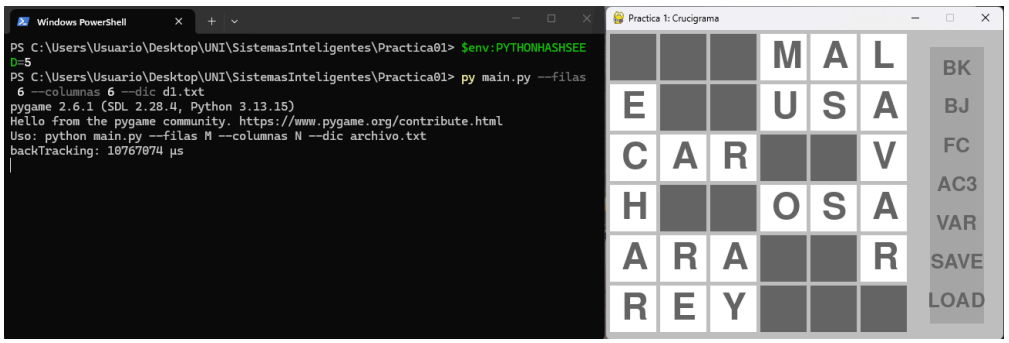


- **AC3 + ForwardChecking - 538 µs(435 µs + 102 µs)**

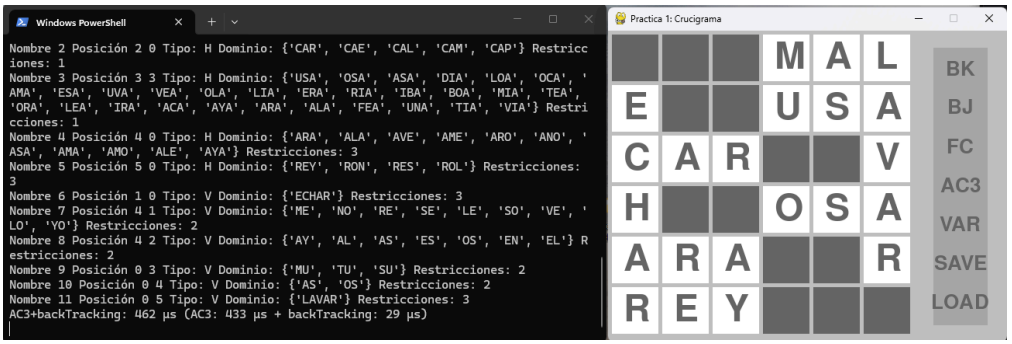


SEMILLA 5

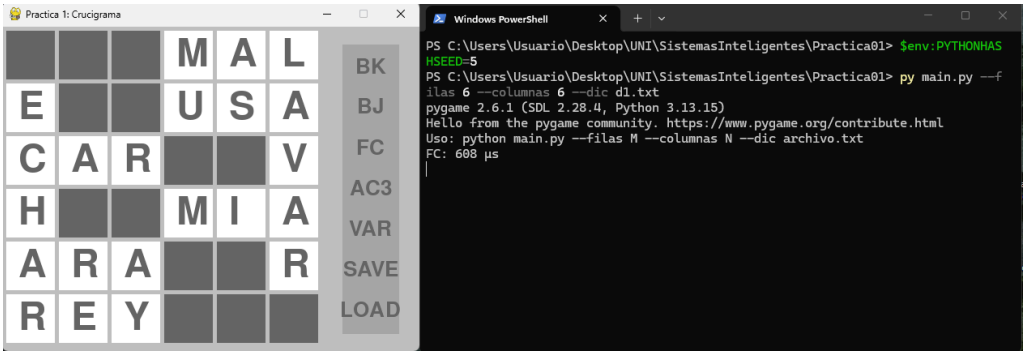
- **BackTracking - 10767074 µs**



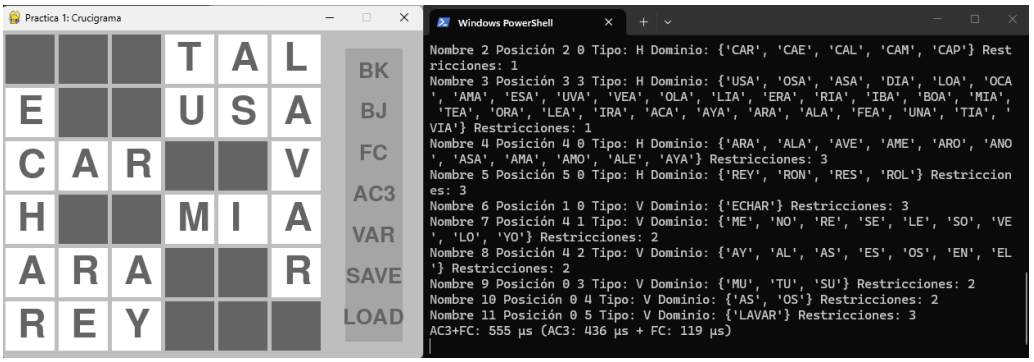
- **AC3 + BackTracking - 462 µs (433 µs + 29 µs)**



- **ForwardChecking - 608 µs**

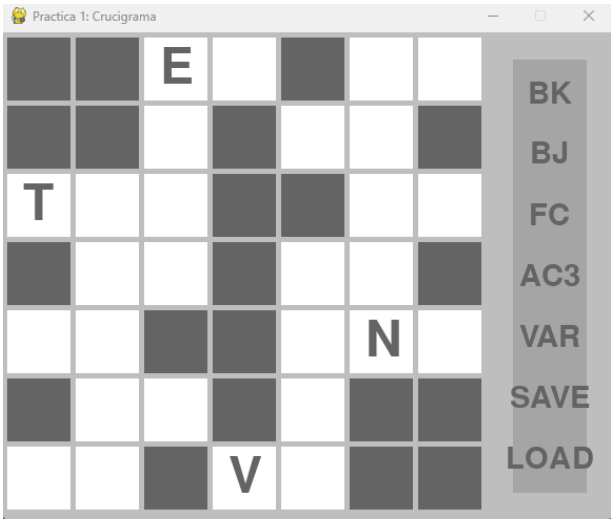


- **AC3 + ForwardChecking - 555 µs(436 µs + 119 µs)**



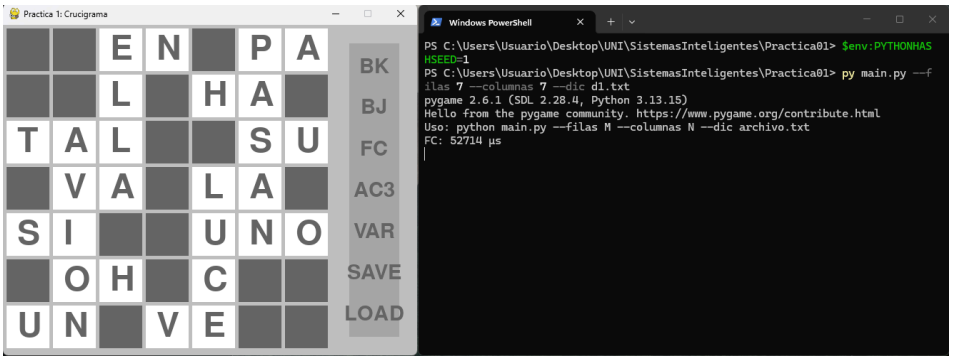
4.3.3 Tablero 7x7

Para el tablero de 7x7, el crucigrama a resolver será el siguiente:

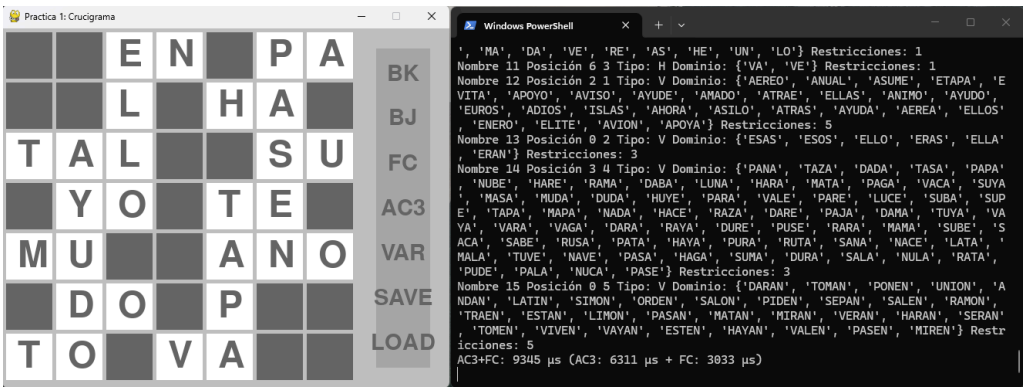


SEMILLA 1

- **BackTracking > 60s**
- **AC3 + BackTracking > 60S**
- **ForwardChecking - 52714 µs**

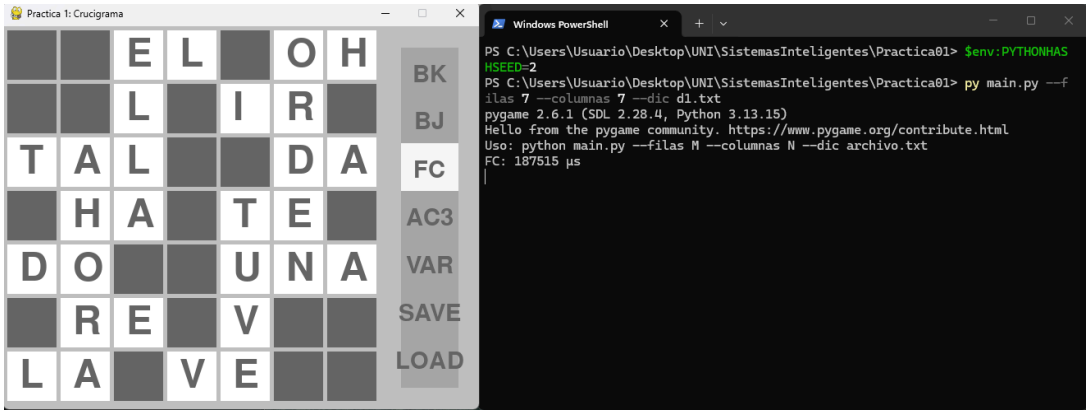


- **AC3 + ForwardChecking - 9345 µs(6311 µs + 3033 µs)**

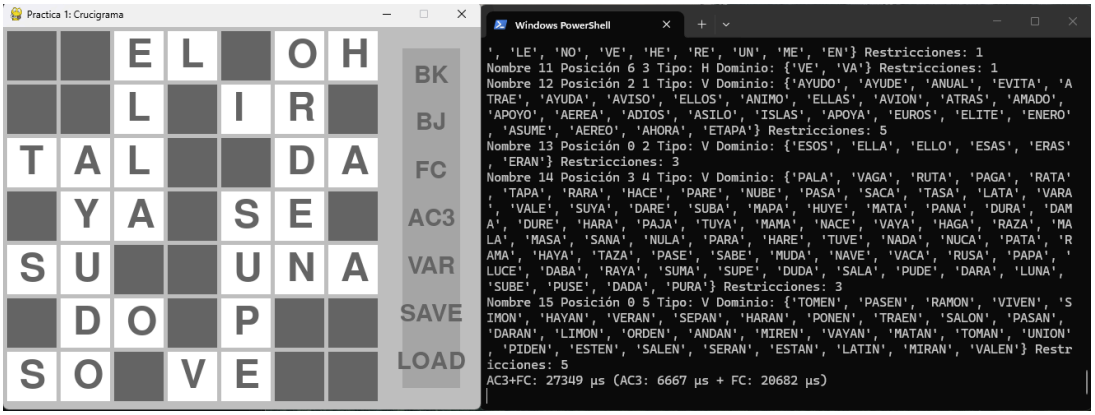


SEMILLA 2

- **BackTracking > 60s**
- **AC3 + BackTracking > 60S**
- **ForwardChecking - 187515 µs**

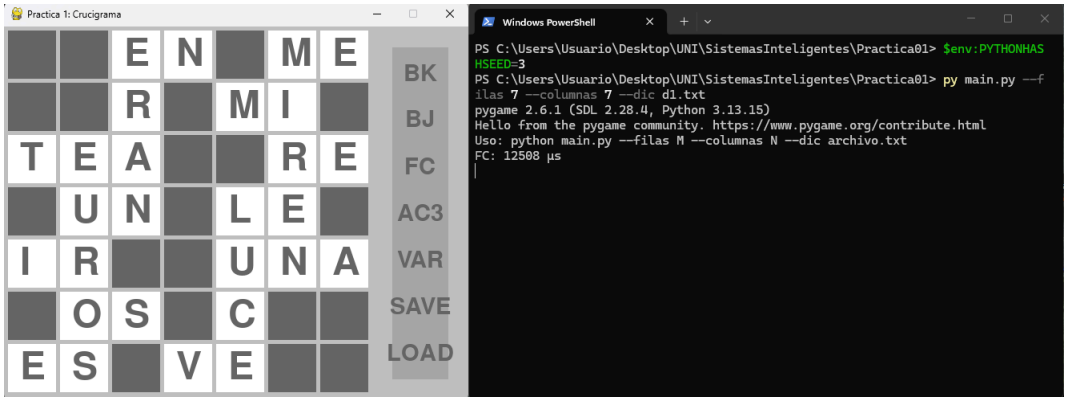


- **AC3 + ForwardChecking - 27349 µs(6667 µs + 20682 µs)**

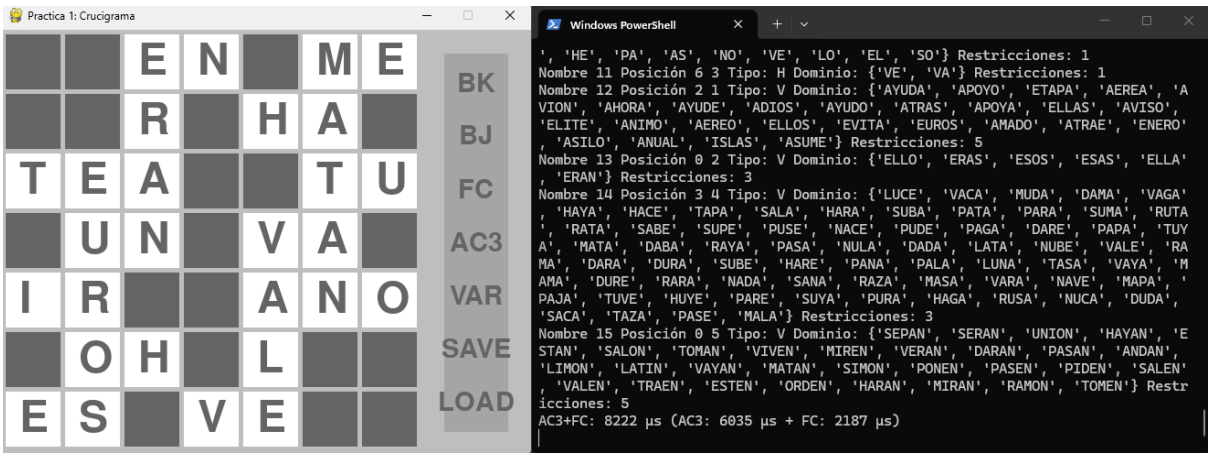


SEMILLA 3

- **BackTracking > 60s**
- **AC3 + BackTracking > 60S**
- **ForwardChecking - 12508 µs**

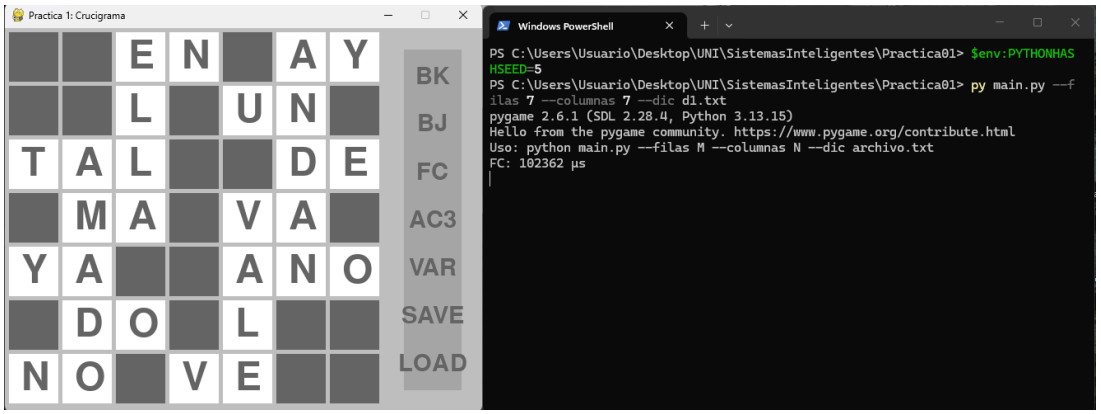


- **AC3 + ForwardChecking - 8222 µs(6035 µs + 2187 µs)**

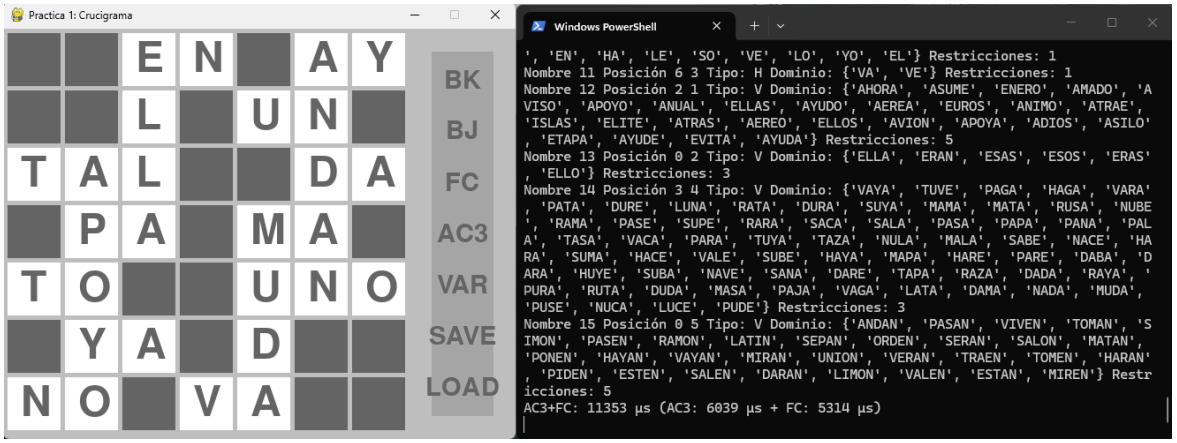


SEMILLA 5

- **BackTracking > 60s**
- **AC3 + BackTracking > 60S**
- **ForwardChecking - 102362 µs**

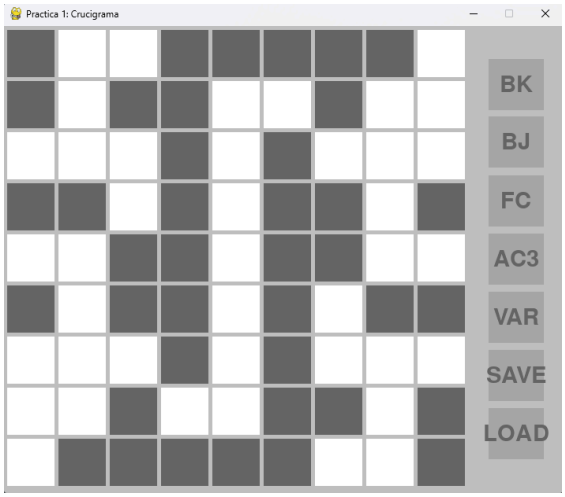


- **AC3 + ForwardChecking - 11353 µs(6039 µs + 5314 µs)**



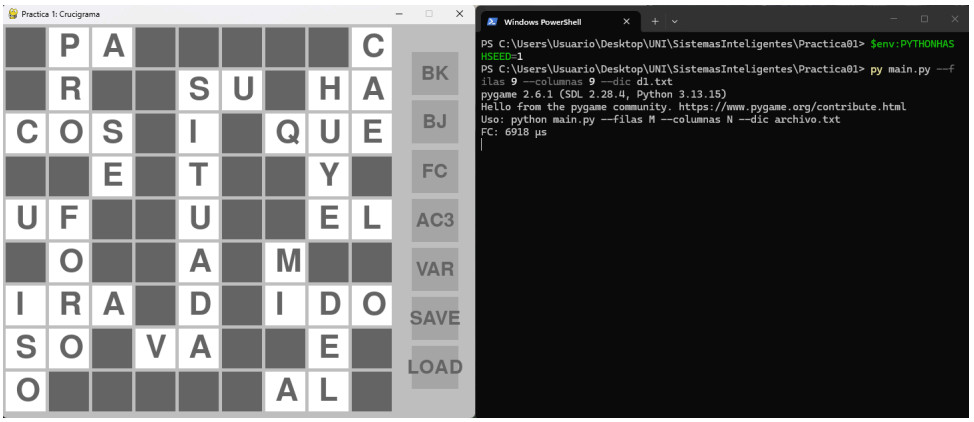
4.3.4 Tablero 9x9

Para el tablero de 9x9, el crucigrama a resolver será el siguiente:

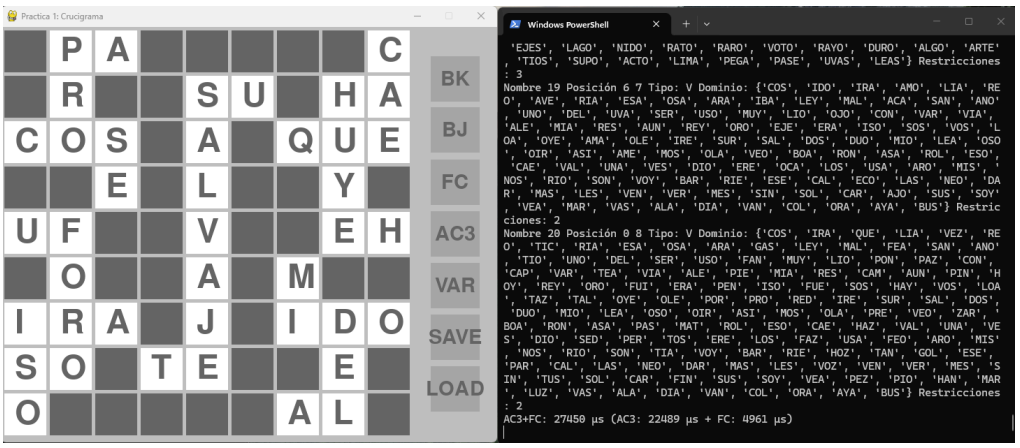


SEMILLA 1

- **BackTracking > 60s**
- **AC3 + BackTracking > 60S**
- **ForwardChecking - 6918 µs**

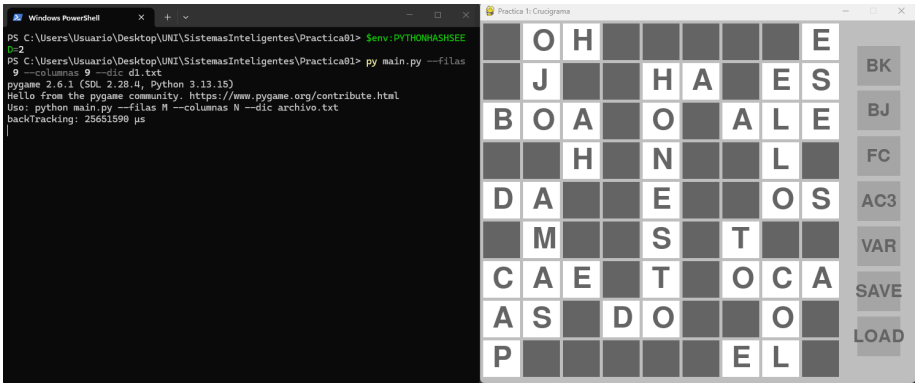


- **AC3 + ForwardChecking - 27450 µs(22489 µs + 4961 µs)**

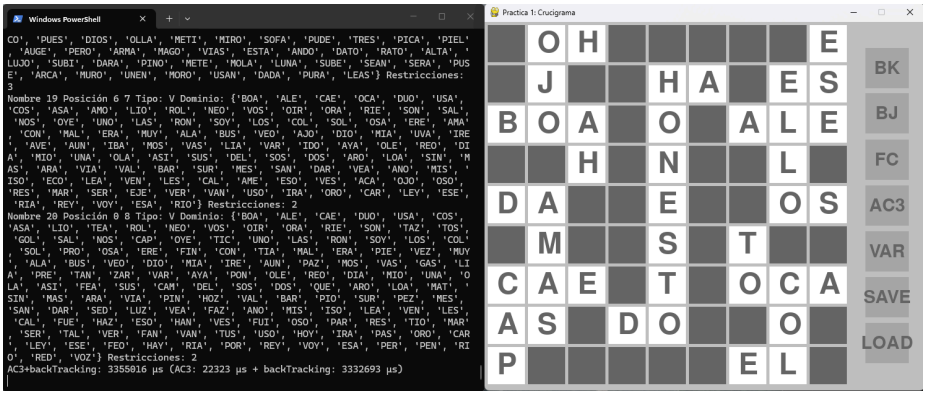


SEMILLA 2

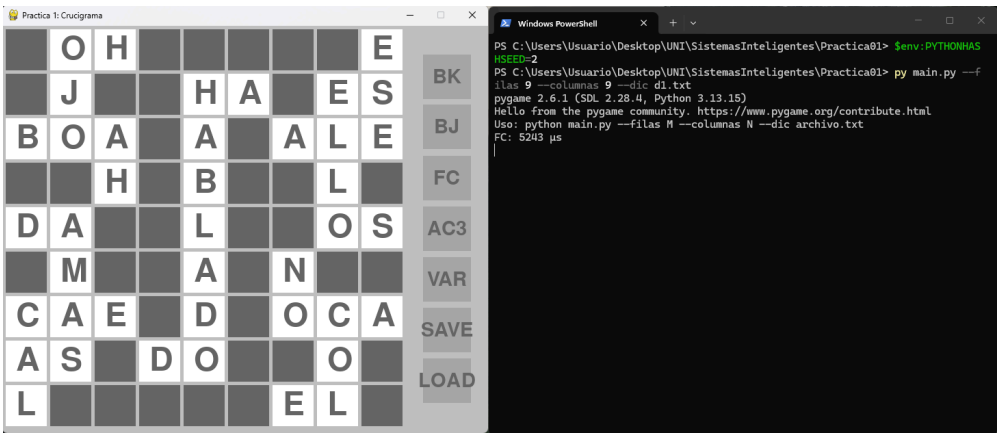
- **BackTracking - 25651590 µs**



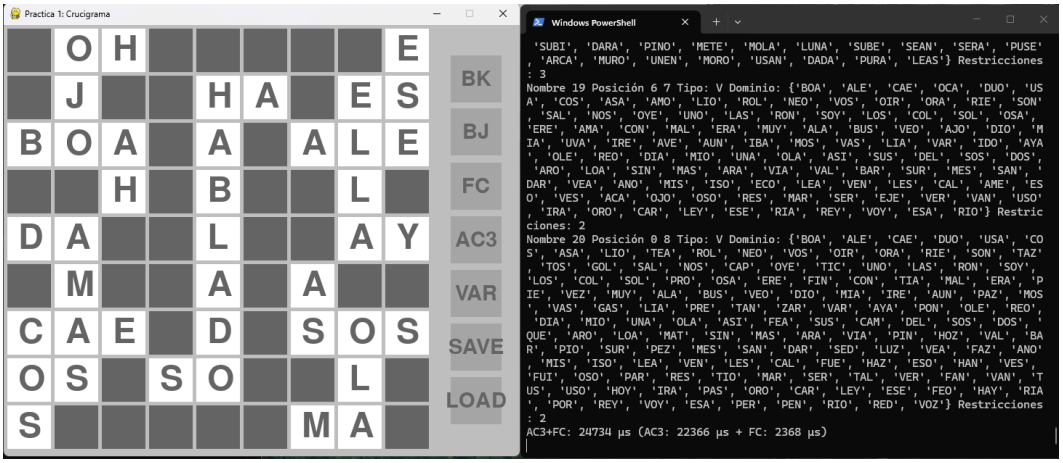
- **AC3 + BackTracking - 3355016 µs (22323 µs + 3332693 µs)**



- **ForwardChecking - 5243 µs**

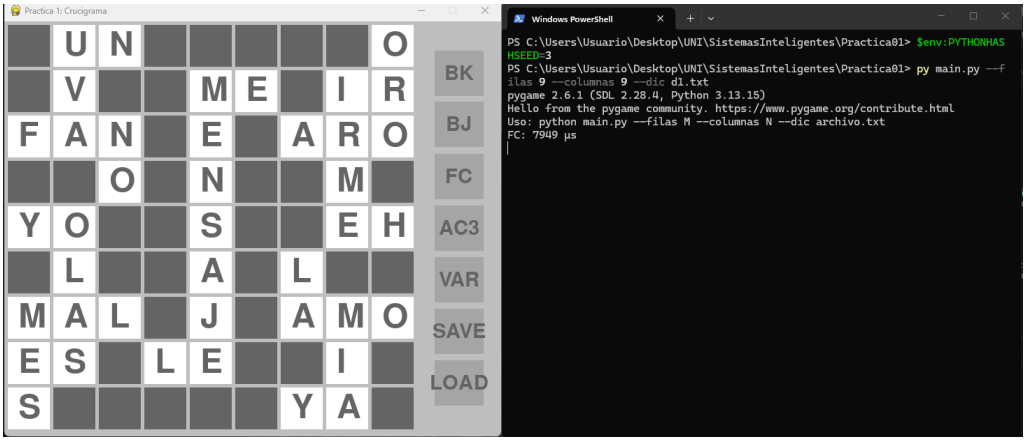


- **AC3 + ForwardChecking - 24734 µs(22366 µs + 2368 µs)**

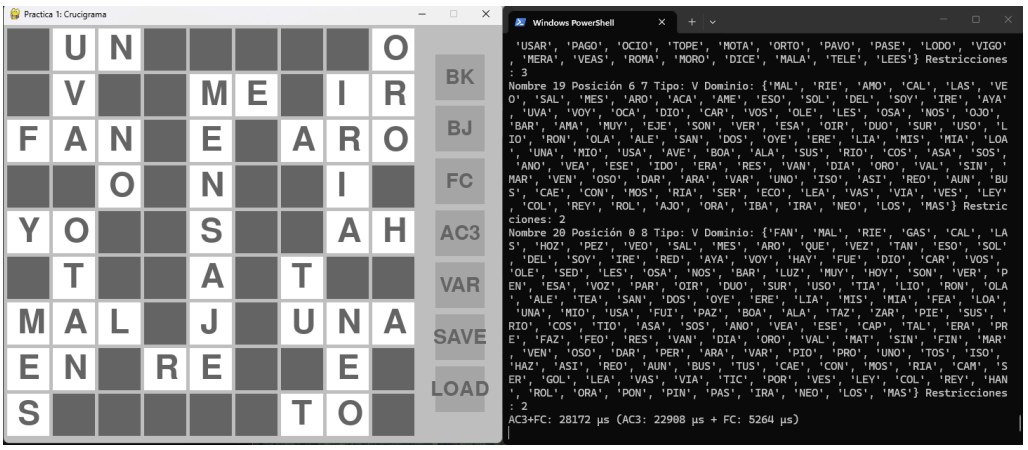


SEMILLA 3

- **BackTracking > 60s**
- **AC3 + Backtracking > 60S**
- **ForwardChecking - 7949 µs**

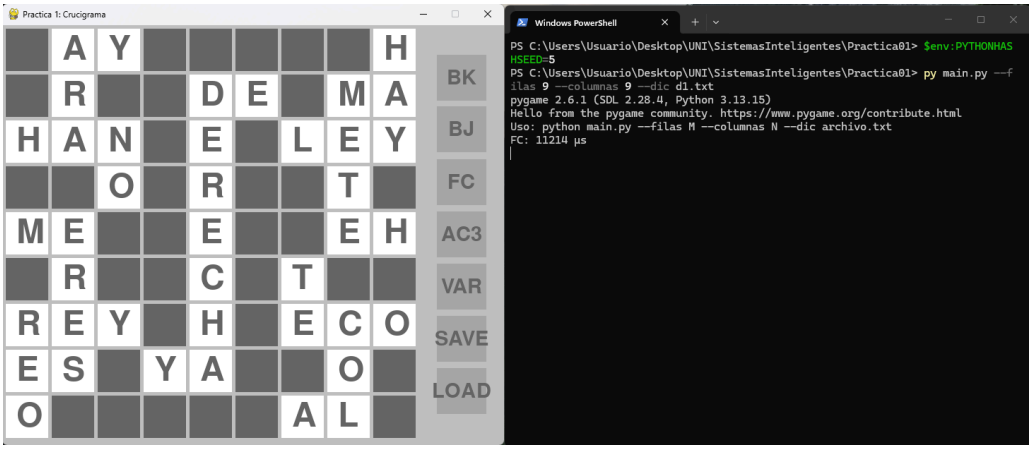


- **AC3 + ForwardChecking - 28172 µs(22908 µs + 5264 µs)**

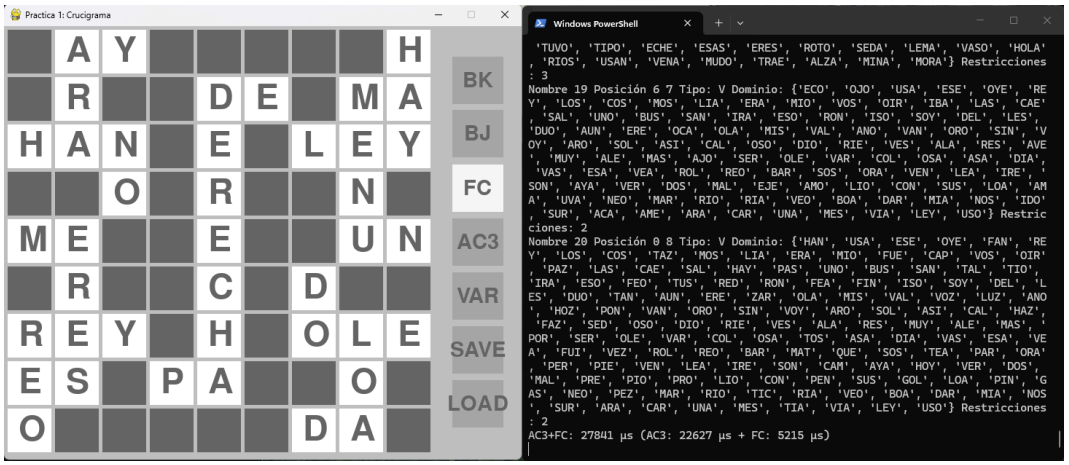


SEMILLA 5

- **BackTracking > 60s**
- **AC3 + BackTracking > 60S**
- **ForwardChecking - 11214 µs**



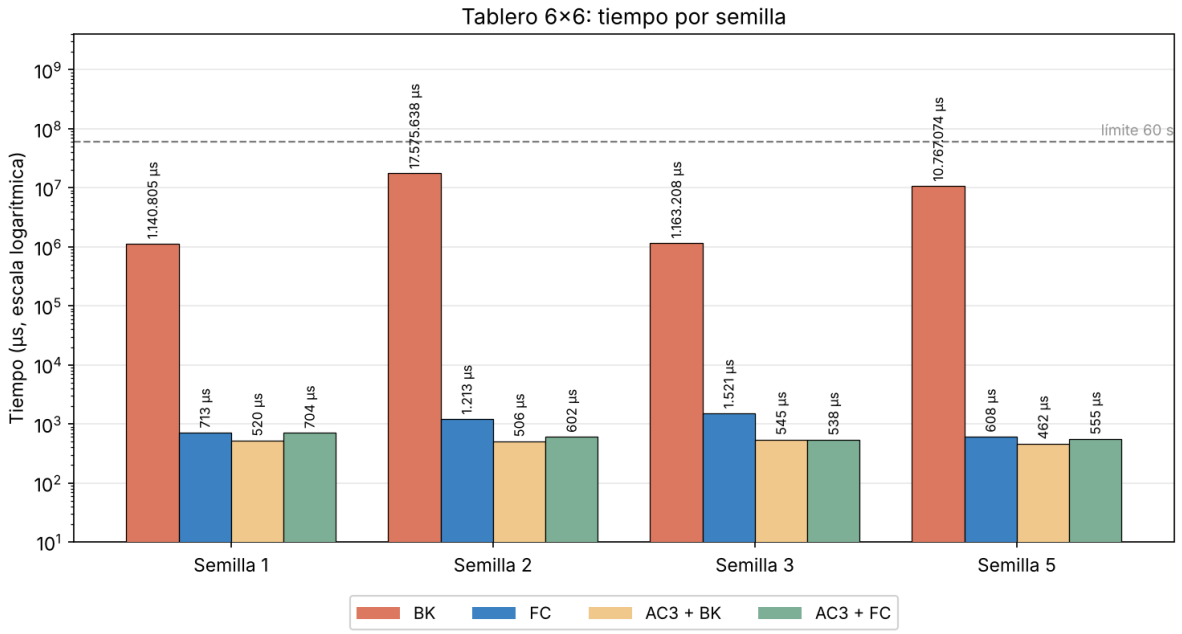
- **AC3 + ForwardChecking - 27841 µs(22627 µs + 5215 µs)**



4.3.5 Análisis de los resultados obtenidos

Tablero 6x6

Algoritmo	Semilla 1 - µs	Semilla 2 - µs	Semilla 3 - µs	Semilla 5 - µs	Media - µs
BK	1.140.805	17.575.638	1.163.208	10.767.074	7.661.681
FC	713	1.213	1.521	608	1.014
AC3+BK	520 (AC3 461)	506 (AC3 454)	545 (AC3 502)	462 (AC3 433)	508
AC3+FC	704 (AC3 562)	602 (AC3 486)	538 (AC3 435)	555 (AC3 436)	600



Como se puede observar, BackTracking es con diferencia el algoritmo más lento, con tiempos de entre 1,1 y 17,6 segundos dependiendo de la semilla, y esto se debe a que BackTracking solo comprueba cada palabra con las variables ya asignadas, por lo que no detecta los conflictos hasta que llega a la variable que se cruza con ella, haciendo que por el camino explore muchísimas combinaciones que no van a llevar a ninguna solución. Además hay que tener en cuenta que su tiempo varía muchísimo, ya que depende del orden que se prueban las palabras.

Si decidimos aplicar antes AC3 podemos observar que el tiempo baja drásticamente tardando unos 500 µs, siendo unas 15.000 veces más rápido. Esto ocurre debido a que varias variables se quedan con una única palabra posible, como *ECHAR* y *LAVAR*, y el

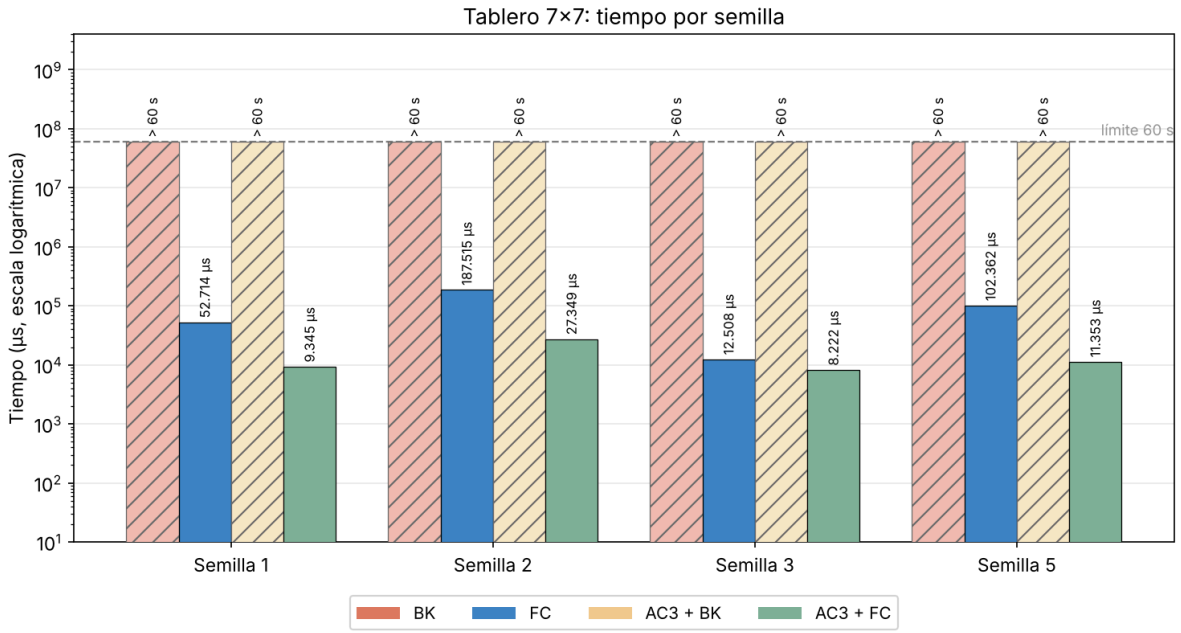
resto que son compatibles con estas son muy pocas, haciendo que BackTracking se ejecute con el problema ya casi resuelto.
En cuanto a ForwardChecking podemos observar que ya de por sí solo es muy rápido, tardando entre 600 µs y 1.500 µs, gracias a que cada asignación poda los dominios de las variables futuras, descartando rápidamente las ramas que van a fallar y evitando que haga búsquedas innecesarias.

Si aplicamos AC3 a Forward Checking podemos observar que el tiempo baja, aunque ya no tan drásticamente, ya que si podemos observar que los resultados entre las distintas semillas es entre 538 µs y 704 µs, con una media de 600 µs, siendo unas 1,7 veces más rápido que ForwardChecking sin aplicar AC3.

En este tablero, AC3 + BK y AC3 + FC obtienen tiempos casi idénticos ya que en los dos casos el tiempo está dominado por AC3, y la búsqueda que se realiza posteriormente es casi inmediata.

Tablero 7x7

Algoritmo	Semilla 1 - µs	Semilla 2 - µs	Semilla 3 - µs	Semilla 5 - µs	Media - µs
BK	> 60 s	> 60 s	> 60 s	> 60 s	-
FC	52.714	187.515	12.508	102.362	88.775
AC3+BK	> 60 s	> 60 s	> 60 s	> 60 s	-
AC3+FC	9.345 (AC3 6.311)	27.349 (AC3 6.667)	8.222 (AC3 6.035)	11.353 (AC3 6.039)	14.067

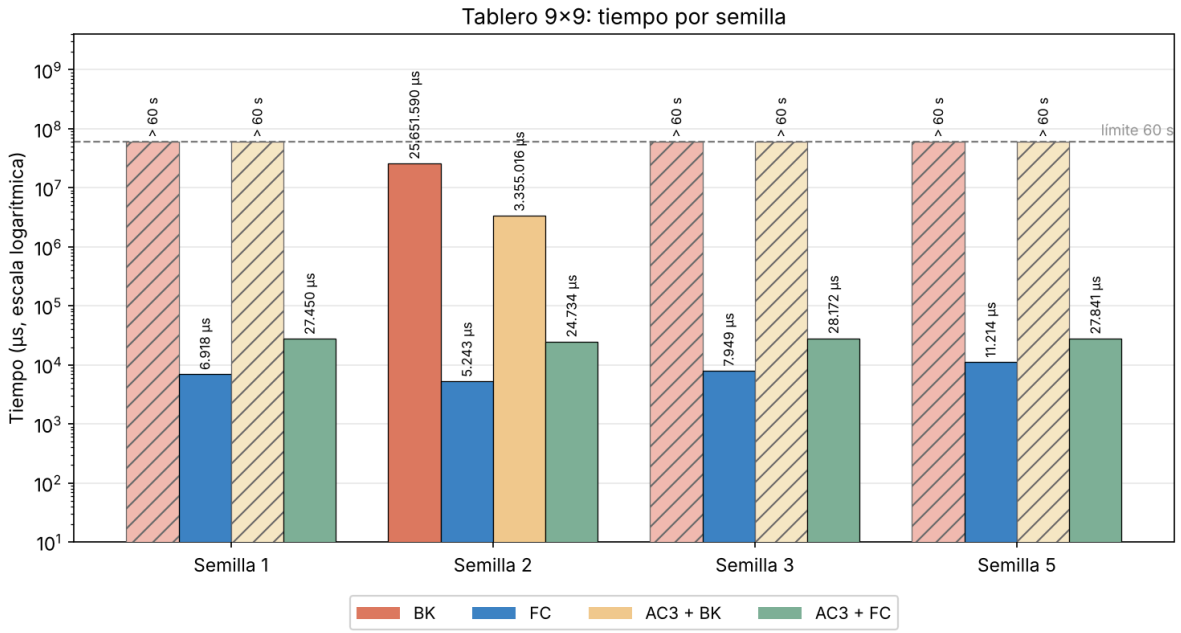


En este tablero BackTracking ya no consigue terminar en menos de 60 segundos, ni siquiera aplicando previamente AC3, y esto se debe a que como los dominios son muchísimo más grandes, el número de combinaciones va creciendo exponencialmente.

En cuanto a ForwardChecking sí es capaz de resolverlo, pero su tiempo varía bastante dependiendo de la semilla dando un resultado de entre 12.500 µs y 187.500 µs. Si decidimos aplicar AC3 previamente el tiempo se reduce significativamente, logrando un resultado entre 8.200 µs y 27.300 µs. Analizando las medias de cada uno, podemos deducir que aplicar AC3 previamente hace que el crucigrama se resuelva unas 6 veces más rápido.

Tablero 9x9

Algoritmo	Semilla 1 - µs	Semilla 2 - µs	Semilla 3 - µs	Semilla 5 - µs	Media - µs
BK	> 60 s	25.651.590	> 60 s	> 60 s	-
FC	6.918	5.243	7.949	11.214	7.831
AC3+BK	> 60 s	3.355.016 (AC3 22.323)	> 60 s	> 60 s	-
AC3+FC	27.450 (AC3 22.489)	24.734 (AC3 22.366)	28.172 (AC3 22.908)	27.841 (AC3 22.627)	27.049



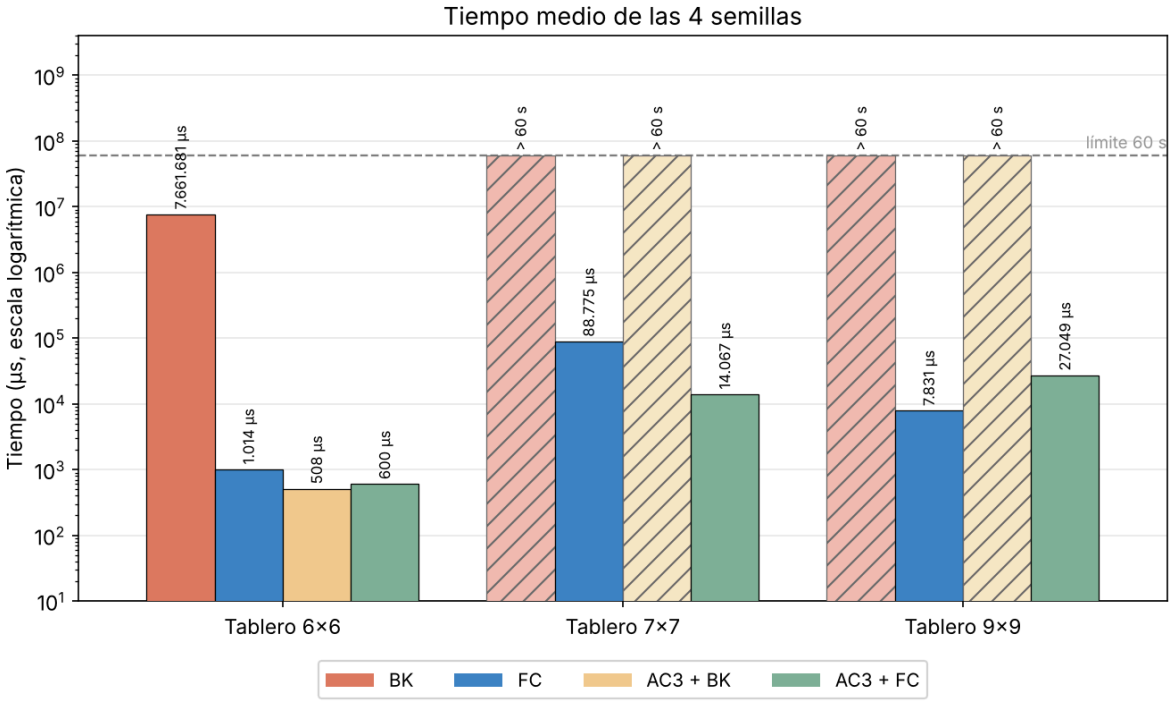
Este tablero es el más interesante de los tres, ya que a medida que hemos ido realizando las ejecuciones, estábamos viendo que combinar AC3 con cualquiera de los algoritmos implementados (*BK y FC*), solo mejoraba el resultado. Sin embargo al ejecutar este tablero podemos observar que no siempre es así.

En todas las semillas, Forward Checking por sí solo ha resuelto el problema más rápido que combinándolo con AC3, tardando entre 5.200 µs y 11.200 µs frente a los entre 24.700 µs y 28.200 µs de AC3 + FC, unas 3,5 veces más de media. Y esto se debe principalmente al coste de AC3. En este tablero AC3 tarda alrededor de 22.600 µs, ya que es el que tiene más variables y cruces, y además no tiene ninguna letra fija, siendo los dominios mucho más grandes. En todas las semillas, la búsqueda de Forward Checking tras haber aplicado AC3 es incluso más rápida que sin él (entre 2.400 µs y 5.300 µs frente a entre 5.200 µs y 11.200 µs), pero el tiempo que tarda AC3 hace que el total acabe siendo mayor.

Por lo que sabiendo esto, podemos concluir que AC3 reduce la búsqueda posterior, pero añade un coste fijo, en el caso de este tablero de unos 22.600 µs, que solo compensa cuando la búsqueda va a ser costosa. En este tablero Forward Checking ya es rápido de por sí, por lo que el coste de AC3 hace que el total sea mayor.

En cuanto a BackTracking, podemos observar que en este tablero ha conseguido solo una solución por debajo de los 60 segundos, concretamente en la semilla 2 con un tiempo de 25,65 segundos, y que aplicando AC3 previamente, el problema se puede ejecutar en tan solo 3,36 segundos, unas 7,6 veces más rápido. Sin embargo este tiempo sigue siendo mucho mayor a comparación de ForwardChecking.

Conclusión



Comparando los tres tablero, podemos extraer las siguientes conclusiones:

- **BackTracking como límite exponencial**: a medida que va creciendo los dominios de las variables, el tiempo de ejecución de este algoritmo para resolver el problema va creciendo exponencialmente, ya que si no tiene suerte en la rama que quiere recorrer, su tiempo de ejecución va a ser muy alto.
- **Forward Checking es mucho más rápido que BackTracking** en todos los tableros, gracias a que poda las variables futuras y evita recorrer innecesariamente muchas ramas, aunque no se salva de poder elegir un mal camino y tener que retroceder.
- El coste de **AC3 crece con el tamaño del problema** y compensa cuando la búsqueda va a ser muy costosa, pero cuando la búsqueda ya es rápida de por si, puede hacer que el extra de ejecutar AC3 acabe derivando en que el propio ForwardChecking sea más rápido que con AC3.

5. Sesión 7 - Implementación del algoritmo BackJumping

5.1 Contexto

En esta sesión se va a implementar el algoritmo de BackJumping, que es una mejora sobre BackTracking.

Esta mejora funciona ya que BackTracking cuando se le terminan las palabras de una variable que está en el *nivel n*, siempre retrocede justo a la variable anterior, la del *nivel n-1*, para probar otro valor. El problema está que el conflicto puede ser que no la haya causado esta variable, y que el verdadero culpable sea una variable asignada varios niveles antes, como por ejemplo en el *nivel n-3.*

En este caso BackTracking empieza probando todas las palabras del *nivel n-1*, y con cada una de ellas vuelve a bajar al *nivel n*, que sigue fallando ya que la causa del conflicto como hemos comentado en este ejemplo, está en el *nivel n-3.* Por lo que cuando agota todas las palabras del *nivel n-1*, retrocede al *nivel n-2*, cambia su palabra, y de nuevo vuelve a probar todas las combinaciones de *n-1*, y *n.*

De esta manera BackTracking está ocasionando que se hagan una enorme cantidad de combinaciones innecesarias, y para evitarlo BackJumping salta directamente al nivel del culpable. Para conseguirlo, cada variable va a almacenar un conjunto de *conflictos*, que guarda qué variables anteriores le han impedido usar alguna palabra de su dominio, y cuando una variable se quede sin palabras, en vez de retroceder al *nivel n-1*, saltaremos a la variable más reciente del conjunto *conflictos*, en este caso la del *nivel n-3*, sin probar ninguna combinación de los niveles intermedios.

5.2 Modificación de variable.py

Para poder guardar el conjunto de conflictos de cada variable, se ha añadido un nuevo atributo a la clase *Variable*:

```
self.conflictos = set()
```

Es necesario usar un ***set()*** en lugar de una lista porque una misma variable puede ser culpable varias veces, por ejemplo con distintas palabras, y así no se repite. En el conjunto guardaremos los índices de las variables de *listaVariables*, permitiendo saber el culpable más reciente.

5.3 Reorganización del proyecto

Hasta esta sesión, todos los algoritmos estaban guardados en ***main.py***, junto a la interfaz gráfica y la construcción de las variables, sin embargo para antes de implementar el código de BackJumping, hemos migrado el código de los algoritmos a un nuevo archivo fuente del proyecto llamado ***algorithms.py***, para evitar concentrar todo el código en un solo fichero, y reducir la cantidad de líneas de código del *main.py*.

Las funciones migradas han sido las siguientes:

- *isFeasible y backTracking*
- *esConsistenteFC, forward, restaurar* y *FC*
- *esConsistenteCruce* y *AC3*

Y en el *main.py* hemos añadido la siguiente línea:

```
from algorithms import *
```

5.4 Modificación de algorithms.py

5.4.1 Función conflictosCon

```
def conflictosCon(variable, palabra, k, listaVariables):
    conflictos = set()

    for j in range(k):
        var_j = listaVariables[j]

        # Misma palabra ya usada por una variable anterior
        if var_j.valor == palabra:
            conflictos.add(j)

        # Letra distinta en el cruce con una variable anterior
        for restriction in variable.restricciones:
            if restriction[0] == var_j:
                if palabra[restriction[1]] != var_j.valor[restriction[2]]:
                    conflictos.add(j)

    return conflictos
```

Esta función auxiliar recorre las variables anteriores a Vk (desde 0 hasta k-1), que son las que tienen una palabra asignada, y para cada una de ellas comprueba dos cosas:

1. Si una variable anterior está usando la misma palabra
2. Si una variable anterior se cruza con Vk y la letra de la casilla compartida no coincide.

Si alguna variable incumple estas dos condiciones, su índice se añade al conjunto *conflictos*, que es lo que se devolverá en la función.

La única diferencia de esta función frente a *isFeasible*, es que *isFeasible* devuelve *True* o *False* y se detiene en el primer problema que encuentra, en cambio, *conflictosCon* revisa todas las variables anteriores y devuelve cuales son las culpables, ya que BackJumping no necesita saber si es válida o no la palabra, si no a qué palabra saltar. De esta manera:

- Si el conjunto está vacío, la asignación es válida.
- Si no, esas variables son las responsables de que la palabra no se pueda usar.

5.4.2 Función backJumpingRec

```
def backJumpingRec(k, listaVariables):
    if k == len(listaVariables):
        return True, set()

    actualVar = listaVariables[k]
    actualVar.conflictos = set()

    for valor in actualVar.dominio:
        actualVar.valor = valor
        conflictosValor = conflictosCon(actualVar, valor, k, listaVariables)

        if len(conflictosValor) == 0:
            exito, conflictos = backJumpingRec(k+1, listaVariables)

            if exito:
                return True, set()

            if k not in conflictos:
                actualVar.valor = None
                return False, conflictos
            else:
                actualVar.conflictos = actualVar.conflictos | (conflictos - {k})

        else:
            actualVar.conflictos = actualVar.conflictos | conflictosValor

        actualVar.valor = None

    return False, actualVar.conflictos | {k}
```

La función devuelve una pareja (éxito, conflictos):

- (True, set()) si se ha encontrado una solución.
- (False, conflictos) si no, junto al conjunto de variables culpables.

El caso base de *BackJumping* es el mismo que el de *BackTracking*, es decir, si se ha llegado al final de *listaVariables*, todas las variables tienen una palabra asignada y se ha encontrado solución.

En el caso general vaciamos el conjunto *conflictos* de la variable actual y recorremos las palabras de su dominio, asignando cada palabra a la variable y llamando a la función auxiliar *conflictosCon*.

- Si no encuentra conflicto, la asignación es válida y se realiza la llamada recursiva. Si se encuentra una solución, devolvemos *True*, y si no se encuentra hay que comprobar si la variable actual está entre los culpables que ha devuelto.
    - Si la variable actual no está entre los culpables, no tiene sentido cambiar su palabra, ya que no soluciona el conflicto, por lo que desasignamos y devolvemos el mismo conjunto de conflictos a la variable anterior
    - Si la variable actual si está entre los culpables, el salto termina en esta variable, y añadimos a su conjunto los culpables recibidos *(quitándose a sí misma, ya que no puede ser culpable de sí misma)* y se prueba la siguiente palabra de su dominio.
- Si hay conflictos, la palabra no es válida y sus culpables se añaden al conjunto de conflictos de la variable actual mediante la unión de conjuntos.

Si hemos llegado a probar todas las palabras del dominio sin éxito, devolveremos *False* junto al conjunto *conflictos* acumulado y la propia variable. De esta manera cada variable anterior va a comprobar si está en ese conjunto, y la primera que lo esté será la más reciente de las culpables, que es a lo que tendremos que saltar. Si ninguna variable anterior está en el conjunto, el aviso llegará hasta la primera variable y el problema no tiene solución.

5.4.3 Función backJumping

```
def backJumping(k, listaVariables):
    exito, conflictos = backJumpingRec(k, listaVariables)
    return exito
```

Como *backJumpingRec* devuelve una pareja *(éxito, conflictos)*, que es necesario para poder realizar las diferentes llamadas recursivas, necesitamos adaptarla para que pueda ser utilizada por nuestra función *resuelve* que se encarga de ejecutar un algoritmo, y espera que devuelva *True* si tiene solución y *False* si no tienen solución.

5.5 Modificación de main.py

5.5.1 Botón BJ

Se ha modificado código para poder añadir el botón de *BJ* para ejecutar el algoritmo. para añadir el botón:

```
    posBotBJ=(altoVentana//8)*2-altoBoton/2



    botBK=pygame.Rect(anchoVentana-95, posBotBK, 70, altoBoton)
    botBJ=pygame.Rect(anchoVentana-95, posBotBJ, 70, altoBoton)
```

```
    botFC=pygame.Rect(anchoVentana-95,posBotFC , 70, altoBoton)
    botAC3=pygame.Rect(anchoVentana-95, posBotAC3, 70, altoBoton)
    botVariable=pygame.Rect(anchoVentana-95, posBotVariable, 70, altoBoton)
    botGuardarCrucigrama=pygame.Rect(anchoVentana-95, posBotGuardarCrucigrama, 70, altoBoton)
    botCargarCrucigrama=pygame.Rect(anchoVentana-95, posBotCargarCrucigrama, 70, altoBoton)
```

Y el evento del botón *BJ:*

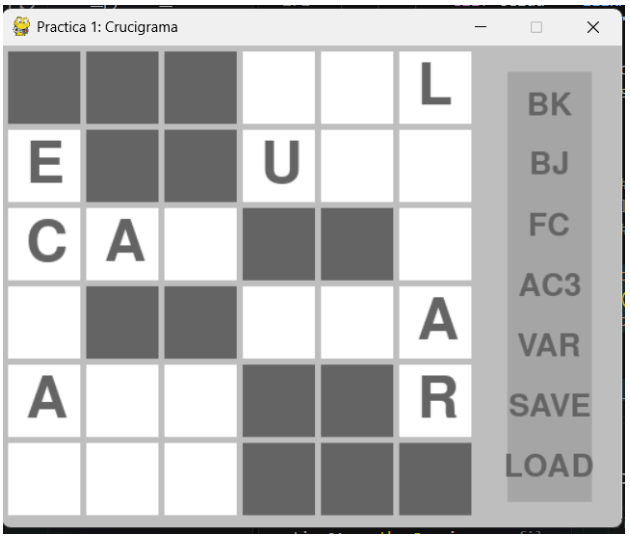
```
elif pulsaBoton(pos, botBJ):
                    if todasVariables is None:
                        todasVariables = obtenerVariables(tablero, almacen)
                    resuelve(backJumping, tablero, todasVariables, tiempoAC3)
                    todasVariables = None
                    tiempoAC3 = None
```

5.6 Ejecución del algoritmo BackJumping

Realizaremos las ejecuciones con las mismas semillas y los mismos crucigramas que la sesión anterior, así podremos comparar BackJumping con los otros algoritmos.

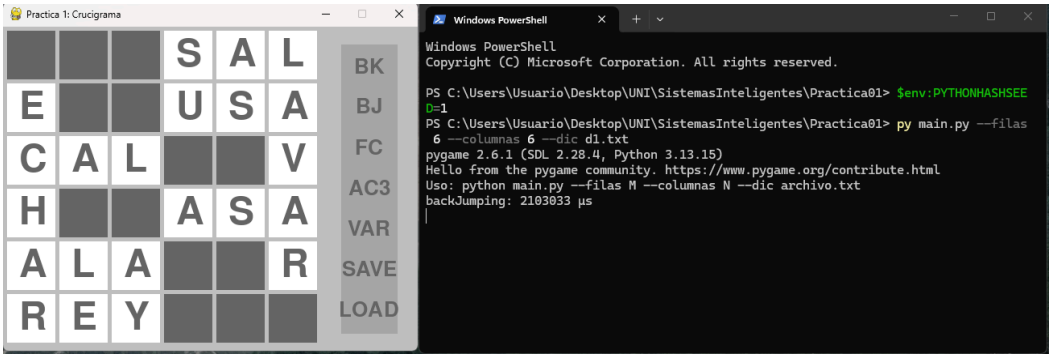
5.6.1 Tablero 6x6

El crucigrama empleado para el tablero 6x6:

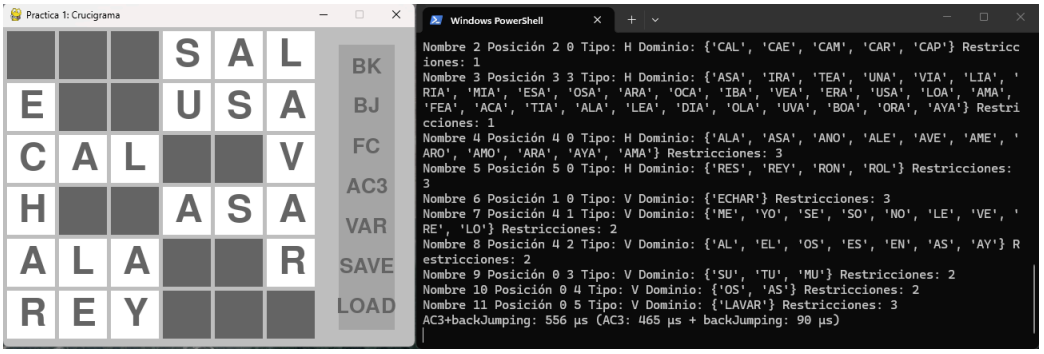


SEMILLA 1

- **BackJumping - 2103033 µs**

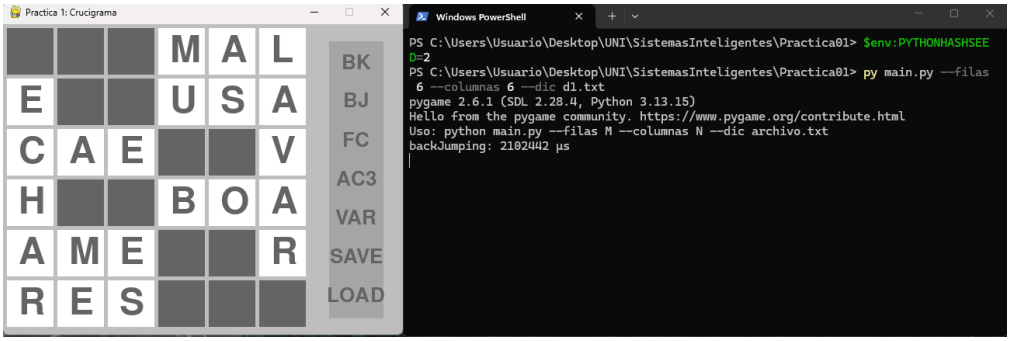


- **AC3 + BackJumping - 556 µs (465 µs + 90 µs)**

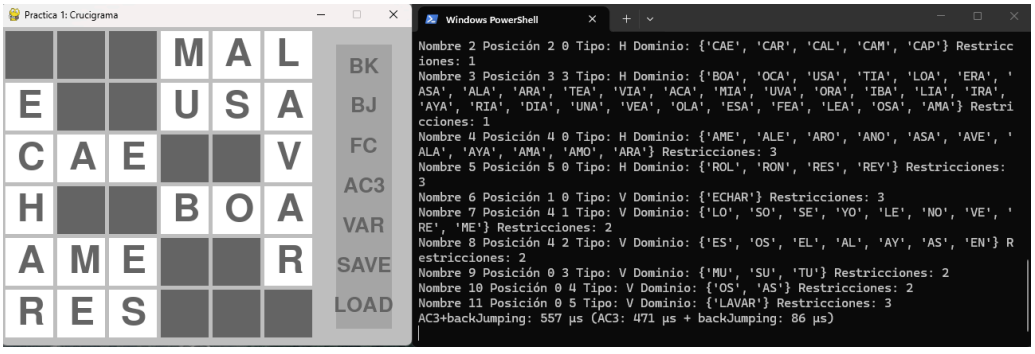


SEMILLA 2

- **BackJumping - 2102442 µs**

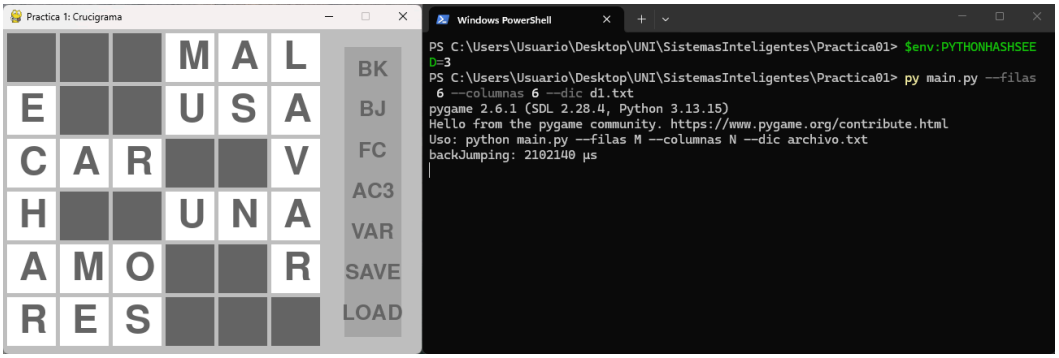


- **AC3 + BackJumping - 557 µs (471 µs + 86 µs)**

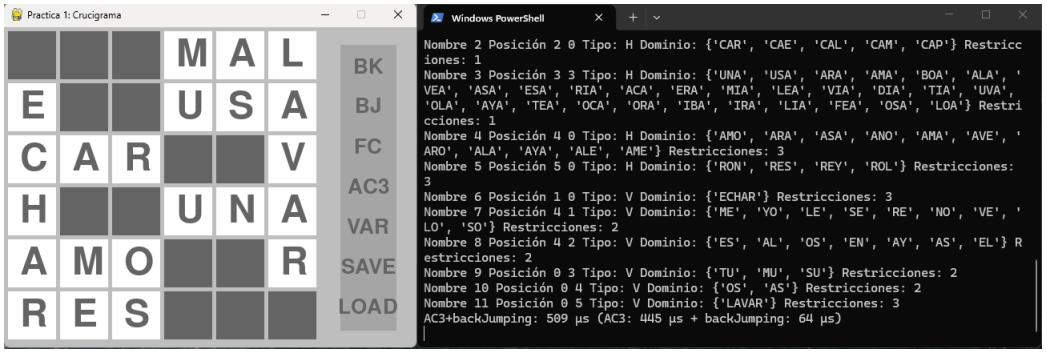


SEMILLA 3

- **BackJumping - 2102140 µs**

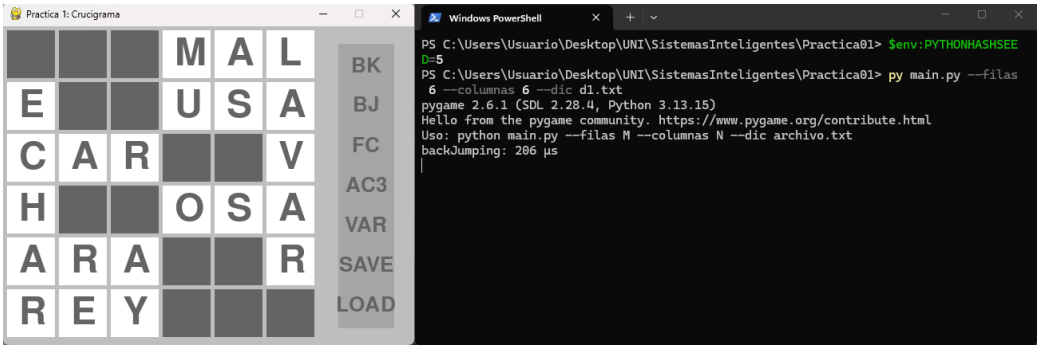


- **AC3 + BackJumping - 509 µs (445 µs + 64 µs)**

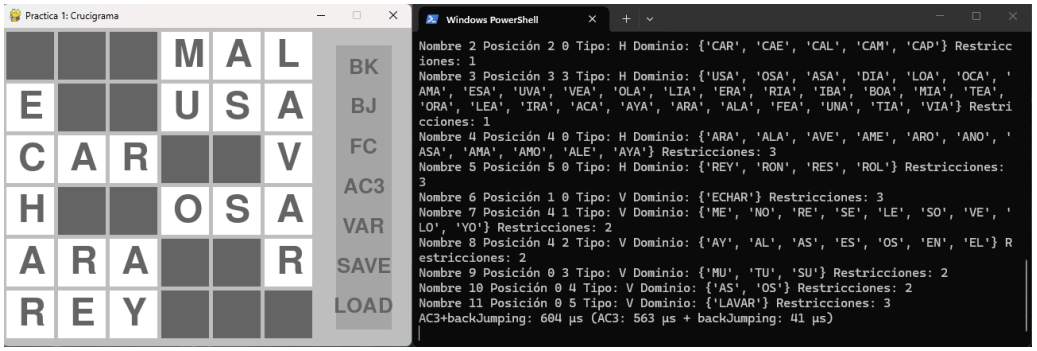


SEMILLA 5

- **BackJumping - 206 µs**

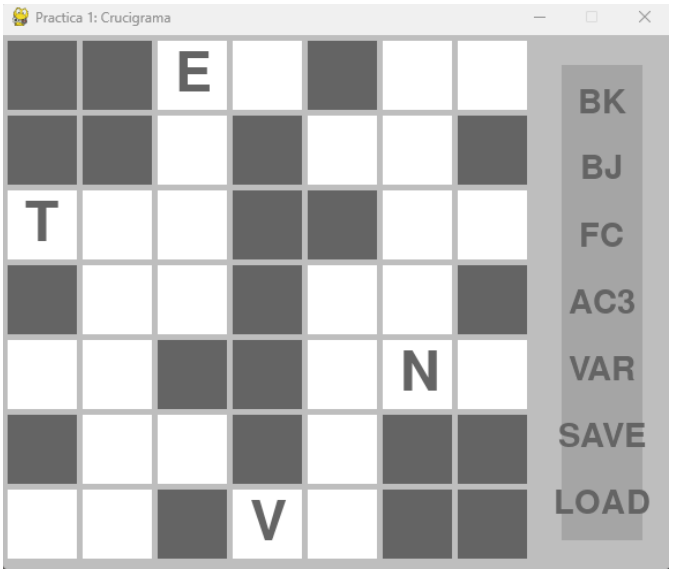


- **AC3 + BackJumping - 604 µs (563 µs + 41 µs)**



5.6.2 Tablero 7x7

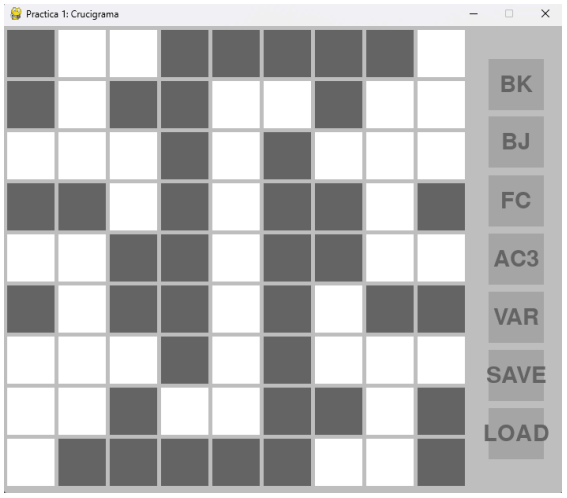
El crucigrama empleado para el tablero 7x7:



Para las cuatro semillas utilizadas, BackJumping, tanto aplicando como sin aplicar AC3 tarda más de 60 segundos.

5.6.3 Tablero 9x9

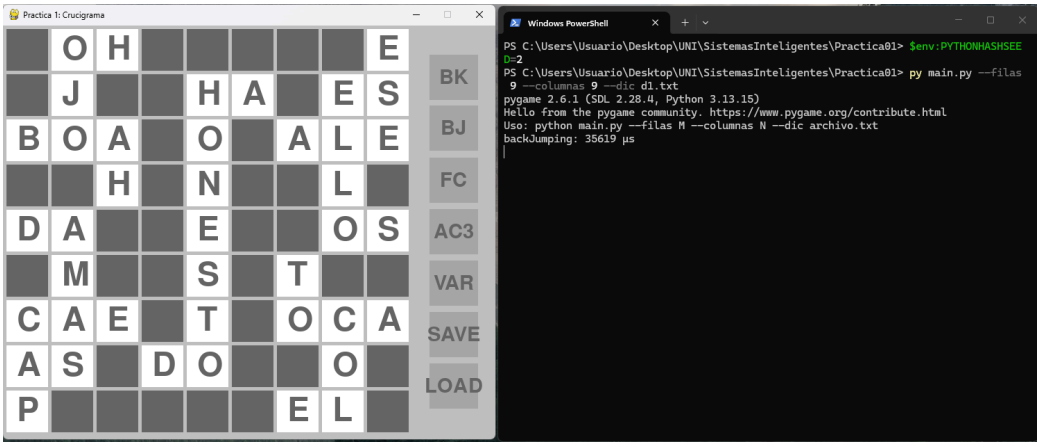
El crucigrama empleado para el tablero 9x9:



En este tablero, la única semilla que ha logrado que el algoritmo se ejecute en menos de 60 segundos es la semilla 2.

SEMILLA 2

- **BackJumping - 35619 µs**



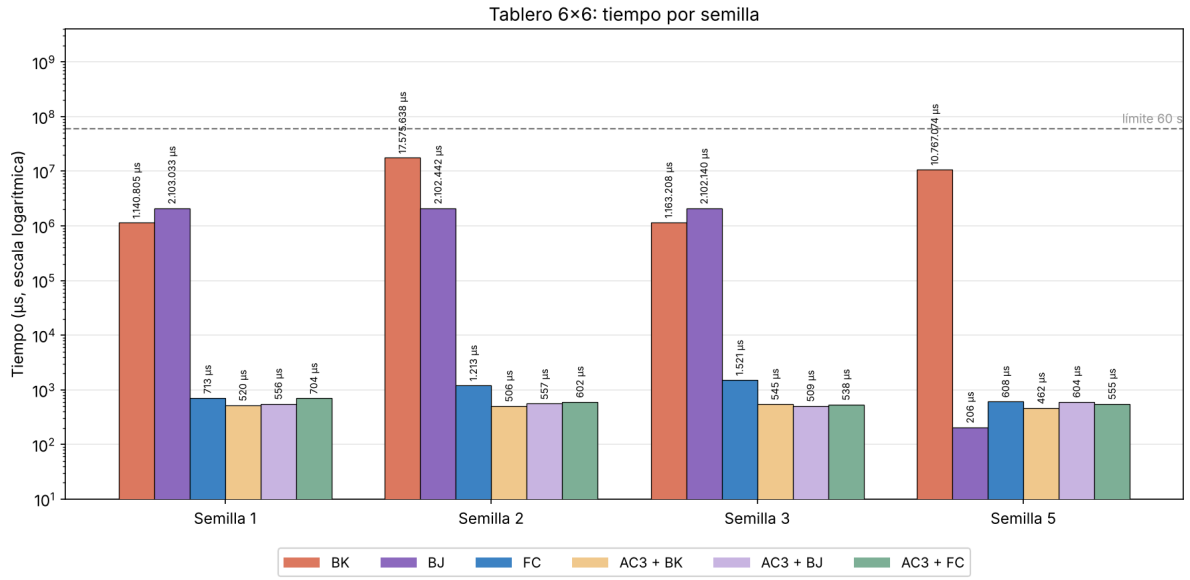
- **AC3 + BackJumping - 28978 µs (23476 µs + 5502 µs)**



5.6.4 Análisis de los resultados obtenidos

Tablero 6x6

Algoritmo	Semilla 1 - µs	Semilla 2 - µs	Semilla 3 - µs	Semilla 5 - µs	Media - µs
BK	1.140.805	17.575.638	1.163.208	10.767.074	7.661.681
FC	713	1.213	1.521	608	1.014
BJ	2.103.033	2.102.442	2.102.140	206	1.576.955
AC3+BK	520 (AC3 461)	506 (AC3 454)	545 (AC3 502)	462 (AC3 433)	508
AC3+FC	704 (AC3 562)	602 (AC3 486)	538 (AC3 435)	555 (AC3 436)	600
AC3+BJ	556 (AC3 465)	557 (AC3 471)	509 (AC3 445)	604 (AC3 563)	557



Como podemos observar, BackJumping consigue mejorar sobre BackTracking, consiguiendo incluso bajar de 17,6 segundos a 2,1 segundos en la semilla 2, siendo más de 8 veces más rápido, y además en la semilla 5 se ha conseguido incluso ejecutar en 206 µs, unas 52.000 veces más rápido que BackTracking e incluso siendo tres veces más rápido que ForwardChecking.
Sin embargo, en las semillas 1 y 3 BackJumping ha sido más lento que BackTracking, tardando unos 2,1 segundos frente a 1,1 segundos, y aunque BackJumping no deba de explorar todas las combinaciones, la función *conflictosCon* tiene que revisar todas las variables anteriores para saber cuales son las culpables, mientras que *isFeasible* se detiene en el primer conflicto. Por eso cuando BackTracking no tiene casi que retroceder, BackJumping no tiene saltos que aprovechar y solo añade ese coste extra.
Aun así, comparándolo con ForwardChecking, excluyendo el suceso de la semilla 5, BackJumping sigue siendo mucho más lento, unas 1.800 veces de media.

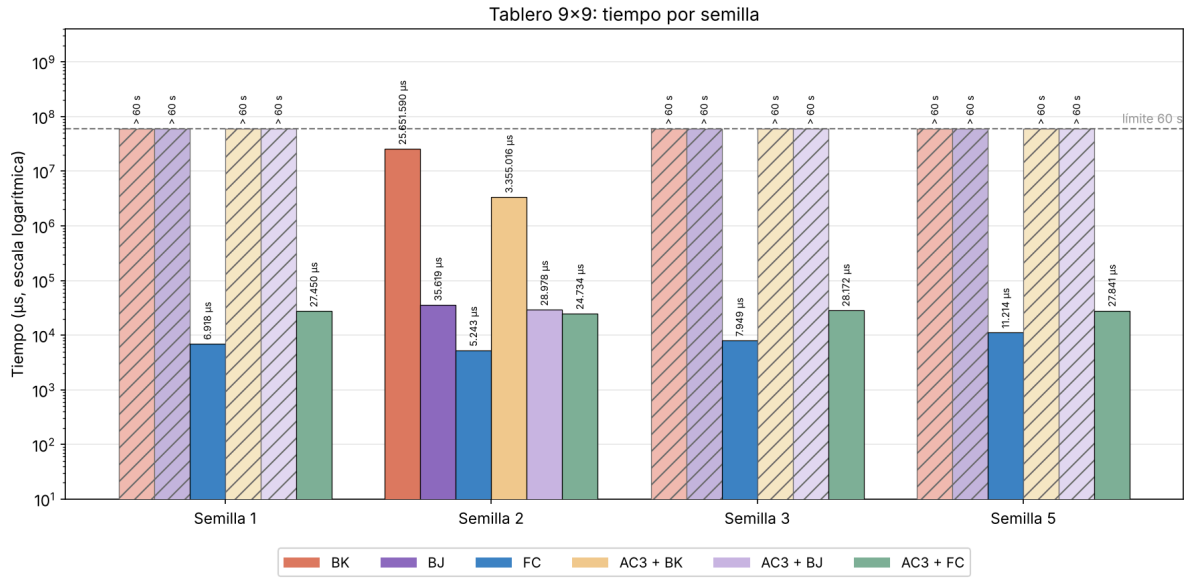
Por último, aplicando AC3 previamente, los tres algoritmos siguen teniendo un tiempo bastante idéntico ya que el propio AC3 reduce drásticamente los dominios, haciendo que la búsqueda sea casi inmediata.

Tablero 7x7

En este tablero BackJumping no ha conseguido terminar en menos de 60 segundos con ninguna semilla, ni siquiera aplicando AC3 previamente, igual que ocurría con BackTracking.

Tablero 9x9

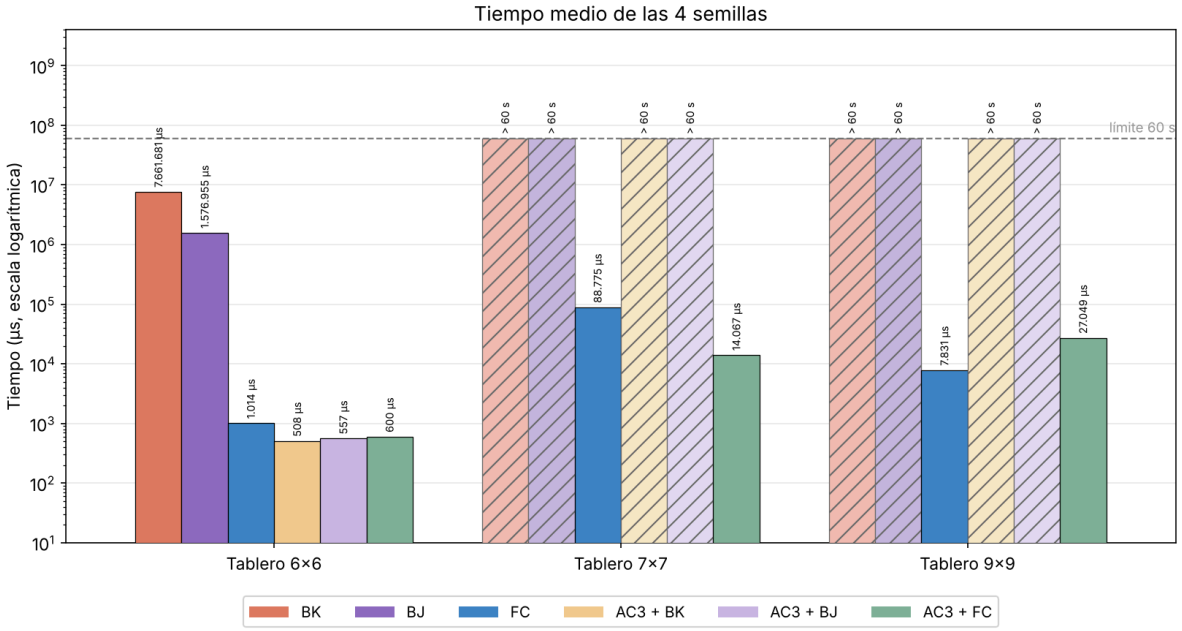
Algoritmo	Semilla 1 - µs	Semilla 2 - µs	Semilla 3 - µs	Semilla 5 - µs	Media - µs
BK	> 60 s	25.651.590	> 60 s	> 60 s	-
FC	6.918	5.243	7.949	11.214	7.831
BJ	> 60 s	35.619	> 60 s	> 60 s	-
AC3+BK	> 60 s	3.355.016 (AC3 22.323)	> 60 s	> 60 s	-
AC3+FC	27.450 (AC3 22.489)	24.734 (AC3 22.366)	28.172 (AC3 22.908)	27.841 (AC3 22.627)	27.049
AC3+BJ	> 60 s	28.978 (AC3 23.476)	> 60 s	> 60 s	-



En este tablero 9x9, BackJumping solo logra terminar en menos de 60 segundos en la semilla 2, igual que BackTracking, y se observa la mejora sobre BackTracking obteniendo la solución en 35,6 ms frente a los 25 segundos, siendo unas 720 veces más rápido. Con este resultado, BackJumping se acerca a ForwardChecking, aunque sigue siendo unas 7 veces más lento. Aplicando AC3 previamente, AC3 + BJ es unas 116 veces más rápido que AC3 + BK, unos 29 ms frente a 3,36 segundos, y se queda muy cerca de AC3+FC.

Sin embargo en el resto de semillas no logra terminar en menos de 60 segundos, por lo que Forward Checking sigue siendo una opción más sólida

Conclusión



Comparando los resultados obtenidos, podemos concluir lo siguiente:

- BackJumping mejora mucho a BackTracking, si BK tiene que retroceder mucho atrás, siendo hasta 52.000 veces más rápido, pero si no retrocede, el coste extra de calcular los conflictos hace que pueda ser más lenta BackJumping.
- ForwardChecking sigue siendo la opción más sólida, ya que es más rápido en casi todos los casos y es capaz de resolver todos los tableros en menos de 60 segundos.

6. Sesión 8 - Documentación y pruebas

6.1 Contexto

En esta última sesión, que está dedicada sobre todo a la documentación y a las pruebas, nos vamos a encargar de completar todo lo que falta para la entrega final.

6.2 Clase Variable

A lo largo de las sesiones, la clase Variable ha ido actualizando sus atributos, ya que cada algoritmo necesitaba algún atributo específico para poder ejecutarse, y la versión final de la clase es la siguiente:

```
class Variable:
    def __init__(self, nombre, tipo, fila, col, longitud, dominio):
        self.nombre = nombre
        self.tipo = tipo        # h/v
        self.fila = fila
        self.col = col
        self.longitud = longitud
        self.dominio = dominio
        self.restricciones = [] # lista de tuplas (vecino, pos_propia, pos_vecino)
        self.valor = None
        self.podado = []    # lista de tuplas (palabra_podada, variable_responsable)
        self.conflictos = set()

    def __str__(self):
        return f"Nombre {self.nombre} Posición {self.fila} {self.col} Tipo: {self.tipo} Dominio:
{self.dominio} Restricciones: {len(self.restricciones)}"
```

Atributo	Para qué sirve
nombre	Identificar la variable
tipo	Indicar si la variable es horizontal (H) o vertical (V)
fila, col	Indicar la posición de la casilla donde empieza la variable
longitud	Indicar el número de casillas que ocupa una variable
dominio	Es un Set con las palabras candidatas a poner en una variable
restricciones	Lista de tuplas con la forma (vecino, pos_propia, pos_vecino) que guarda cada cruce con otra variable
valor	la palabra que se ha asignado a esa variable
podado	Lista de tuplas con la forma (palabra_podada, variable_responsable) con las palabras que ha quitado *forward*

conflictos	Set con los índices de las variables anteriores que impiden usar alguna palabra

6.3 Cómo se han tratado las casillas con carácteres fijos

Las casillas con carácteres fijos se han tratado como una restricción que afecta únicamente a la variable o variables que pertenecen a esa casilla, y por eso al construir el dominio de cada variable, en las funciones *filtraDominioHorizontal* y *filtraDominioVertical*, tal como se explico en los apartados 1.3.2 y 1.3.4: solo se asignarán a dicho dominio las palabras que tengan ese carácter justo en esa casilla.

Así cuando ejecutemos cualquiera de los algoritmos, los dominios ya cumplen con las letras fijas y estos no van a tener que ir comprobando todo el rato si una palabra puede entrar en una variable.

Además también como se explicó en el apartado 1.3.1, las variables de una sola casilla no se crean, ya que una casilla suelta no forma una palabra, además de que el diccionario *d0.txt* que nos proporcionaron no tiene palabras de una letra.

6.4 Especificación formal

UN **CSP** se puede representar como un grafo sobre el que se define una red de restricciones, y esta red es la **tupla (V, E, c, I,a)**:

- **V**: conjunto de nodos
- **E**: conjunto de aristas
- **c**: función que asigna cada arista a una tupla de nodos
- **I**: función que asigna cada arista a una restricción
- **a**: permutación que define el orden de selección de las variables

Formulación general

V = {VO,V1,...,Vn-1}, huecos de dos o más casilla para una palabra
D_i = {palabras del diccionario de longitud $|V_i|$ que tengan las letras fijas de V_i en su posición}, $\forall\ i \in [0, n-1]$
E = {e1, …, em}, intersecciones entre huecos y pares de huecos de la misma longitud
c: E → V × V, asigna cada arista a un par de variables
I: E → {misma letra, palabra distinta, misma letra y palabra distinta}

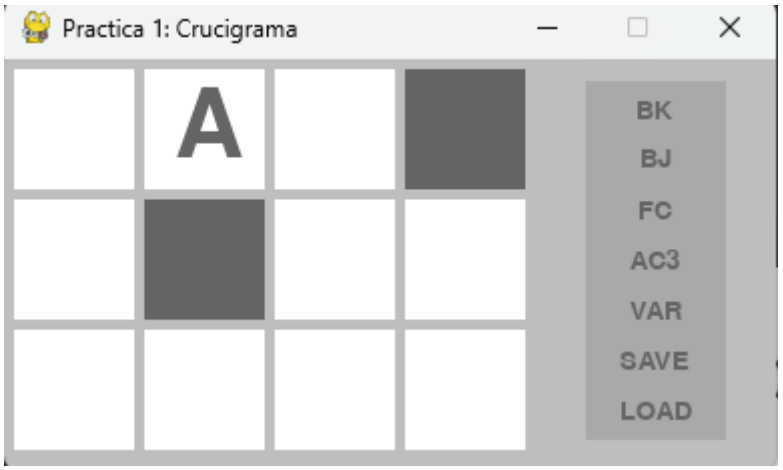
ρ_misma letra((V_i, V□)) = {⟨w_i, w□⟩ ∈ D_i × D□ : w_i[k] = w□[l] para (k, l) = pos_inter(V_i, V□)}
ρ_palabra distinta((V_i, V□)) = {⟨w_i, w□⟩ ∈ D_i × D□ : $w_i \neq$ w□}, solo si $|V_i| = |V□|$
ρ_misma letra y palabra distinta = ρ_misma letra ∩ ρ_palabra distinta
a: permutación de [0, n−1], orden en que se asignan las variables

La restricción de palabra distinta solo se pone entre huecos de la misma longitud, porque dos palabras de longitud distinta nunca son iguales. pos_inter(V_i, V□) es la posición del cruce en cada palabra, que en el código se guarda en la lista restricciones como (vecino,

pos_propia, pos_vecino). Las letras fijas no aparecen como aristas porque son restricciones unarias: se aplican directamente en el dominio (apartado 6.3).

Aplicación al crucigrama de ejemplo

Dado el siguiente tablero



y el diccionario:

AS NO OS SO
LAS LOS SAL SOL OSO
LISO LOSA

V = {V0, V1, V2, V3, V4, V5}, donde:

- V0: horizontal, filas {0}, columnas {0,1,2}
- V1: horizontal, filas {1}, columnas {2,3}
- V2: horizontal, filas {2}, columnas {0,1,2,3}
- V3: vertical, filas {0,1,2}, columnas {0}
- V4: vertical, filas {0,1,2}, columnas {2}
- V5: vertical, filas {1,2}, columnas {3}

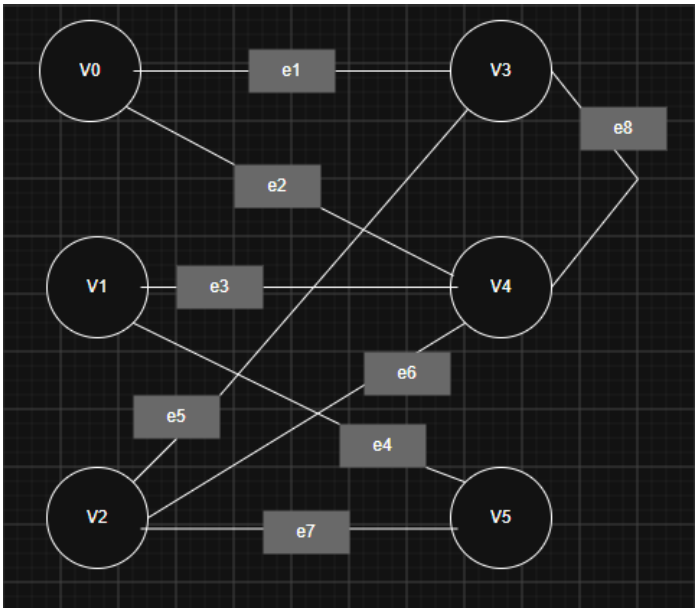
Dominios:

- D0 = {LAS, SAL}. *(El carácter fijo A hace que el dominio se restringa automáticamente)*
- D1 = D5 = {AS, NO, OS, SO}
- D2 = {LISO, LOSA}
- D3 = D4 = {LAS, LOS, OSO, SAL, SOL}

E = {e1, e2, e3, e4, e5, e6, e7, e8}

c(e1) = ⟨V0, V3⟩, c(e2) = ⟨V0, V4⟩, c(e3) = ⟨V1, V4⟩, c(e4) = ⟨V1, V5⟩,
c(e5) = ⟨V2, V3⟩, c(e6) = ⟨V2, V4⟩, c(e7) = ⟨V2, V5⟩, c(e8) = ⟨V3, V4⟩

Quedando así el ***Grafo de restricciones (V,E)***



Para cada arista indicamos la restricción que le asigna ***I*** y el conjunto de pares de palabras que la cumplen, calculado con los dominios anteriores.

Arista	**Restricción**	**I(e_i): pares permitidos**
e1	misma letra (V0[0] = V3[0]) y palabra distinta	{⟨LAS, LOS⟩, ⟨SAL, SOL⟩}
e2	misma letra (V0[2] = V4[0]) y palabra distinta	{⟨LAS, SAL⟩, ⟨LAS, SOL⟩, ⟨SAL, LAS⟩, ⟨SAL, LOS⟩}
e3	misma letra (V1[0] = V4[1])	{⟨AS, LAS⟩, ⟨AS, SAL⟩, ⟨OS, LOS⟩, ⟨OS, SOL⟩, ⟨SO, OSO⟩}
e4	misma letra (V1[1] = V5[0]) y palabra distinta	{⟨AS, SO⟩, ⟨NO, OS⟩, ⟨OS, SO⟩, ⟨SO, OS⟩}
e5	misma letra (V2[0] = V3[2])	{⟨LISO, SAL⟩, ⟨LISO, SOL⟩, ⟨LOSA, SAL⟩, ⟨LOSA, SOL⟩}
e6	misma letra (V2[2] = V4[2])	{⟨LISO, LAS⟩, ⟨LISO, LOS⟩, ⟨LOSA, LAS⟩, ⟨LOSA, LOS⟩}
e7	misma letra (V2[3] = V5[1])	{⟨LISO, NO⟩, ⟨LISO, SO⟩}
e8	palabra distinta	D3 × D4 − {⟨LAS, LAS⟩, ⟨LOS, LOS⟩, ⟨OSO, OSO⟩, ⟨SAL, SAL⟩, ⟨SOL, SOL⟩} … (20 pares)

Las aristas numeradas del 1 al 7, es decir, desde e_1 hasta e_7 son los cruces en el que dos variables, una horizontal y una vertical, comparten una única casilla y deben de tener el mismo carácter en esa casilla. La arista e_8 solo tiene la restricción de que sea una palabra distinta, ya que V3 y V4 tienen la misma longitud pero no se cruzan. Además en la tabla no aparece la restricción de que las aristas deben de ser palabras distintas, no quiere decir que se puedan repetir las palabras, sino que dichas variables tienen longitudes diferentes y por tanto es imposible que puedan usar la misma palabra.

En la tabla además podemos observar que hay palabras que no aparecen en ningún par permitido de alguna arista como *LOSA*, que no aparece en ningún par de *e7*, ni tampoco *NO* como valor de V1 en la arista *e3*.

Por último, la permutación *a* define el orden en el que se asignan las variables, con el orden original, *a = (V0, V1, V2, V3, V4, V5)*, asignando primero las horizontales y luego las verticales.

Con todo esto, el problema tiene dos soluciones, que solo se diferencian en V1 y V4:

- V0 = SAL, V1 = AS, V2 = LISO, V3 = SOL, V4 = LAS, V5 = SO
- V0 = SAL, V1 = OS, V2 = LISO, V3 = SOL, V4 = LOS, V5 = SO

6.5 Traza de Forward Checking

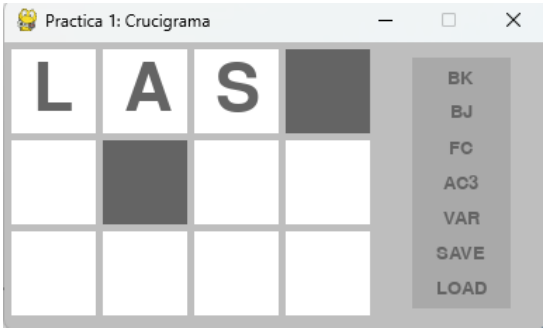
A continuación se realizará la traza de Forward Checking sobre el crucigrama del apartado 6.4, mostrando cómo evolucionan los dominios de las variables y qué palabras se podan tras cada asignación. Además la traza se va a realizar con el orden original de las variables, a = (V0, V1, V2, V3, V4, V5), y probando las palabras de cada dominio en orden alfabético.

Dominios iniciales:

CD0 = {LAS, SAL}
CD1 = {AS, NO, OS, SO}
CD2 = {LISO, LOSA}
CD3 = {LAS, LOS, OSO, SAL, SOL}
CD4 = {LAS, LOS, OSO, SAL, SOL}
CD5 = {AS, NO, OS, SO}

i=0 (V0), paso 1:

Seleccionamos en nuestra primera variable V0 el valor de *LAS*, y para hacerlo más visual iremos escribiendo a mano en el crucigrama:
V0: {**LAS**,SAL}



Al realizar esta selección se llama a *forward*, y V0 se cruza con V3 y V4, así que se podan de sus dominios las palabras que no son compatibles:

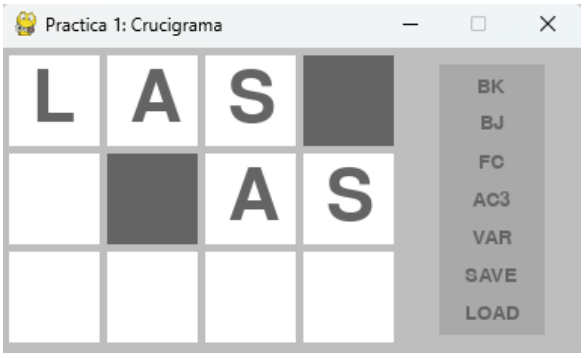
V3: {~~LAS~~, LOS, ~~OSO, SAL, SOL~~} → CD3 = {LOS}
V4: {~~LAS, LOS, OSO~~, SAL, SOL} → CD4 = {SAL,SOL}

Hemos podado en V3 las palabras que no empiezan por L (arista e1), y también LAS por ser la misma palabra que V0, y en V4 las que no empiezan por S (arista e2). Ningún dominio se queda vacío, así que forward devuelve CIERTO y pasamos a la siguiente variable.

i=1(V1), paso 2:

Pasamos a V1 y seleccionamos y seleccionamos el valor AS, que es la primera palabra de su dominio:

V1: {**AS**,NO,OS,SO}



Al realizar esta selección, llamamos a la función *forward*, y como V1 se cruza con V4 y V5:

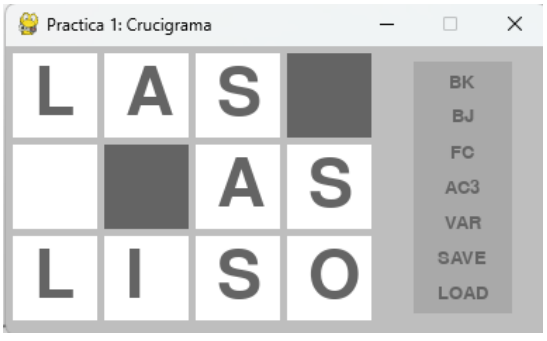
V4: {SAL,~~SOL~~} → CD4 = {SAL}
V5: {~~AS,NO,OS~~,SO} → CD5 = {SO}

Para V4 hemos podado SOL ya que esta palabra no tiene una *A* en el centro (arista e3), y en V5 podamos las que no empiezan por S (arista e4). Como ningún dominio se queda vacío, seguimos.

i = 2 (V2), paso 3:

Pasamos a V2 y seleccionamos *LISO:*

v2: {**LISO**, LOSA}



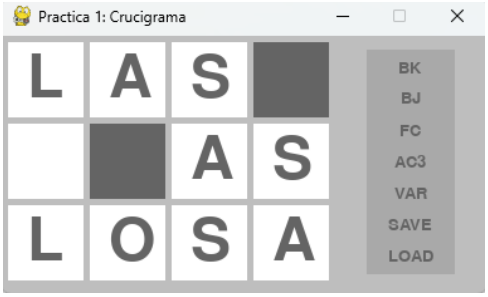
Al llamar a forward, la primera variable futura que se revisa es V3. Por la arista e5, V3 tiene que acabar en L, pero la única palabra que le queda, LOS, acaba en S:

V3: {~~LOS~~} → CD3 = { }

El dominio de V3 se queda vacío, así que forward devuelve FALSO y ya no revisa V4 ni V5. Se deshace V2 = LISO y restaurar devuelve LOS a V3. Probamos el siguiente valor de V2.

i = 2 (V2), paso 4:

Seleccionamos LOSA, el siguiente valor de V2:
V2: {~~LISO~~, **LOSA**}



Pasa lo mismo que con LISO, ya que LOSA también empieza por L:
V3: {~~LOS~~}→ CD3 = { }

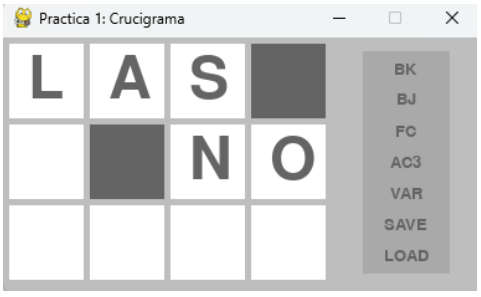
Forward devuelve FALSO y restaurar devuelve LOS a V3. V2 ya no tiene más valores, así que FC devuelve FALSO y volvemos a i = 1 (V1). Se deshace V1 = AS, y restaurar devuelve SOL a V4 y AS, NO y OS a V5, que quedan como estaban después del paso 1:

CD4 = {SAL, SOL}
CD5 = {AS, NO, OS, SO}

Probamos el siguiente valor de V1

i = 1 (V1), paso 5:

Seleccionamos NO, el siguiente valor de V1:
V1: {~~AS~~, **NO**, OS, SO}



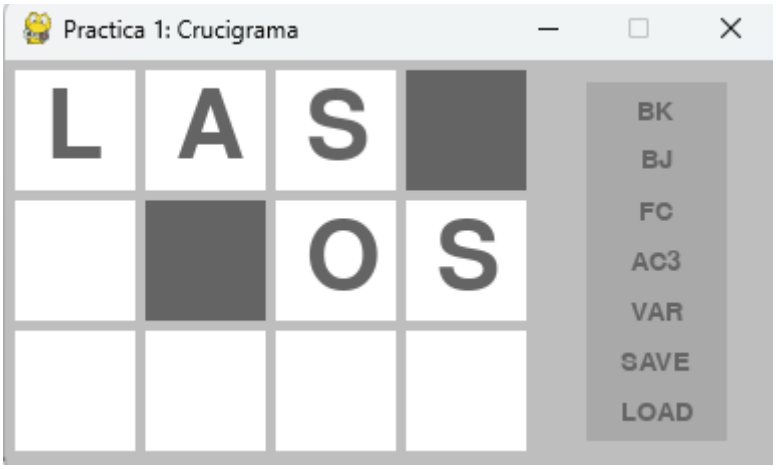
Al llamar a forward, V4 tendría que tener una N en el centro (arista e3), y ninguna de sus palabras la tiene:

V4: {~~SAL, SOL~~} → CD4 = { }

Forward devuelve FALSO. Se deshace V1 = NO y restaurar devuelve SAL y SOL a V4. Probamos el siguiente valor de V1.

i = 1 (V1), paso 6:

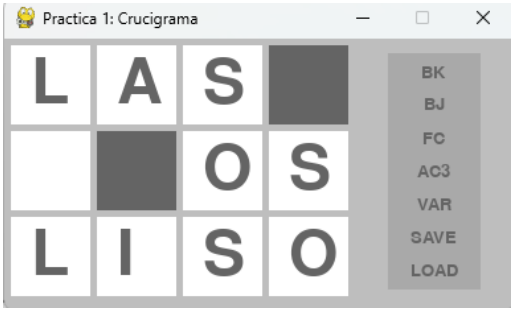
Seleccionamos OS, el siguiente valor de V1:
V1: {~~AS, NO~~, **OS**, SO}



Al llamar a forward se podan de V4 las palabras que no tienen una O en el centro (arista e3), y de V5 las que no empiezan por S (arista e4):
V4: {~~SAL~~, SOL} → CD4 = {SOL}
V5: {~~AS, NO, OS~~,SO} → CD5 = {SO}
Ningún dominio se queda vacío, así que forward devuelve CIERTO y pasamos a la siguiente variable.

i = 2 (V2), paso 7:

Seleccionamos LISO:
V2: {**LISO**, LOSA}

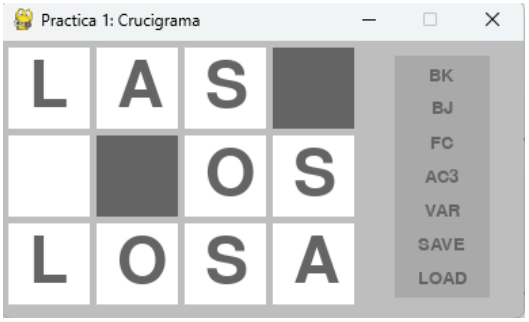


V3: {~~LOS~~} → CD3 = { }

Igual que en el paso 3, el dominio de V3 se queda vacío. Forward devuelve FALSO y restaurar devuelve LOS a V3. Probamos el siguiente valor de V2.

i = 2 (V2), paso 8:

Seleccionamos LOSA:
V2: {~~LISO~~, **LOSA**}



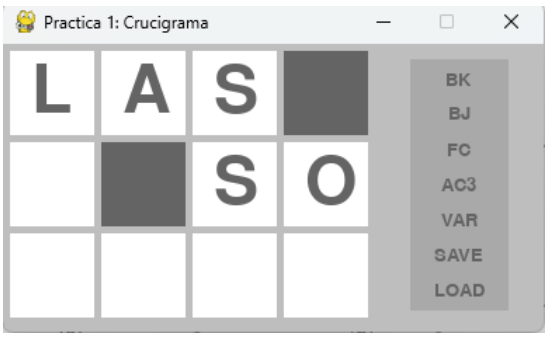
V3: {~~LOS~~} → CD3 = { }

Igual que en el paso 4, forward devuelve FALSO y restaurar devuelve LOS a V3. V2 ya no tiene más valores, así que volvemos a i = 1 (V1). Se deshace V1 = OS, y restaurar devuelve SAL a V4 y AS, NO y OS a V5:

CD4 = {SAL, SOL}
CD5 = {AS, NO, OS, SO}
Probamos el siguiente valor de V1.

i = 1 (V1), paso 9:

Seleccionamos SO, el último valor de V1:
V1: {~~AS, NO, OS~~, **SO**}



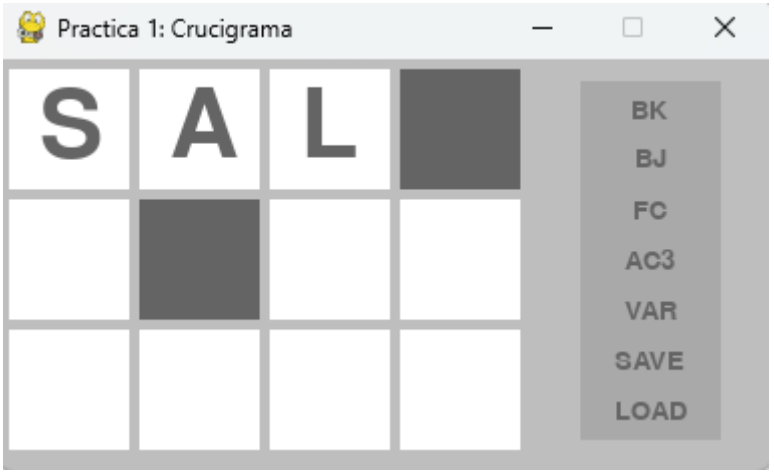
Al llamar a forward, V4 tendría que tener una S en el centro (arista e3), y ninguna de sus palabras la tiene:

V4: {~~SAL,~~ ~~SOL}~~ → CD4 = { }

Forward devuelve FALSO y restaurar devuelve SAL y SOL a V4. V1 ya no tiene más valores, así que FC devuelve FALSO y volvemos a i = 0 (V0). Se deshace V0 = LAS, y restaurar devuelve LAS, OSO, SAL y SOL a V3, y LAS, LOS y OSO a V4. Todos los dominios vuelven a ser los iniciales. Probamos el siguiente valor de V0.

i = 0 (V0), paso 10:

Seleccionamos SAL, el siguiente valor de V0:
V0: {~~LAS~~, **SAL**}



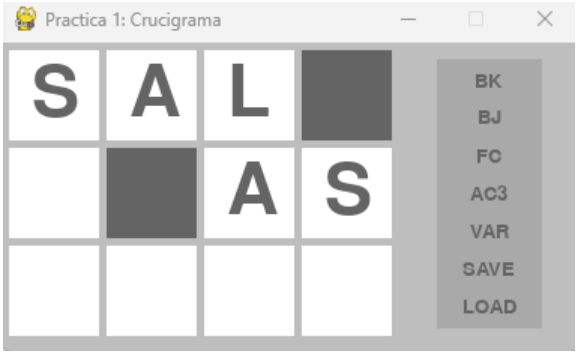
Al llamar a forward se podan de nuevo los dominios de V3 y V4:

V3: {~~LAS,LOS,OSO,SAL,~~ SOL} → CD3 = {SOL}
V4: {LAS, LOS, ~~OSO,SAL,SOL}~~ → CD4 = {LAS, LOS}

En V3 se podan las palabras que no empiezan por S (arista e1), y también SAL por ser la misma palabra que V0. En V4 se podan las que no empiezan por L (arista e2). Ningún dominio se queda vacío, así que forward devuelve CIERTO y pasamos a la siguiente variable.

i = 1 (V1), paso 11:

Seleccionamos AS:
V1: {**AS**, NO, OS, SO}

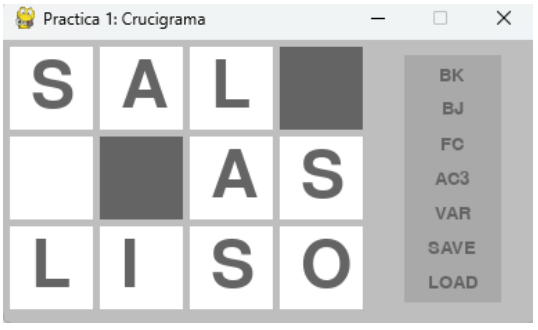


V4: {LAS,~~LOS~~} → CD4 = {LAS}
V5: {~~AS,NO,OS~~, SO} → CD5 = {SO}

En V4 se poda LOS porque no tiene una A en el centro (arista e3), y en V5 se podan las palabras que no empiezan por S (arista e4). Forward devuelve CIERTO y pasamos a la siguiente variable.

i = 2 (V2), paso 12:

Seleccionamos LISO:
V2: {**LISO**, LOSA}

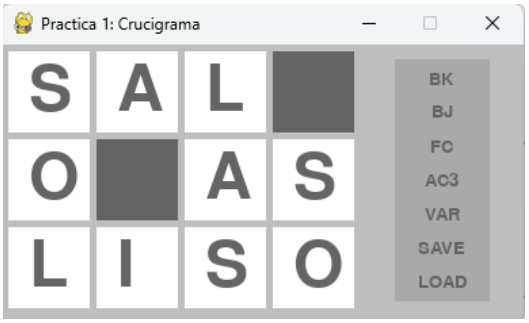


Al llamar a forward no se poda nada, ya que las palabras que quedan en las variables

futuras son compatibles con LISO: SOL acaba en L (arista e5), LAS acaba en S (arista e6) y SO acaba en O (arista e7). Forward devuelve CIERTO y pasamos a la siguiente variable.

i = 3 (V3), paso 13:

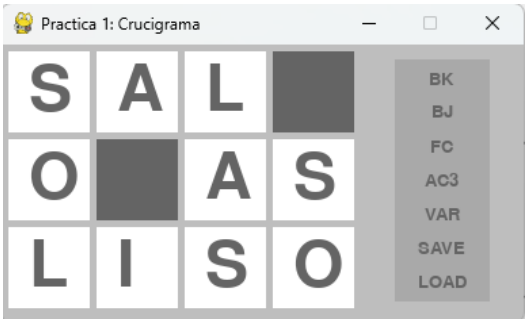
V3 solo tiene un valor, así que seleccionamos SOL:
V3: {**SOL**}



Forward no poda nada: LAS, la palabra que le queda a V4, es distinta de SOL (arista e8), y V5 no está relacionada con V3. Pasamos a la siguiente variable.

i = 4 (V4), paso 14:

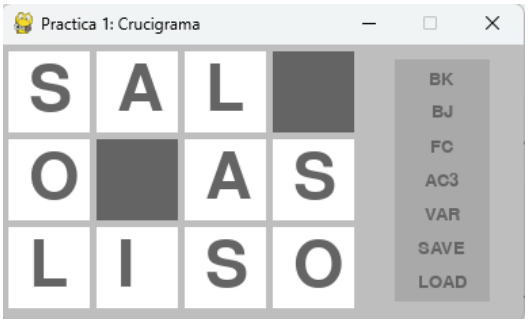
Seleccionamos LAS, el único valor de V4:
V4: {**LAS**}



Forward no poda nada, ya que V5 no está relacionada con V4. Pasamos a la siguiente variable.

i = 5 (V5), paso 15:

Seleccionamos SO, el único valor de V5:
V5: {**SO**}



V5 es la última variable, así que no hay variables futuras que revisar. En la siguiente llamada i llega a 6, que es el número de variables, por lo que todas están asignadas y FC devuelve CIERTO: hemos encontrado una solución.

Solución: V0 = SAL, V1 = AS, V2 = LISO, V3 = SOL, V4 = LAS, V5 = SO, que es la solución 1 del apartado 6.4.

6.6 Traza de AC3

A continuación se realizará la traza de AC3 sobre el crucigrama del apartado 6.4, mostrando cómo evolucionan los dominios de las variables y ver que palabras se borran tras revisar cada arco.

Dominios iniciales:

CD0 = {LAS, SAL}
CD1 = {AS, NO, OS, SO}
CD2 = {LISO, LOSA}
CD3 = {LAS, LOS, OSO, SAL, SOL}
CD4 = {LAS, LOS, OSO, SAL, SOL}
CD5 = {AS, NO, OS, SO}

Q inicial, con los 14 arcos (cada arista de cruce en sus dos sentidos):
Q = [⟨V0,V3⟩, ⟨V0,V4⟩, ⟨V1,V4⟩, ⟨V1,V5⟩, ⟨V2,V3⟩, ⟨V2,V4⟩, ⟨V2,V5⟩, ⟨V3,V0⟩, ⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩]

Paso 1: ⟨V0, V3⟩

Cogemos el primer arco de Q y comprobamos, para cada palabra de V0, si hay alguna palabra en V3 que cumpla la arista e1 (V0[0] = V3[0]). LAS tiene apoyo en LAS y LOS, ya que empiezan por L, y SAL tiene apoyo en SAL y SOL. Al no borrarse nada, no se añade ningún arco a Q y como AC3 solo comprueba el cruce, LAS tiene apoyo en LAS aunque sea la misma palabra.

Q = [⟨V0,V4⟩, ⟨V1,V4⟩, ⟨V1,V5⟩, ⟨V2,V3⟩, ⟨V2,V4⟩, ⟨V2,V5⟩, ⟨V3,V0⟩, ⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩]

Paso 2: ⟨V0, V4⟩

Arista e2 (V0[2] = V4[0]). LAS acaba en S y tiene apoyo en SAL y SOL, y SAL acaba en L y tiene apoyo en LAS y LOS. Sin cambios.

Q = [⟨V1,V4⟩, ⟨V1,V5⟩, ⟨V2,V3⟩, ⟨V2,V4⟩, ⟨V2,V5⟩, ⟨V3,V0⟩, ⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩]

Paso 3: ⟨V1, V4⟩

Arista e3 (V1[0] = V4[1]): la primera letra de V1 tiene que ser la del centro de V4. Ninguna palabra de V4 tiene una N en el centro, así que NO no tiene apoyo y se borra:

D1 = {AS, ~~NO,~~ OS, SO} → D1 = {AS, OS, SO}

Como D1 ha cambiado, habría que añadir a Q el arco que tiene a V1 en la parte derecha, menos el de V4, que es ⟨V5,V1⟩. Como ya estaba en Q, no se añade de nuevo.

Q = [⟨V1,V5⟩, ⟨V2,V3⟩, ⟨V2,V4⟩, ⟨V2,V5⟩, ⟨V3,V0⟩, ⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩]

Paso 4: ⟨V1, V5⟩

Arista e4 (V1[1] = V5[0]). AS y OS tienen apoyo en SO, y SO tiene apoyo en OS. Sin cambios.

Q = [⟨V2,V3⟩, ⟨V2,V4⟩, ⟨V2,V5⟩, ⟨V3,V0⟩, ⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩]

Paso 5: ⟨V2, V3⟩

Arista e5 (V2[0] = V3[2]). LISO y LOSA empiezan por L y tienen apoyo en SAL y SOL, que acaban en L. Sin cambios.

Q = [⟨V2,V4⟩, ⟨V2,V5⟩, ⟨V3,V0⟩, ⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩]

Paso 6: ⟨V2, V4⟩

Arista e6 (V2[2] = V4[2]). LISO y LOSA tienen una S en la posición 2 y tienen apoyo en LAS y LOS, que acaban en S. Sin cambios.

Q = [⟨V2,V5⟩, ⟨V3,V0⟩, ⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩]

Paso 7: ⟨V2, V5⟩

Arista e7 (V2[3] = V5[1]): la última letra de V2 tiene que ser la segunda de V5. LOSA acaba en A y ninguna palabra de V5 acaba en A, así que se borra:

D2 = {LISO, ~~LOSA}~~ → D2 = {LISO}
D2 ha cambiado, así que habría que añadir ⟨V3,V2⟩ y ⟨V4,V2⟩, pero ya estaban en Q.
Q = [⟨V3,V0⟩, ⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩]

Paso 8: ⟨V3, V0⟩

Arista e1, ahora desde V3 (V3[0] = V0[0]). Las palabras de V0 empiezan por L o por S, así que OSO no tiene apoyo y se borra:

D3 = {LAS, LOS, ~~OSO,~~ SAL, SOL} → D3 = {LAS, LOS, SAL, SOL}
D3 ha cambiado, así que se añade ⟨V2,V3⟩, que ya se había revisado en el paso 5.
Q = [⟨V3,V2⟩, ⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩, ⟨V2,V3⟩]

Paso 9: ⟨V3, V2⟩

Arista e5, desde V3 (V3[2] = V2[0]). La única palabra que le queda a V2 es LISO, que empieza por L, así que V3 tiene que acabar en L. LAS y LOS acaban en S y se borran:
D3 = {~~LAS,~~ ~~LOS,~~ SAL, SOL} → D3 = {SAL, SOL}
D3 ha cambiado, así que se añade ⟨V0,V3⟩.
Q = [⟨V4,V0⟩, ⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩, ⟨V2,V3⟩, ⟨V0,V3⟩]

Paso 10: ⟨V4, V0⟩

Arista e2, desde V4 (V4[0] = V0[2]). Las palabras de V0 acaban en S o en L, así que OSO, que empieza por O, se borra:

D4 = {LAS, LOS, ~~OSO,~~ SAL, SOL} → D4 = {LAS, LOS, SAL, SOL}
D4 ha cambiado, así que se añaden ⟨V1,V4⟩ y ⟨V2,V4⟩.
Q = [⟨V4,V1⟩, ⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩, ⟨V2,V3⟩, ⟨V0,V3⟩, ⟨V1,V4⟩, ⟨V2,V4⟩]

Paso 11: ⟨V4, V1⟩

Arista e3, desde V4 (V4[1] = V1[0]). LAS y SAL tienen apoyo en AS, y LOS y SOL tienen apoyo en OS. Sin cambios.

Q = [⟨V4,V2⟩, ⟨V5,V1⟩, ⟨V5,V2⟩, ⟨V2,V3⟩, ⟨V0,V3⟩, ⟨V1,V4⟩, ⟨V2,V4⟩]

Paso 12: ⟨V4, V2⟩

Arista e6, desde V4 (V4[2] = V2[2]). LISO tiene una S en la posición 2, así que V4 tiene que acabar en S. SAL y SOL acaban en L y se borran:

D4 = {LAS, LOS, ~~SAL,~~ ~~SOL}~~ → D4 = {LAS, LOS}
D4 ha cambiado, así que se añade ⟨V0,V4⟩. También habría que añadir ⟨V1,V4⟩, pero ya estaba en Q.
Q = [⟨V5,V1⟩, ⟨V5,V2⟩, ⟨V2,V3⟩, ⟨V0,V3⟩, ⟨V1,V4⟩, ⟨V2,V4⟩, ⟨V0,V4⟩]

Paso 13: ⟨V5, V1⟩

Arista e4, desde V5 (V5[0] = V1[1]). Las palabras de V1 tienen una S o una O en la segunda posición, así que AS y NO se borran:

D5 = {~~AS,~~ ~~NO,~~ OS, SO} → D5 = {OS, SO}
D5 ha cambiado, así que se añade ⟨V2,V5⟩.
Q = [⟨V5,V2⟩, ⟨V2,V3⟩, ⟨V0,V3⟩, ⟨V1,V4⟩, ⟨V2,V4⟩, ⟨V0,V4⟩, ⟨V2,V5⟩]

Paso 14: ⟨V5, V2⟩

Arista e7, desde V5 (V5[1] = V2[3]). LISO acaba en O, así que V5 tiene que acabar en O. OS acaba en S y se borra:

D5 = {~~OS,~~ SO} → D5 = {SO}
D5 ha cambiado, así que se añade ⟨V1,V5⟩.
Q = [⟨V2,V3⟩, ⟨V0,V3⟩, ⟨V1,V4⟩, ⟨V2,V4⟩, ⟨V0,V4⟩, ⟨V2,V5⟩, ⟨V1,V5⟩]

Paso 15: ⟨V2, V3⟩

LISO empieza por L y tiene apoyo en SAL y SOL. Sin cambios.

Q = [⟨V0,V3⟩, ⟨V1,V4⟩, ⟨V2,V4⟩, ⟨V0,V4⟩, ⟨V2,V5⟩, ⟨V1,V5⟩]

Paso 16: ⟨V0, V3⟩

Este arco ya se revisó en el paso 1 sin cambios, pero se volvió a añadir en el paso 9 porque D3 cambió. Ahora las dos palabras de V3, SAL y SOL, empiezan por S, así que LAS no tiene apoyo y se borra:

D0 = {~~LAS,~~ SAL} → D0 = {SAL}
D0 ha cambiado, así que se añade ⟨V4,V0⟩.
Q = [⟨V1,V4⟩, ⟨V2,V4⟩, ⟨V0,V4⟩, ⟨V2,V5⟩, ⟨V1,V5⟩, ⟨V4,V0⟩]

Paso 17: ⟨V1, V4⟩

Ahora las palabras de V4 son LAS y LOS, con una A o una O en el centro, así que SO no tiene apoyo y se borra:

D1 = {AS, OS, ~~SO}~~ → D1 = {AS, OS}
D1 ha cambiado, así que se añade ⟨V5,V1⟩.
Q = [⟨V2,V4⟩, ⟨V0,V4⟩, ⟨V2,V5⟩, ⟨V1,V5⟩, ⟨V4,V0⟩, ⟨V5,V1⟩]

Pasos 18 a 23

Se revisan los arcos que quedan en Q, y en ninguno se borra nada, ya que todas las palabras tienen apoyo:

- ⟨V2,V4⟩: LISO tiene apoyo en LAS y LOS.
- ⟨V0,V4⟩: SAL tiene apoyo en LAS y LOS.
- ⟨V2,V5⟩: LISO tiene apoyo en SO.
- ⟨V1,V5⟩: AS y OS tienen apoyo en SO.
- ⟨V4,V0⟩: LAS y LOS tienen apoyo en SAL.
- ⟨V5,V1⟩: SO tiene apoyo en AS y OS.

Q = [ ]

La cola se ha quedado vacía y ningún dominio está vacío, así que AC3 devuelve CIERTO.
Los dominios finales son:
D0 = {SAL}
D1 = {AS, OS}
D2 = {LISO}
D3 = {SAL, SOL}
D4 = {LAS, LOS}
D5 = {SO}

AC3 revisa 23 arcos y en 10 de ellos borra alguna palabra, logrando que haya entre todos los dominios un total de 9 palabras, frente a las 22 que había antes de ejecutarse, y como ningún dominio se queda vacío AC3 no detecta que el problema no tenga solución.

6.7 Heurística de ordenación de variables

El orden en que se asignan las variables influye muchísimo en el tiempo de resolución de los distintos algoritmos, y por ello vamos a analizar las siguientes heurísticas para determinar cuál es la mejor elección.

Heurística	Criterio
original	El orden en el que se crean las variables: primero las horizontales y luego las verticales.
dominio	Primero las variables con menos palabras en su dominio.
grado	Primero las variables que se cruzan con más variables.
conectado	Se empieza por la variable con el dominio más pequeño. Después se elige siempre la que más se cruza con las variables ya ordenadas. Los empates se deshacen con el dominio más pequeño y, si siguen empatadas, con el mayor número de cruces.

6.7.1 Función ordenarVariables

```
def ordenarVariables(listaVariables, heuristica):
    if heuristica == "original":
        return listaVariables
    if heuristica == "dominio":
        return sorted(listaVariables, key=lambda v: len(v.dominio))
    if heuristica == "grado":
        return sorted(listaVariables, key=lambda v: -len(v.restricciones))
    if heuristica == "conectado":
        pendientes = list(listaVariables)
        orden = []
        while len(pendientes) > 0:
            mejor = None
            for v in pendientes:
                cruces = 0
                for restriction in v.restricciones:
                    if restriction[0] in orden:
                        cruces += 1
                clave = (cruces, -len(v.dominio), len(v.restricciones))
                if mejor is None or clave > mejorClave:
                    mejor = v
                    mejorClave = clave
            orden.append(mejor)
            pendientes.remove(mejor)
        return orden
    return listaVariables
```

Esta función recibe la lista de las variables y se encarga de ordenarla según la heurística que se desea emplear. En los casos *dominio* y *grado* basta con ordenar la lista con *sorted*,

según el tamaño del dominio o según el número de cruces de cada variable, y en el caso de *conectado* parece un poco más complejo, pero simplemente va construyendo el orden variable a variable, empezando por la del dominio más pequeño. En cada vuelta del bucle elige, de entre las variables pendientes, la que más se cruza con las que ya están en orden. Si hay empate, se queda con la de dominio más pequeño y, si sigue el empate, con la que tiene más cruces y para ello se compara la tupla clave, que tiene estos tres valores en ese orden. La variable elegida se añade a orden y se quita de pendientes, hasta que no queda ninguna.

6.7.2 Modificación main.py

Hemos añadido una variable constante llamada *HEURISTICA* para poder elegir el tipo de heurística a emplear.

```
HEURISTICA = "conectado"
```

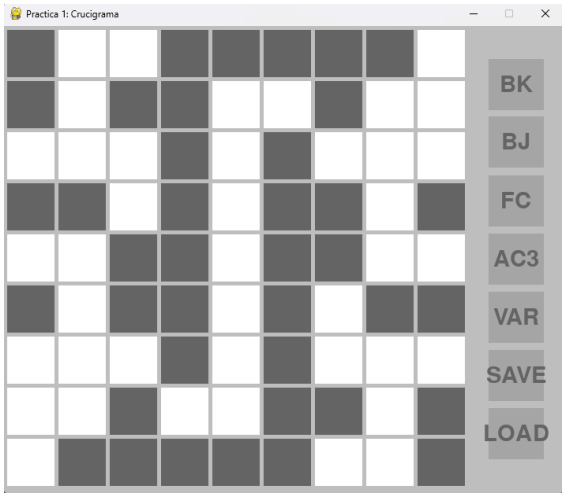
Y también hemos modificado el retorno de la función *obtenerVariables*, para indicar la heurística:

```
return ordenarVariables(todasVariables, HEURISTICA)
```

Si queremos probar otra Heurística, basta con modificar la variable constante.

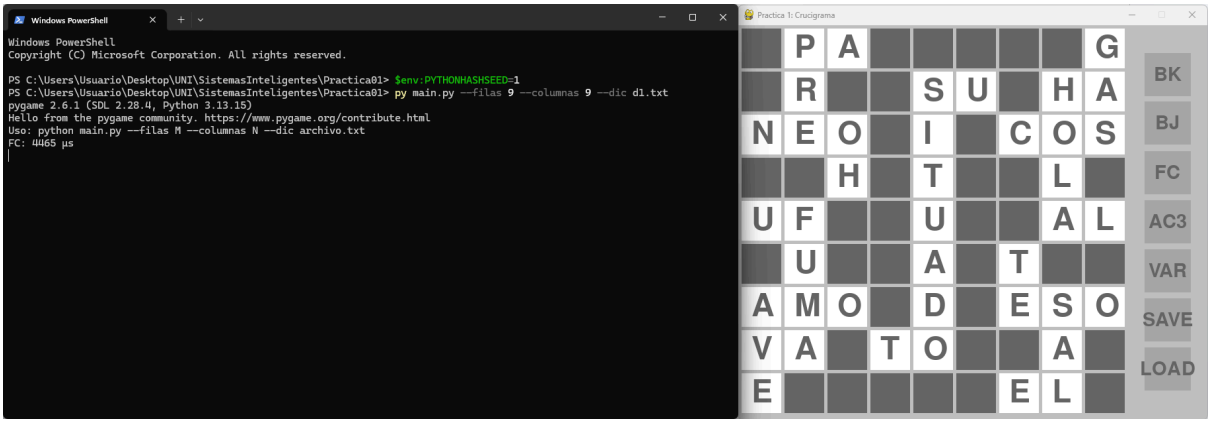
6.7.3 Ejecución de heurísticas en tablero 9x9

Para analizar las diferentes heurísticas, vamos a usar el siguiente tablero 9x9, que es el que hemos usado para analizar todos los algoritmos, y vamos a ejecutar *ForwardChecking*.

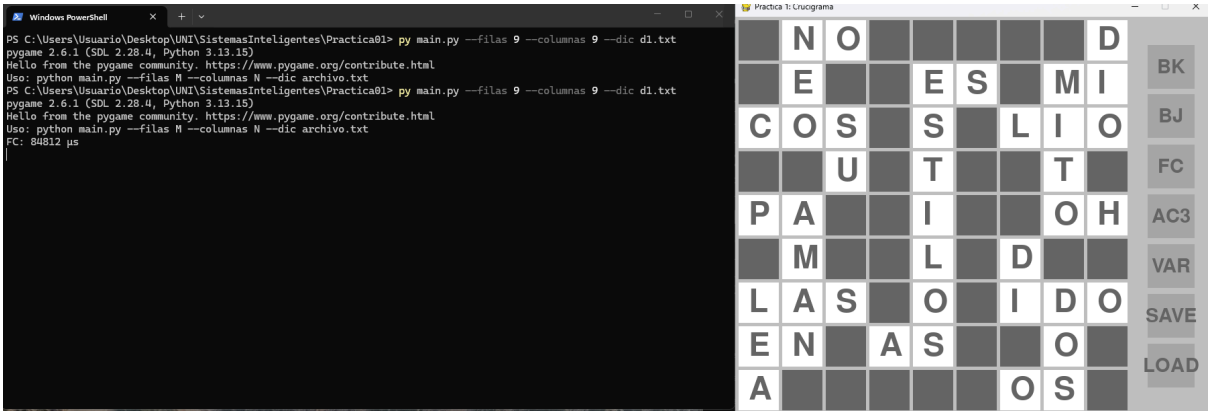


SEMILLA 1

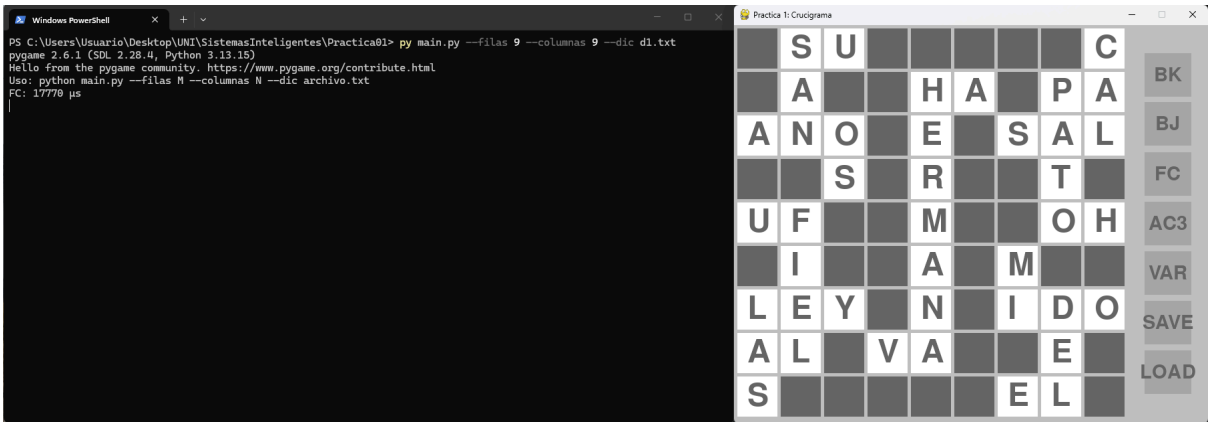
- **dominio - 4465 µs**



- **grado - 84812 µs**

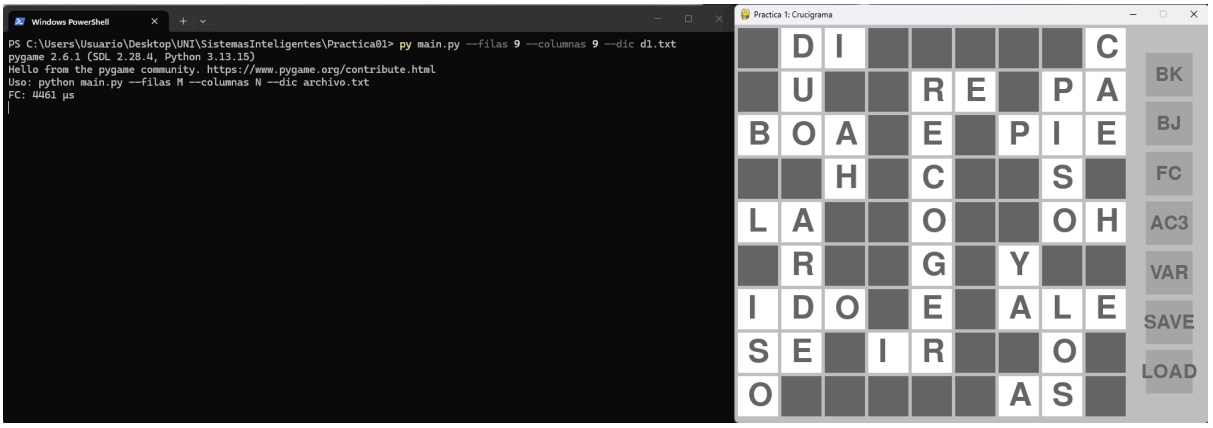


- **conectado - 17770 µs**

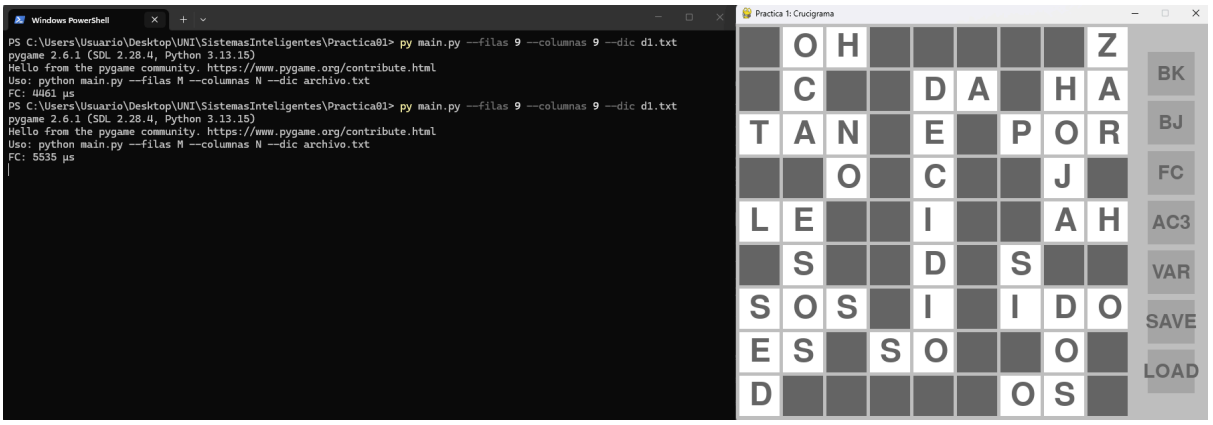


SEMILLA 2

- **dominio: > 60s**
- **grado - 4461 µs**

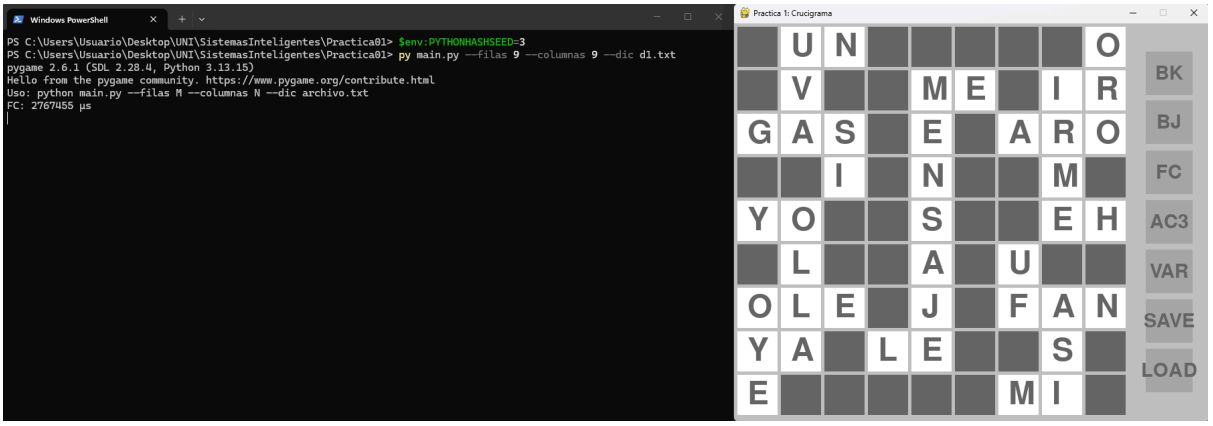


- **conectado - 5535 µs**

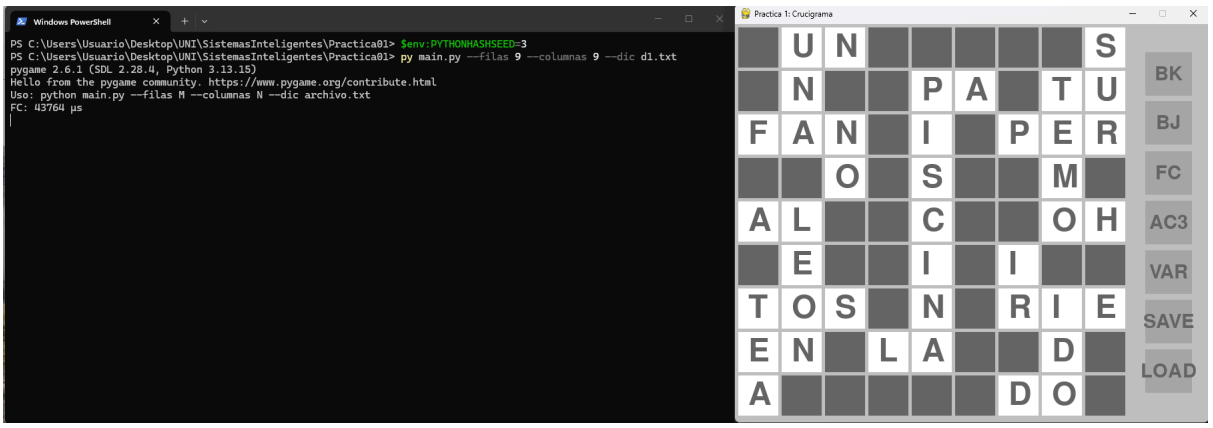


SEMILLA 3

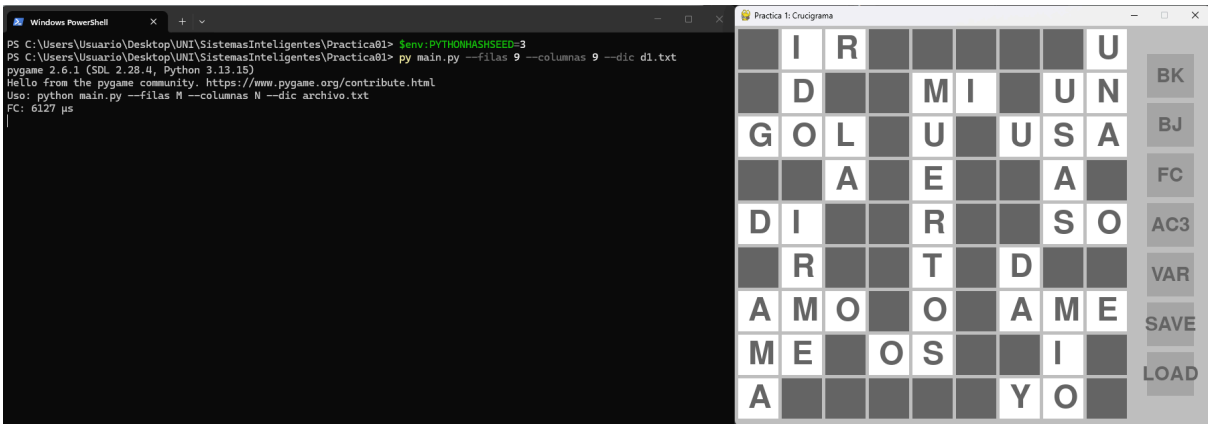
- **dominio - 2767455 µs**



- **grado - 43764 µs**



- **conectado - 6127 µs**

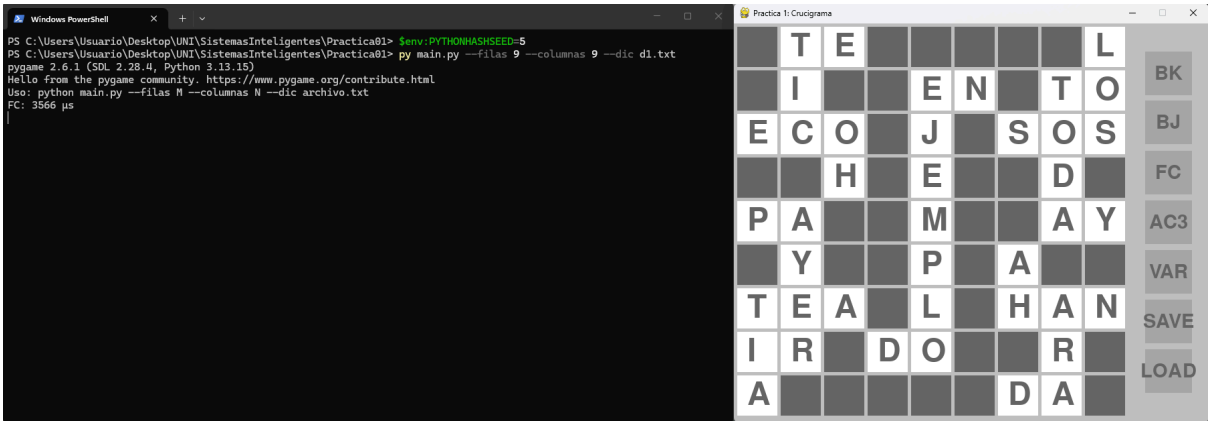


SEMILLA 5

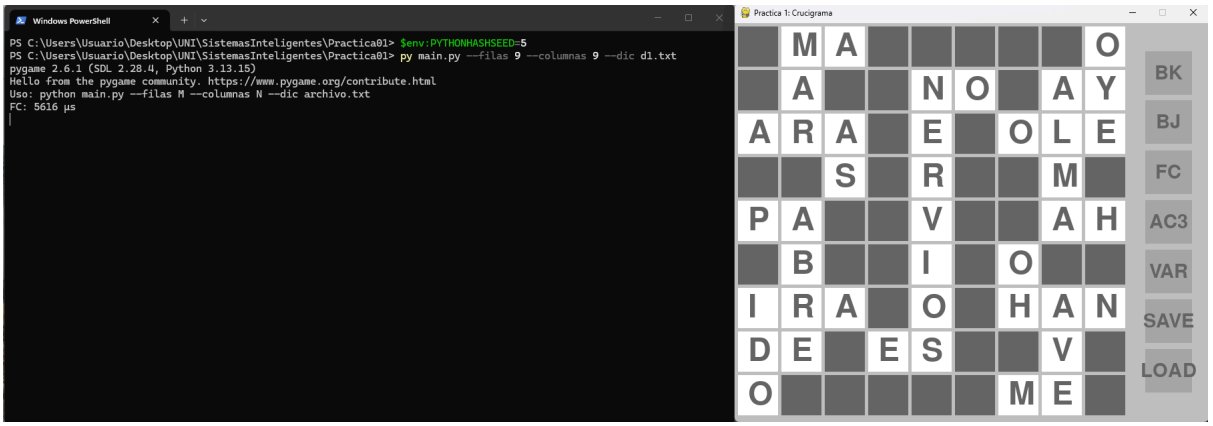
- **dominio - 2416287 µs**



- **grado - 3566 µs**

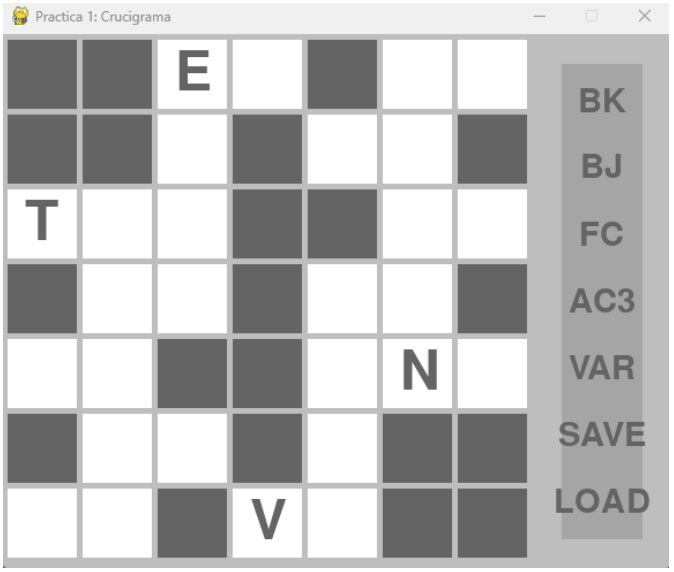


- **conectado - 5616 µs**



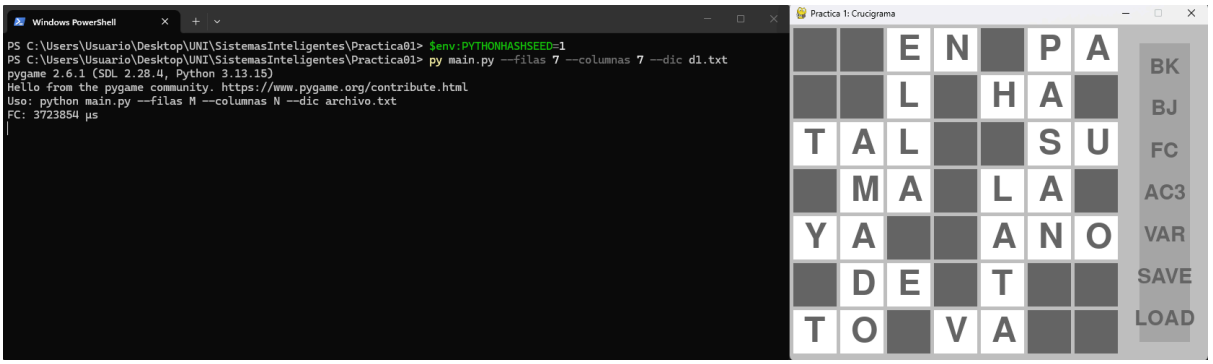
6.7.4 Ejecución de heurísticas en tablero 7x7

También analizaremos las heurísticas con el siguiente tablero:

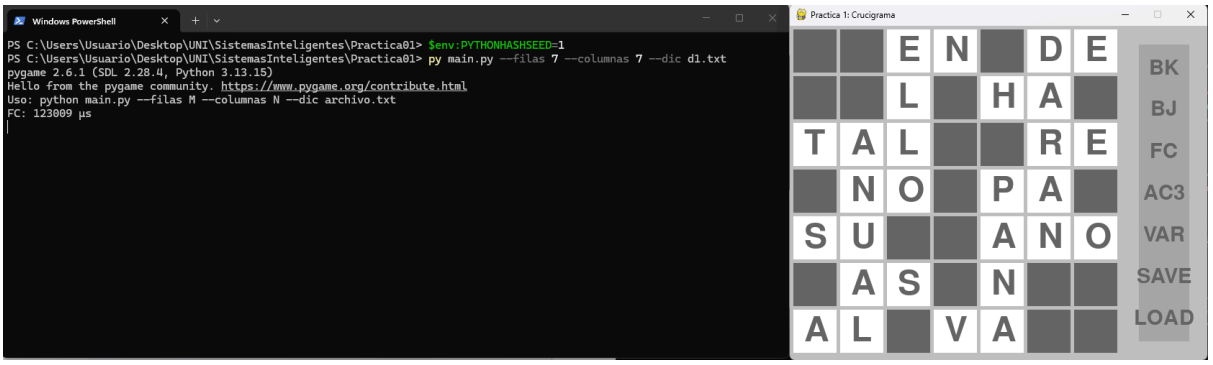


SEMILLA 1

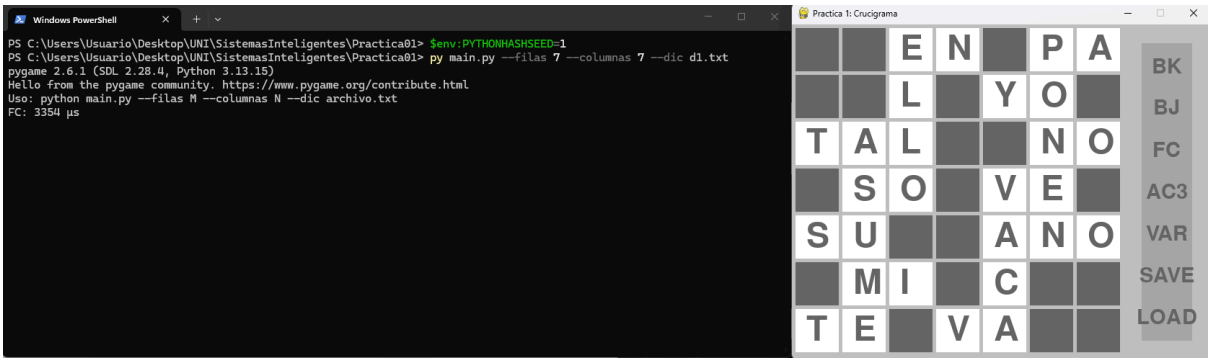
- **dominio - 3723854 µs**



- **grado - 123009 µs**

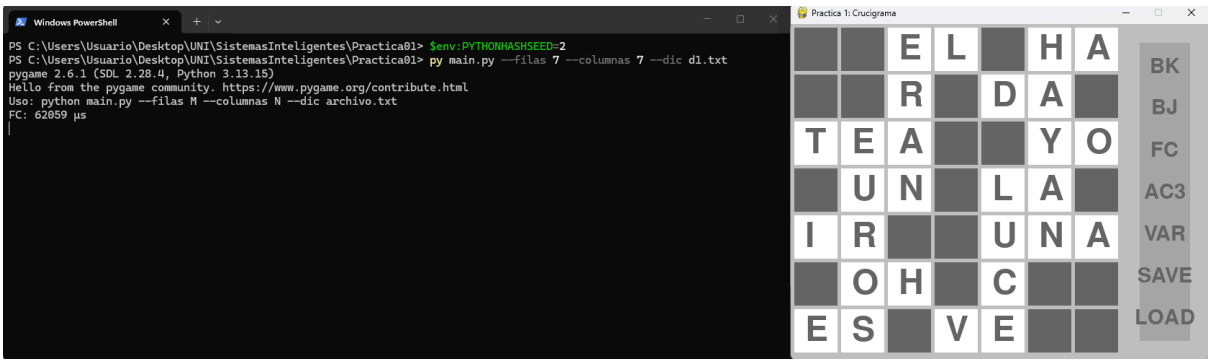


- **conectado - 3354 µs**

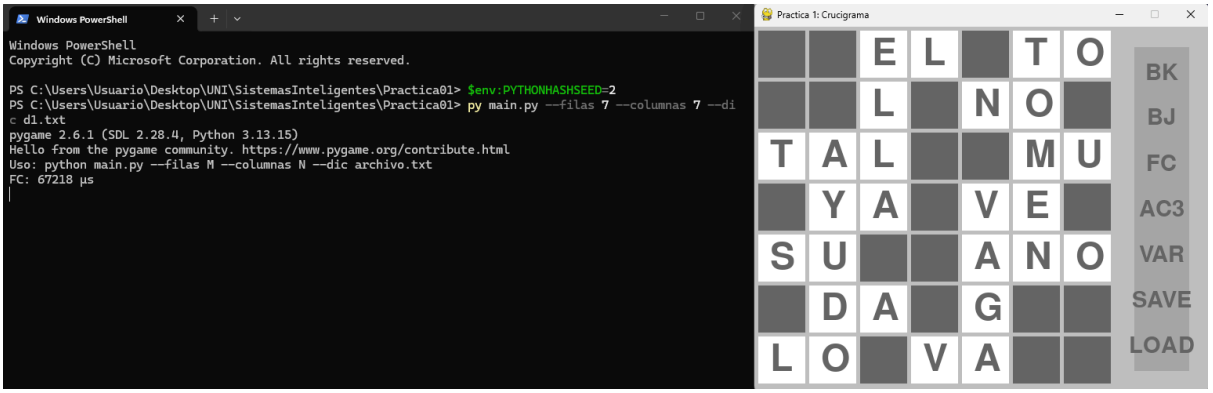


SEMILLA 2

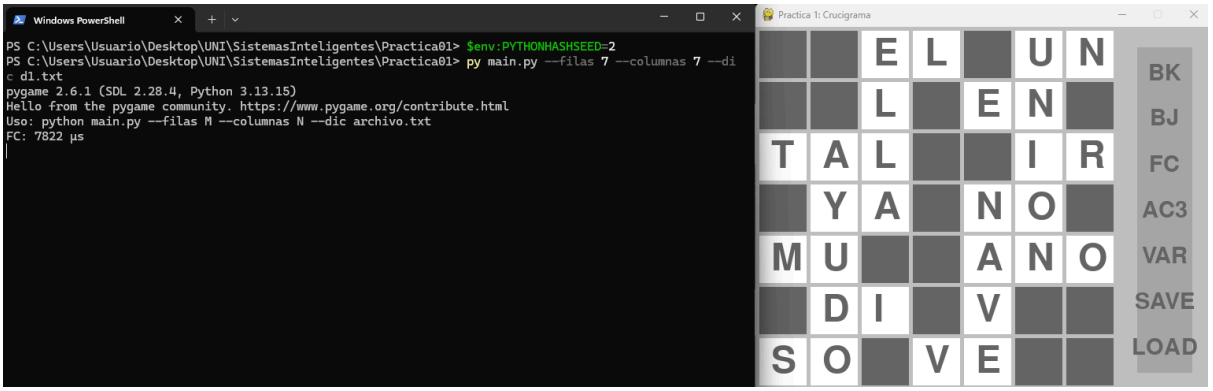
- **dominio - 62059 µs**



- **grado - 67218 µs**

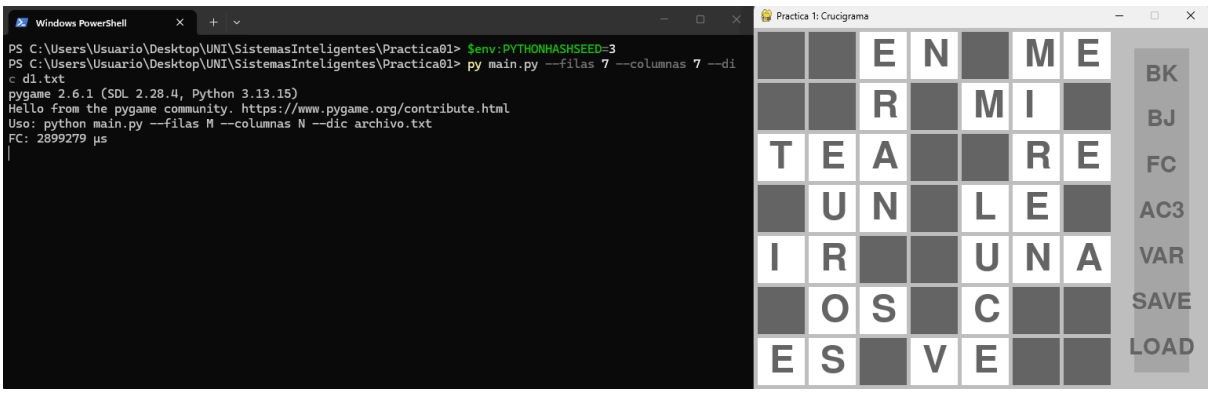


- **conectado - 7822 µs**

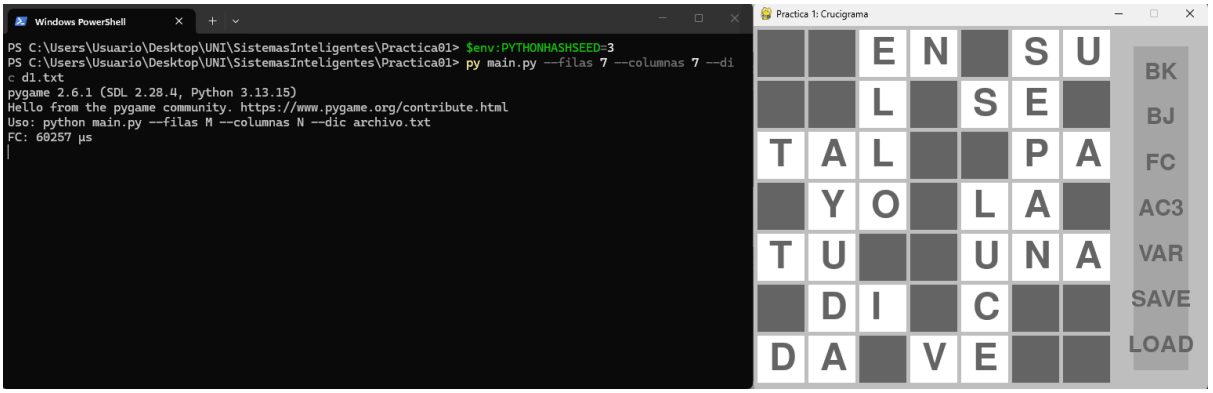


SEMILLA 3

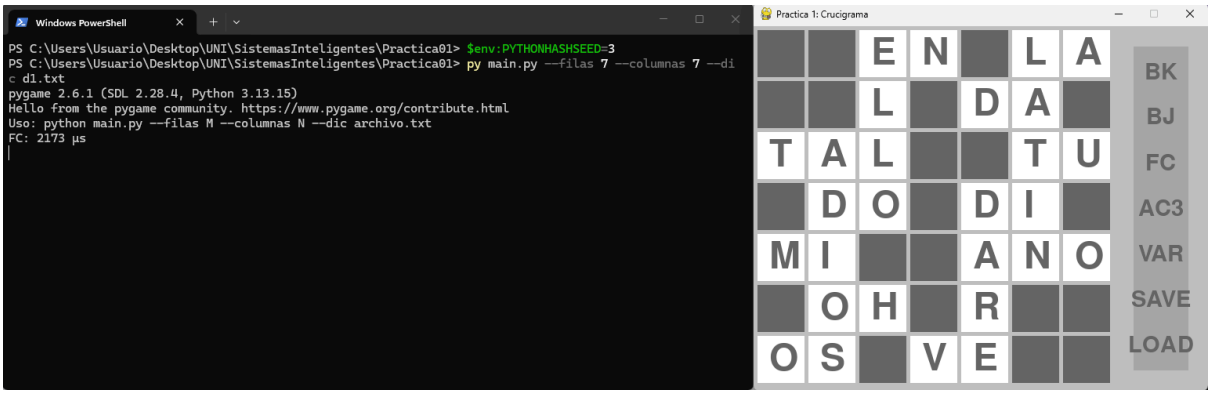
- **dominio - 2899279 µs**



- **grado - 60257 µs**

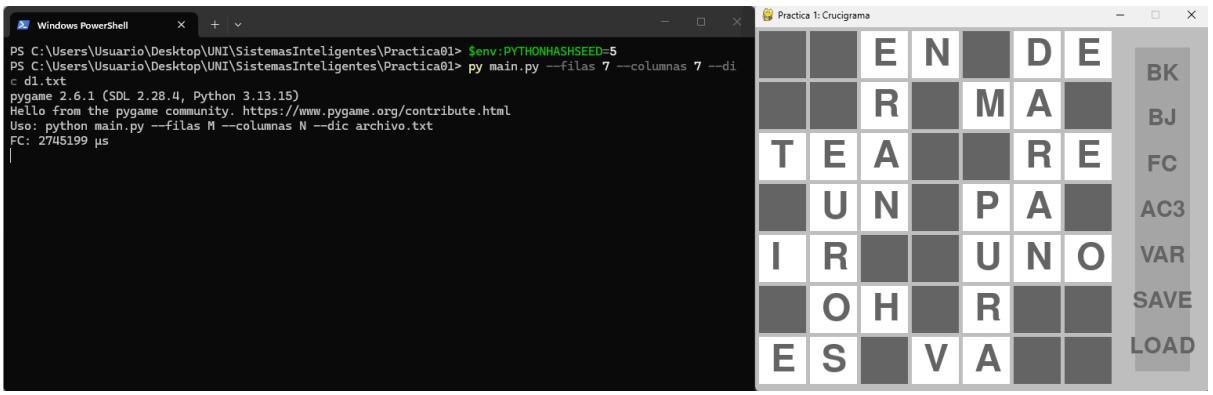


- **conectado - 2173 µs**

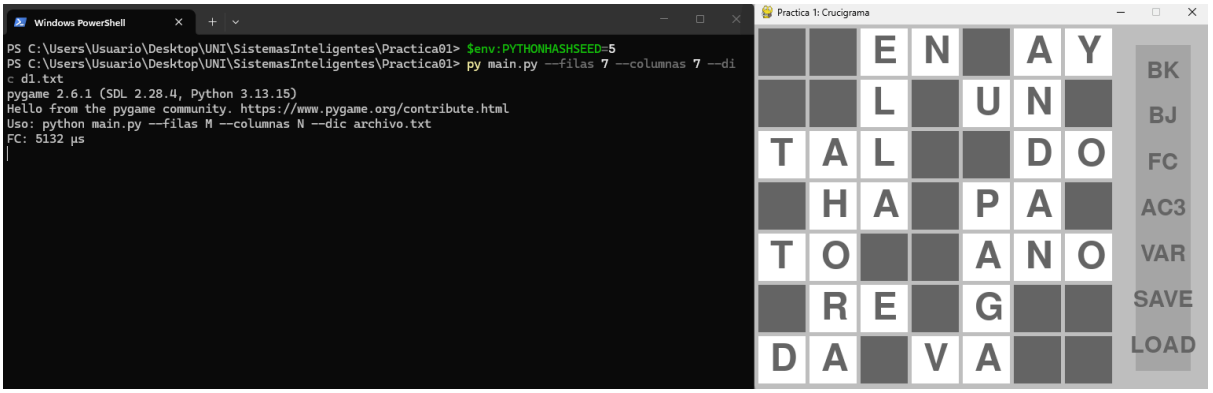


SEMILLA 5

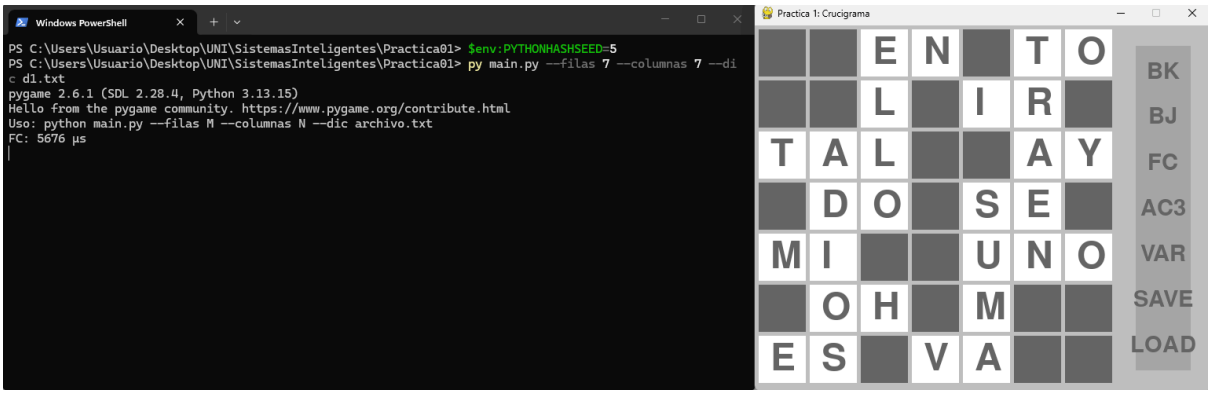
- **dominio - 2745199 µs**



- **grado - 5132 µs**



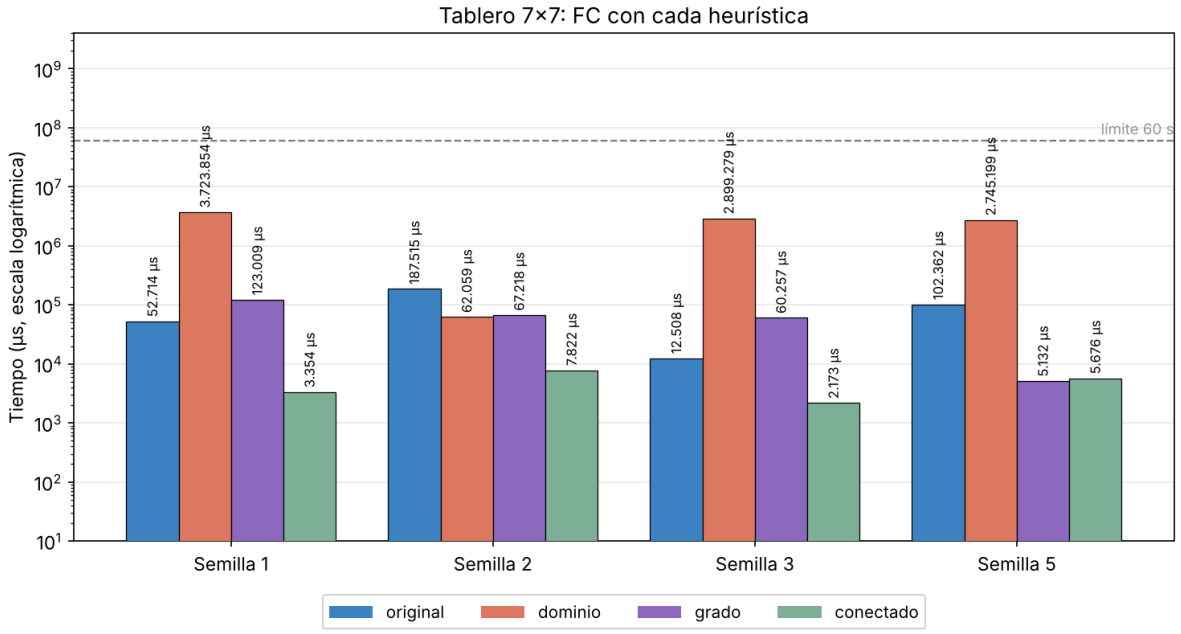
- **conectado - 5676 µs**



6.7.5 Resultados de las heurísticas

Tablero 7x7

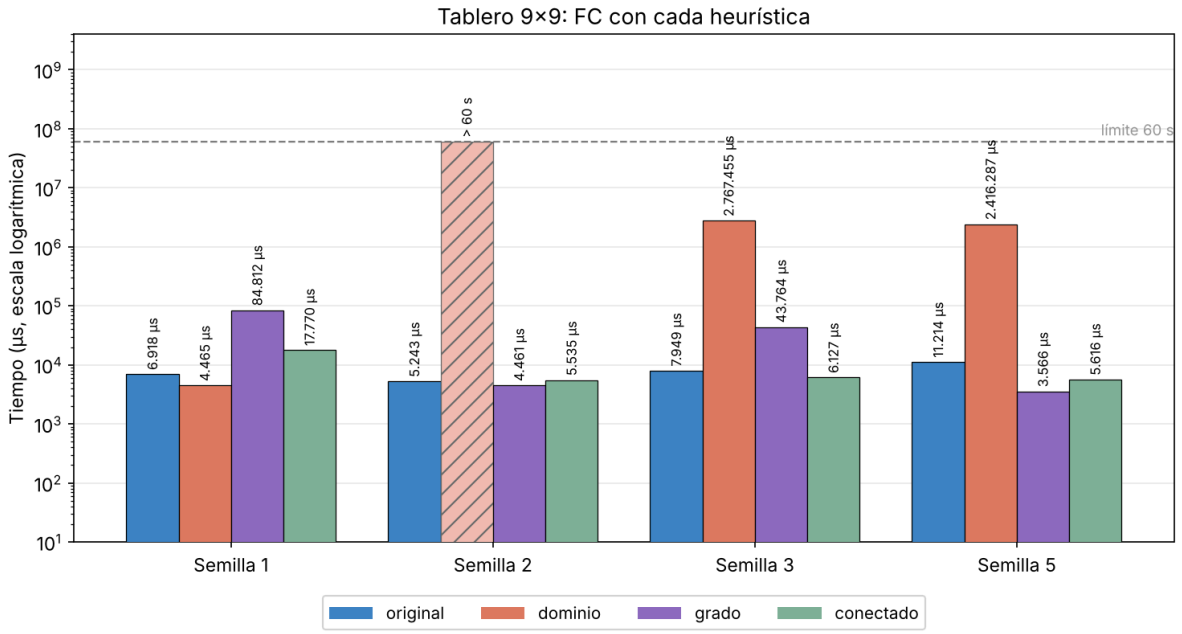
Heurística	Semilla 1 - µs	Semilla 2 - µs	Semilla 3 - µs	Semilla 5 - µs	Media - µs
original	52.714	187.515	12.508	102.362	88.775
dominio	3.723.854	62.059	2.899.279	2.745.199	2.357.598
grado	123.009	67.218	60.257	5.132	63.904
conectado	3.354	7.822	2.173	5.676	4.756



En el tablero 7x7 es donde más se nota la diferencia entre las heurísticas, ya que con el orden original ForwardChecking tardaba entre 12.508 µs y 187.515 µs, con una media de 88.775 µs, mientras que con *conectado* tarda entre 2.173 µs y 7.822 µs, con una media de 4.756 µs, siendo unas 19 veces más rápido. *Conectado* es la mejor en tres de las cuatro semillas, y en la semilla 5 queda prácticamente empatada con *grado*, con 5.676 µs frente a 5.132 µs. Por otro lado, *grado* mejora al orden original en las semillas 2 y 5 pero empeora en la 1 y la 3, y *dominio* vuelve a ser la peor con diferencia, con una media de 2,36 segundos.

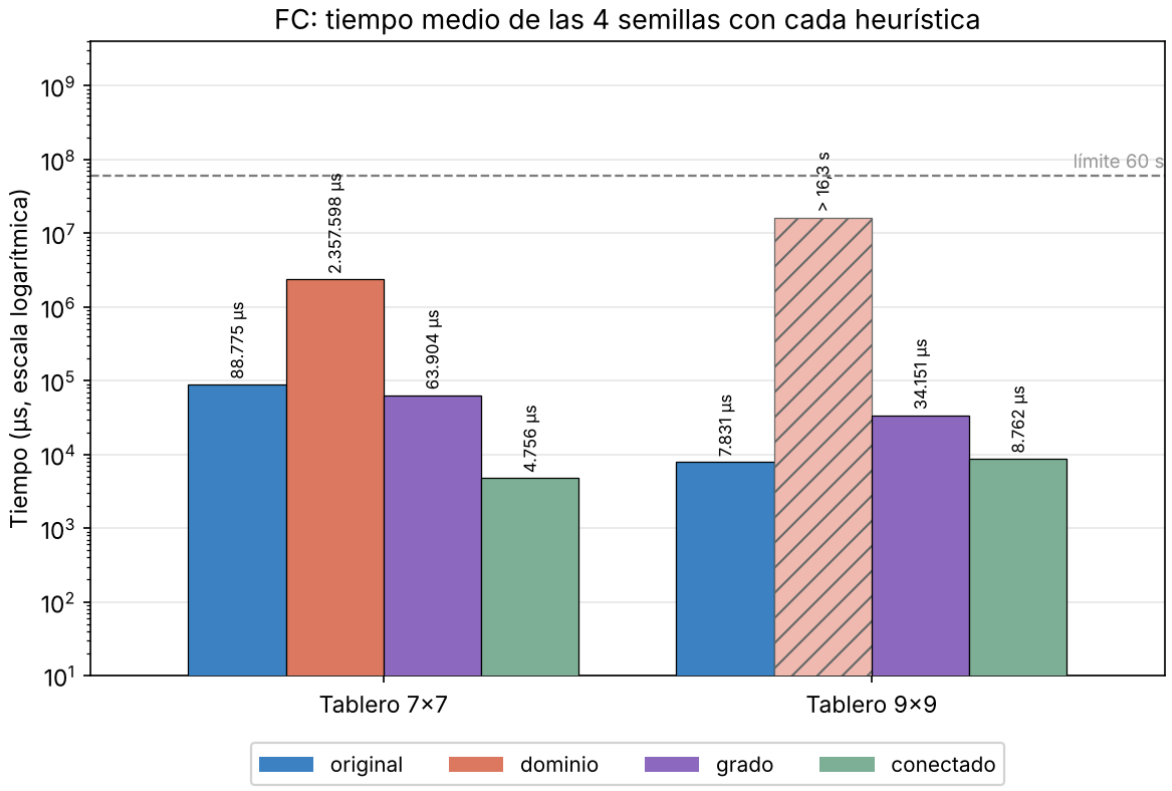
Tablero 9x9

Heurística	Semilla 1 - µs	Semilla 2 - µs	Semilla 3 - µs	Semilla 5 - µs	Media - µs
original	6.918	5.243	7.949	11.214	7.831
dominio	4.465	> 60 s	2.767.455	2.416.287	-
grado	84.812	4.461	43.764	3.566	34.151
conectado	17.770	5.535	6.127	5.616	8.762



En el tablero 9x9, en cambio, podemos observar que el orden original y *conectado* obtienen tiempos muy parecidos, con una media de 7.831 µs y 8.762 µs, y esto se debe a que en este tablero ForwardChecking ya era rápido con el orden original, por lo que no hay mucho margen de mejora. *Grado* es muy irregular, ya que según la semilla tarda desde 3.566 µs hasta 84.812 µs, y *dominio* es la peor con diferencia, ya que aunque con la semilla 1 es la más rápida con 4.465 µs, en las otras tres semillas tarda entre 2,4 segundos y más de 60 segundos. Esto se debe a que dominio empieza por las variables de 2 letras, que son las que menos palabras tienen en su dominio, pero están repartidas por el tablero y casi no se cruzan entre sí, por lo que FC las asigna sin que unas poden a las otras y los fallos aparecen mucho más tarde.

6.7.6 Conclusiones sobre la heurística



En la gráfica del tiempo medio, dominio en el 9x9 aparece como más de 16,3 segundos, ya que al superar la semilla 2 los 60 segundos no podemos calcular la media exacta, pero contando esa semilla como 60 segundos la media ya sería de 16,3 segundos.

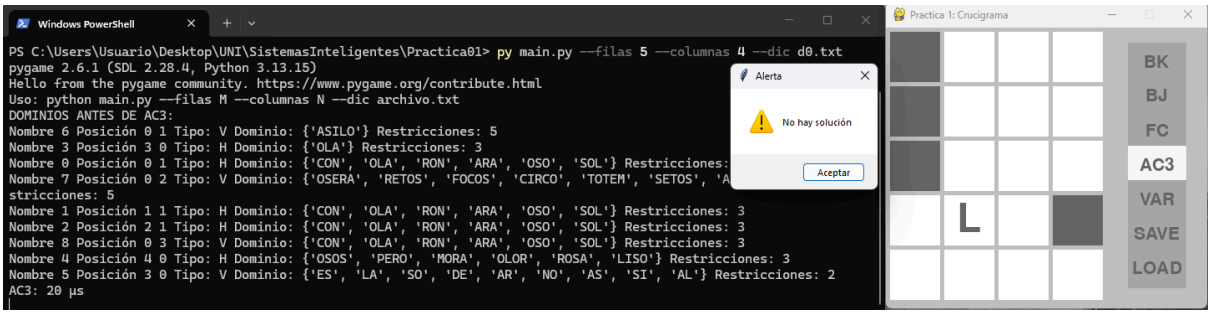
Por lo que sabiendo esto, podemos concluir que la heurística más adecuada es conectado, ya que en el tablero 7x7 es con diferencia la más rápida, con una media de 4.756 µs frente a los 88.775 µs del orden original, y en el 9x9 obtiene prácticamente el mismo tiempo que el orden original. Además, es la más estable, ya que su peor tiempo en todas las ejecuciones es de 17.770 µs, mientras que el orden original llega a 187.515 µs, grado a 123.009 µs y dominio pasa de los 60 segundos. Esto se debe a que con conectado cada variable se cruza con el mayor número posible de variables anteriores, por lo que FC ya ha podado su dominio antes de que le llegue su turno y los fallos se detectan enseguida. Por este motivo, en el programa entregado hemos dejado ***HEURISTICA = "conectado"***, manteniendo el resto de heurísticas en el código aunque no se utilicen, y los estudios de tiempos del resto de la memoria se han realizado con el orden original, para que la única diferencia entre las ejecuciones fuera el algoritmo.

6.8 Casuísticas

El objetivo de estas pruebas va a ser comprobar que hace *AC3* en cada una de las tres situaciones posibles y que hace después la búsqueda.

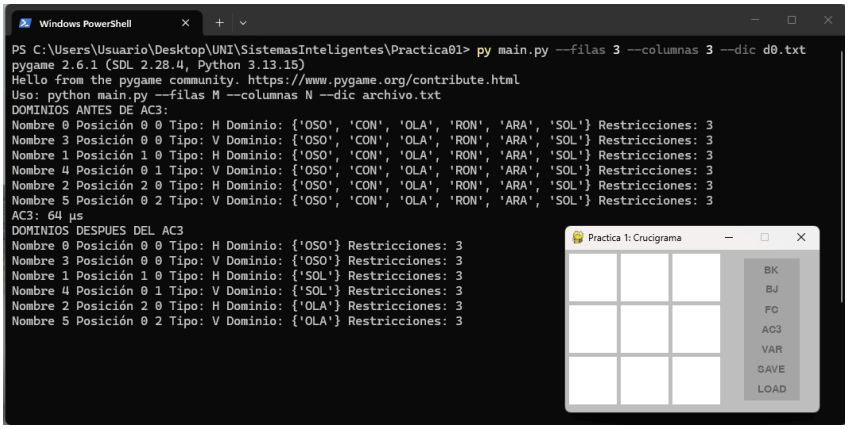
6.8.1 Caso 1: sin solución detectado por AC3

Utilizando un tablero de 5x4 con las casillas configuradas que se van a mostrar a continuación, junto al diccionario original de la práctica *d0.txt*, al aplicar *AC3* obtenemos el siguiente resultado:

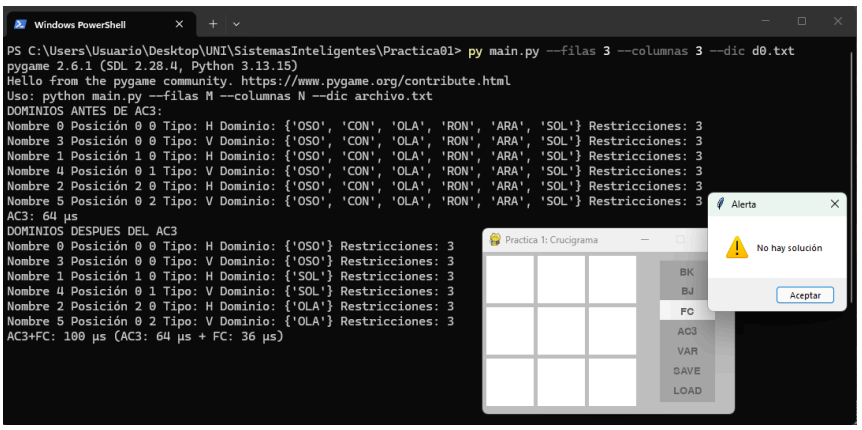


6.8.2 Caso 2: AC3 deja valores, pero no hay solución

Utilizando un tablero de 3x3 con las casillas configuradas que se van a mostrar a continuación, junto al diccionario original de la práctica *d0.txt*, al aplicar *AC3* obtenemos el siguiente resultado:



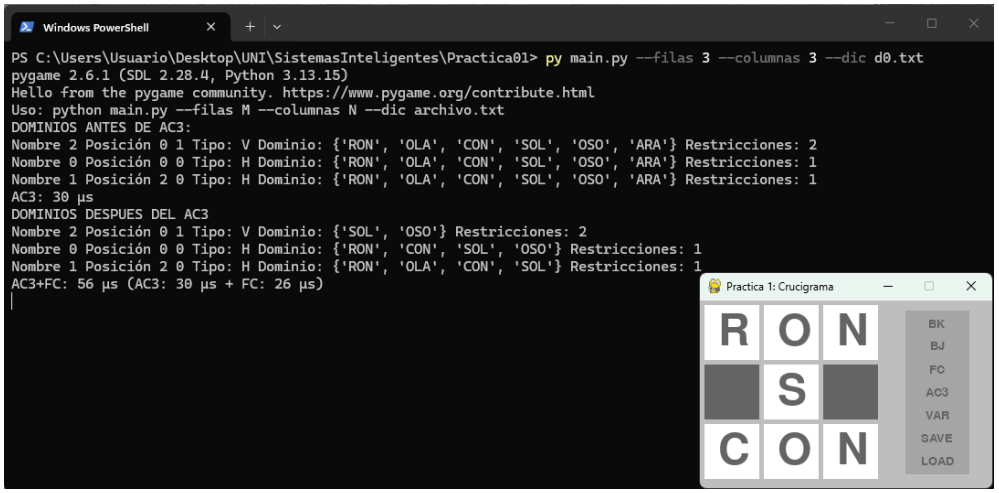
Como podemos observar, no salta ningún error al realizar *AC3*, y nos deja un solo valor en el dominio de cada variable, y si intentamos aplicar ahora *FC*:



El resultado es que no hay solución, y esto nos sirve para comprobar que *AC3* no se encarga de solucionar el problema.

6.8.3 Caso 3: con solución

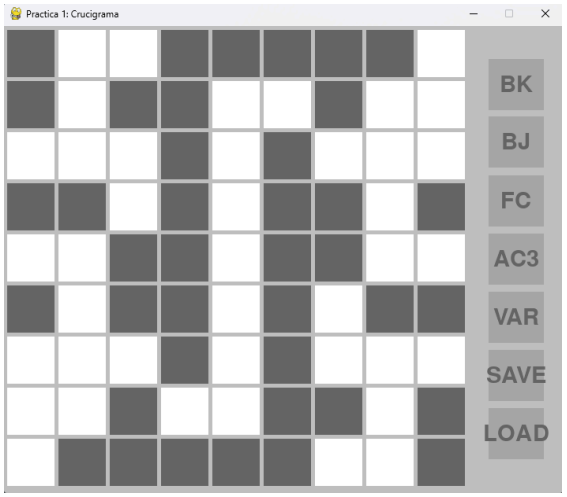
Utilizando un tablero de 3x3 con las casillas configuradas que se van a mostrar a continuación, junto al diccionario original de la práctica *d0.txt*, al aplicar *AC3* obtenemos el siguiente resultado:



Podemos comprobar cómo *AC3* ha reducido los dominios, y además *FC* ha podido encontrar la solución con esos dominios reducidos.

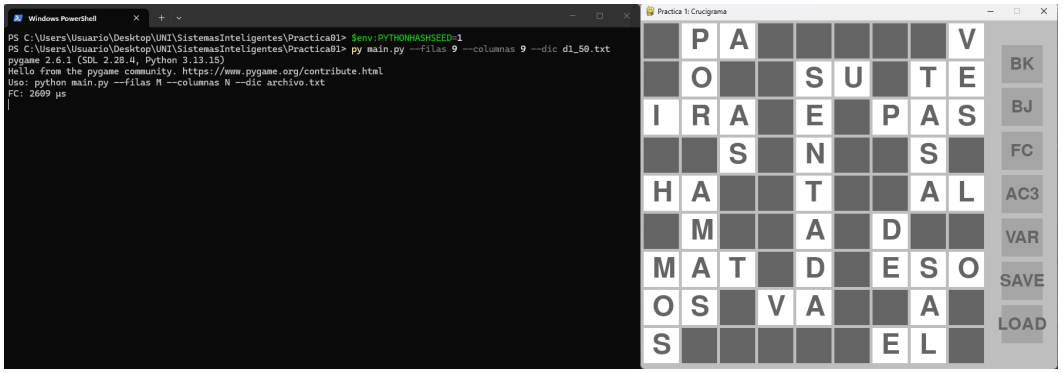
6.9 Estudio de tiempo con otro diccionario

En anteriores apartados realizamos el estudio de los diferentes algoritmos en tres tableros distintos y concluimos resultados con un solo diccionario, para ver como pueden diferir los tiempos en otro diccionario distinto, se ha crado *d1_50.txt* con una cantidad de 3.013 palabras, y que usaremos para ejecutar todos los algoritmos en el tablero 9x9 que hemos ido utilizando.



SEMILLA 1

- **BackTracking > 60s**
- **AC3 + BackTracking > 60s**
- **ForwardChecking - 2609 µs**



- **AC3 + ForwardChecking - 9695 µs(8623 µs + 1073 µs)**

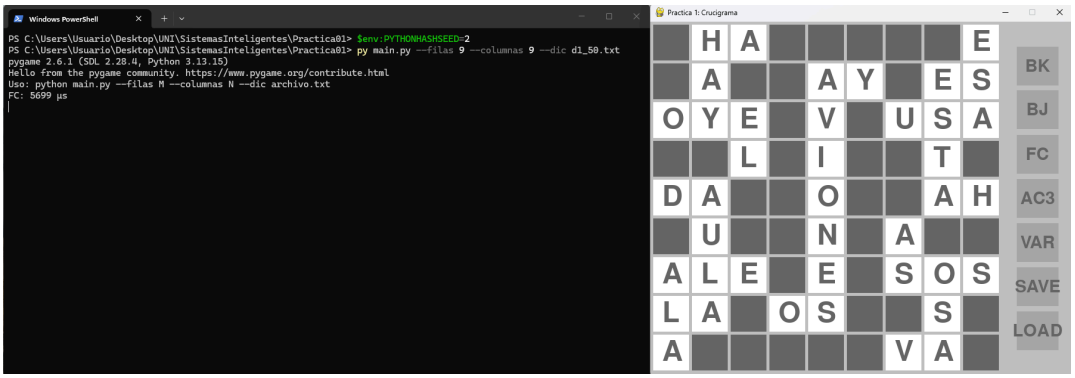


- **BackJumping > 60s**
- **AC3 + BackJumping - 32372 µs(9326 µs + 23047 µs)**



SEMILLA 2

- **BackTracking > 60s**
- **AC3 + BackTracking > 60s**
- **ForwardChecking - 5699 µs**



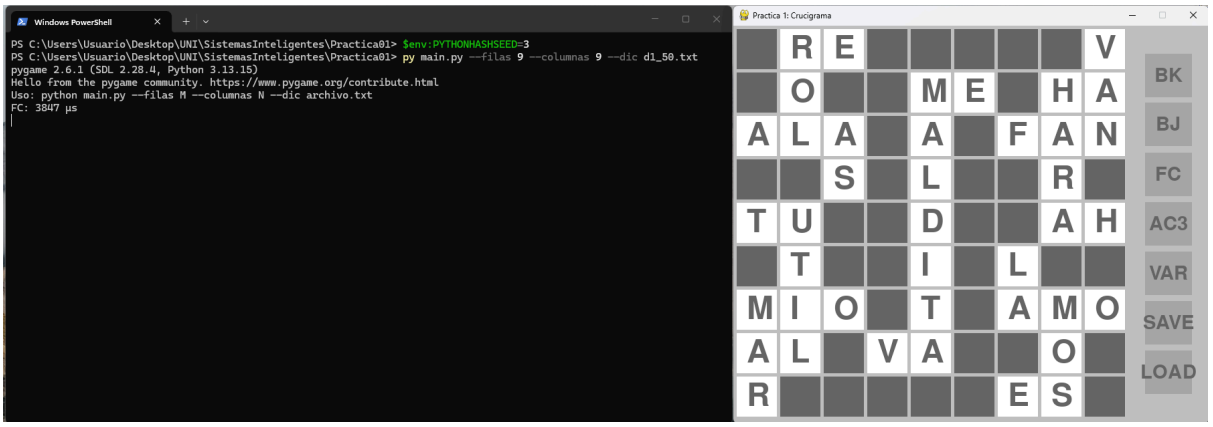
- **AC3 + ForwardChecking - 11881 µs(8482 µs + 3399 µs)**



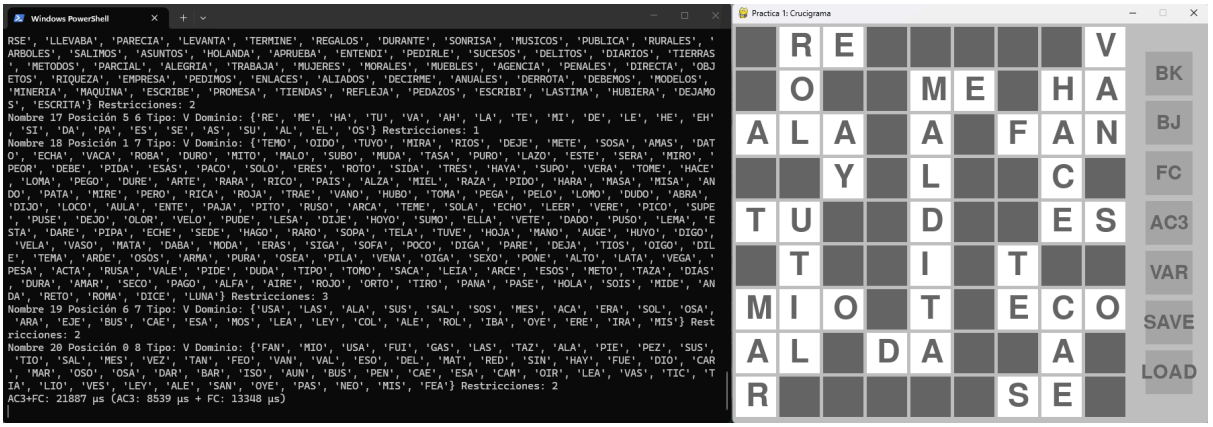
- **BackJumping > 60s**
- **AC3 + BackJumping > 60s**

SEMILLA 3

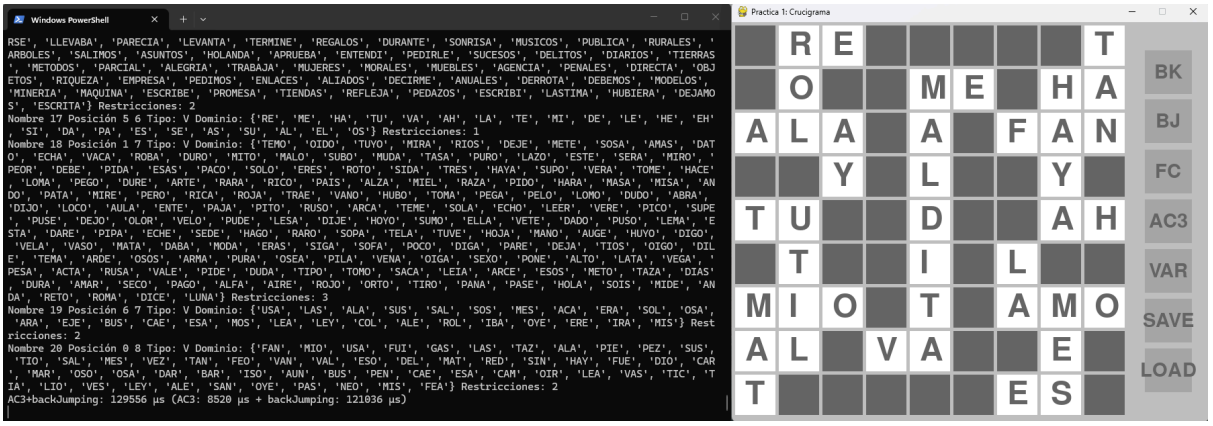
- **BackTracking > 60s**
- **AC3 + BackTracking > 60s**
- **ForwardChecking - 3847 µs**



- **AC3 + ForwardChecking - 21887 µs(8539 µs + 13348 µs)**

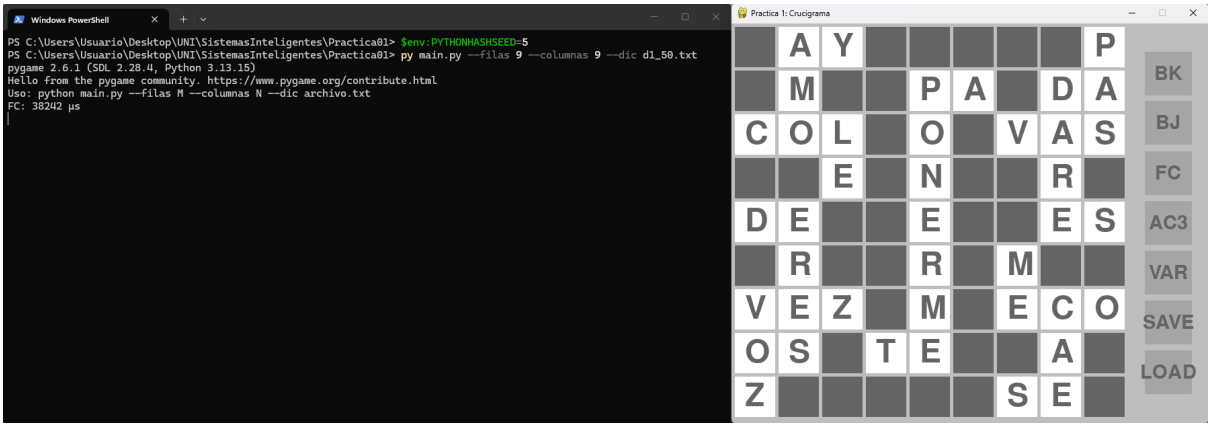


- **BackJumping > 60s**
- **AC3 + BackJumping 129556 µs (8520 µs + 121036 µs)**



SEMILLA 5

- **BackTracking > 60s**
- **AC3 + BackTracking > 60s**
- **ForwardChecking - 38242 µs**



- **AC3 + ForwardChecking - 10165 µs(8306 µs + 1859 µs)**

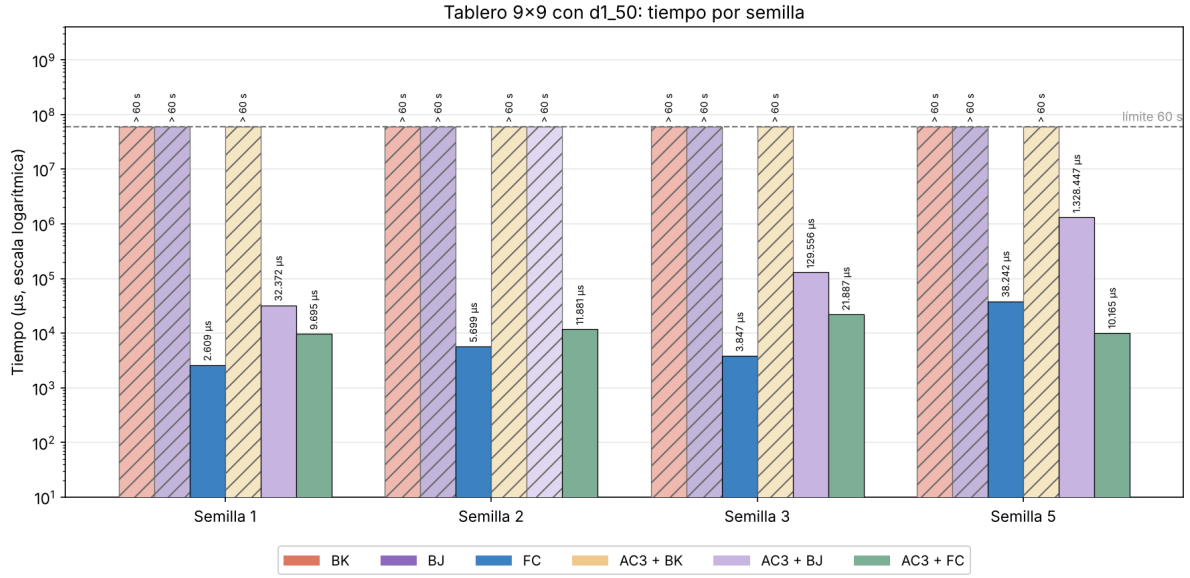


- **BackJumping > 60s**
- **AC3 + BackJumping 1328447 µs(8290 µs + 1320157 µs)**



Tabla y gráfica del diccionario d1_50.txt

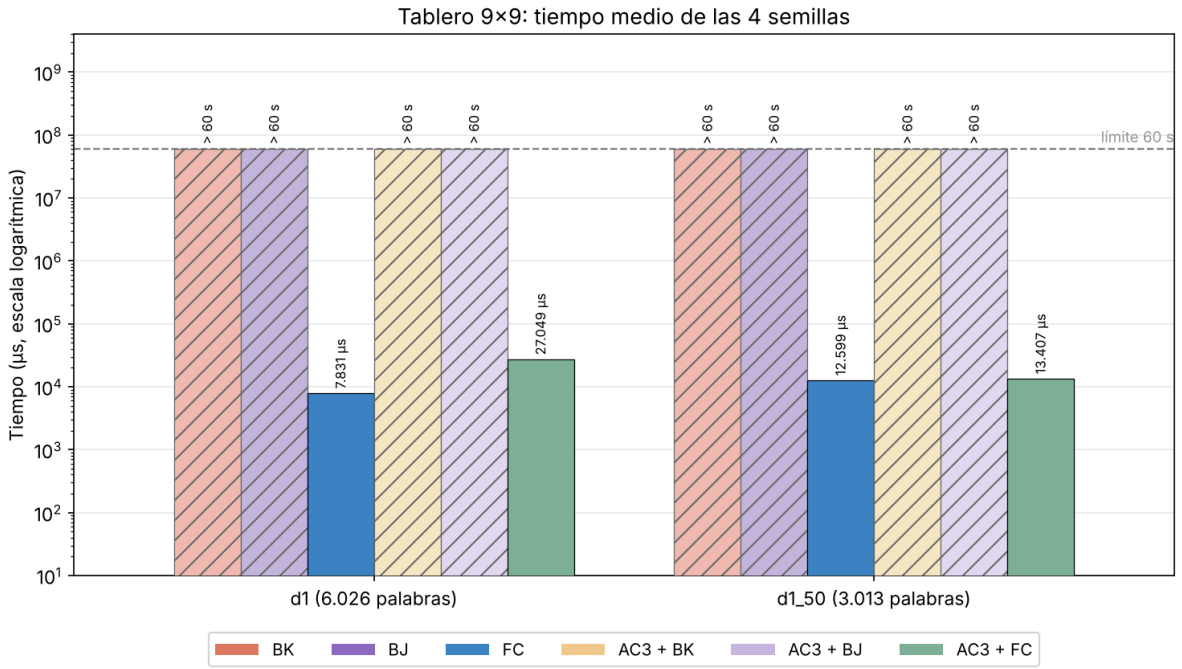
Algoritmo	Semilla 1 - µs	Semilla 2 - µs	Semilla 3 - µs	Semilla 5 - µs	Media - µs
BK	> 60 s	> 60 s	> 60 s	> 60 s	-
FC	2.609	5.699	3.847	38.242	12.599
BJ	> 60 s	> 60 s	> 60 s	> 60 s	-
AC3+BK	> 60 s	> 60 s	> 60 s	> 60 s	-
AC3+FC	9.695 (AC3 8.623)	11.881 (AC3 8.482)	21.887 (AC3 8.539)	10.165 (AC3 8.306)	13.407
AC3+BJ	32.372 (AC3 9.326)	> 60 s	129.556 (AC3 8.520)	1.328.447 (AC3 8.290)	-



Podemos observar que con el diccionario *d1_50.txt* los algoritmos de BackTracking, BackJumping y AC3 + BackTracking no terminan en menos de 60 segundos en ninguna semilla, mientras *Forward Checking*, con y sin AC3 encuentra solución rápidamente en un tiempo de entre 2.609 µs y 38.242 µs, y AC3 + BackJumping es más irregular, ya que según la semilla puede encontrar solución desde los 32.372 µs hasta 1,33 segundos, o puede no encontrarlo en menos de 60 segundos.

Comparación entre distintos diccionarios

Algoritmo	d1 (6.026 palabras)	d1_50 (3.013 palabras)
BK	Solo termina con la semilla 2	> 60 s en todas las semillas
FC	Media 7.831 µs	Media 12.599 µs
BJ	Solo termina con la semilla 2	> 60 s en todas las semillas
AC3+BK	Solo termina con la semilla 2	> 60 s en todas las semillas
AC3+FC	Media 27.049 µs	Media 13.407 µs
AC3+BJ	Solo termina con la semilla 2	Termina en las semillas 1, 3 y 5



Podemos observar que lo primero que cambia es el tiempo de ejecución de *AC3*, ya que tarda unos 8.500 µs frente a 22.600 µs, ya que los dominios al tener la mitad de palabras, tiene muchas menos parejas que comprobar, y por ello *AC3 + FC* consigue pasar de una

media de 27.049 µs a 13.407 µs, estando muy cerca de *Forward Checking*. En cambio *Forward Checking* empeora pasando de una media de 7.831 µs a 12.599 µs, y esto es debido a que como hay menos palabras, también hay menos soluciones posibles, y por tanto más caminos sin salida.

7. Dificultades encontradas

Cuando implementé por primera vez el algoritmo de Forward Checking, utilizaba una función auxiliar llamada esConsistenteFC, que comprobaba si dos palabras coincidían en la casilla en la que se cruzan sus variables. Al tener que implementar AC3, pensé que podía reutilizar la misma función, ya que AC3 también tiene que comprobar los cruces, y para la restricción de que las palabras no se repitan, Forward Checking usaba también la función isFeasible de BackTracking, que comprueba cada palabra con las variables que ya están asignadas.
Sin embargo no fue hasta las primeras pruebas para comprobar BackTracking, Forward Checking y AC3 cuando en el tablero 9x9 con la semilla 3, Forward Checking tardaba unos 4,5 segundos, cuando lo normal es que tardase unos pocos miles de microsegundos. Revisando el código, vimos que el problema era que forward solo podaba por los cruces, por lo que las palabras repetidas no se eliminaban de los dominios de las variables futuras, y Forward Checking no detectaba que una variable se iba a quedar sin palabras válidas hasta que le llegaba su turno, explorando muchas más combinaciones de las necesarias. Para solucionarlo, hicimos que esConsistenteFC comprobase tanto el cruce como que las palabras sean distintas, eliminamos la llamada a isFeasible de Forward Checking y creamos una nueva función, esConsistenteCruce, que solo comprueba el cruce y es la que usa AC3. Con este cambio, ese mismo caso pasó de unos 4,5 segundos a 7.949 µs, aunque tuvimos que repetir todas las ejecuciones de Forward Checking y AC3 + FC.

8. Conclusiones

En esta práctica hemos visto cómo un problema como la generación de crucigramas se puede plantear como un problema de satisfacción de restricciones en el que las variables son los huecos del tablero, los dominios son las palabras del diccionario que tienen la longitud adecuada y se ajustan a las letras fijas, y las restricciones son que dos variables que se cruzan tienen la misma letra en la casilla que comparten y que además no se repite ninguna palabra.

Este problema planteado se puede resolver con distintos algoritmos que hemos ido implementando cada sesión y hemos podido aprender como funcionan cada uno y cuando implementarlos.

Por ejemplo, BackTracking es el algoritmo más sencillo, ya que va a probar todas las combinaciones posibles hasta encontrar el resultado, pero este solo es útil en tableros pequeños, ya que ha medido que íbamos aumentando el tamaño de los tableros, su tiempo crece exponencialmente, como podemos ver que el tablero que usamos de 6x6 era el único que lo ejecutaba en menos de 60 segundos constantemente, y aun así tardaba mucho más que otros algoritmos.

En cuanto a ForwardChecking, hemos visto cómo se trata de una mejora de BackTracking, ya que esté poda las variables futuras que aún no se han asignado, consiguiendo evitar que el propio algoritmo esté buscando entre palabras que no van a estar en la solución, y este algoritmo es el que ha presentado los tiempos de resolución más bajos.

También hemos implementado BackJumping, que sigue siendo una mejora sobre BackTracking, sin embargo no consigue tan buenos resultados como Forward Checking, y además hay ejecuciones que de por sí solo el algoritmo tarda más de 60 segundos en el tablero 9x9.

Y sobre los tres algoritmos mencionados anteriormente, hemos visto la aplicación del algoritmo *AC3* para reducir los dominios de las variables, y en algunos casos consigue muy buenos resultados, sobre todo si lo usamos con BackTracking, o con BackJumping, ya que casi siempre les mejora los resultados, además de que puede detectarnos rápidamente de que un problema no tiene solución, sin embargo con Forward Checking depende del tablero: en el 6x6 y sobre todo en el 7x7 AC3 + FC es más rápido, pero en el 9x9, donde Forward Checking ya es rápido de por sí, el coste de AC3 hace que sea más rentable no usarlo.

Visto y analizado cómo se comportan los distintos algoritmos con un solo diccionario, también era interesante descubrir como serían los resultados con otro diccionario, y por ello en el tablero 9x9, usando un diccionario distinto con la mitad de palabras que el primero, hemos podido comprobar como el coste extra de *AC3* se redució, ya que los dominios tienen la mitad de palabras que recorrer, y también hemos visto como el tiempo de ejecución de *ForwardChecking* se incrementó debido a que como hay la mitad de palabras, también hay menos soluciones, y por tanto hace que el algoritmo entre en más ramas sin solución.

Por último, hemos visto como la utilización de diferentes heurísticas puede ayudar o perjudicar en el tiempo de resolución de los problemas, en concreto la que más nos ha ayudado ha sido la heurística de *conectado*, que es la que hemos dejado en el código final,sin embargo, es importante saber que todas las ejecuciones de los algoritmos para estudiarlos se han hecho con el orden original de las variables.

9. Uso de IA

La Inteligencia Artificial ha sido un componente muy útil de cara a este trabajo, ya que ha permitido acelerar brutalmente el conocimiento del contenido de la asignatura, y ayudar con la implementación de esta práctica. Eso sí, es muy importante saber cómo utilizamos la Inteligencia Artificial, ya que nos puede dar todo el trabajo hecho y no aprender nada, y por ello se ha utilizado de la siguiente manera:

Mediante el modelo *Opus 5.5* de *Claude, Anthropic,* con esfuerzo en *Extra*, le hemos pasado todo el contenido de esta asignatura junto a los enunciados de las prácticas, y gracias a su enorme cantidad de contexto que puede cargar, es capaz de asesorarte adecuadamente. Para ello le hemos especificado un prompt a la Inteligencia Artificial:

Actúa como un profesor universitario de la asignatura Sistemas Inteligentes. Te adjunto los apuntes del tema de problemas de satisfacción de restricciones y el enunciado de la práctica. Vas a guiarme paso a paso en la realización de la práctica, sesión por sesión, pero es muy importante que:

- *No me des el código de la solución. Explícame los conceptos, hazme preguntas y dame pistas para que sea yo quien lo implemente.*
- *Cuando te pase mi código, señala los errores y explícame por qué fallan, pero deja que sea yo quien los corrija.*
- *Si me equivoco en algún concepto de la teoría, corrígeme y remíteme a la parte de los apuntes donde se explica.*
- *Asegúrate de que lo que implemento sigue el pseudocódigo del enunciado y de la teoría.*

Con esto, tenemos a nuestra disponibilidad una especie de profesor particular que nos ayuda a realizar la práctica.

Además del uso para poder realizar y comprender la práctica, también la hemos utilizado para generar código que nos permite crear las imágenes de las distintas gráficas que hemos utilizado, y también para la generación de los dos diccionarios *d1.txt* y *d1_50.txt*.

Y por último, para comprobar que toda la práctica cumple con los requisitos y que no ha habido ningún problema o error de implementación, le pasamos un zip con toda la práctica y la memoria, y aquí es justo donde la Inteligencia Artificial detectó la anomalía de la ejecución de *ForwardChecking* en el tablero 9x9, que en una semilla daba un resultado de 4,5 segundos, y además de distintos apartados de la memoria que no aparecían y eran requisitos para la entrega, que ahora sí están implementados y corregidos.

10. Referencias bibliográficas

1. Universidad de Alicante (2026). *Tema 3.2: Búsqueda en problemas de satisfacción de restricciones*. Sistemas Inteligentes, curso 2026/2027. Departamento de Ciencia de la Computación e Inteligencia Artificial.
2. Universidad de Alicante (2026). *Práctica 1. Satisfacción de restricciones*. Sistemas Inteligentes, curso 2026/2027.
3. Russell, S. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed., Global Edition). Pearson. Capítulo 6: Constraint Satisfaction Problems.
4. Haralick, R. M. y Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3), 263–313. (Forward Checking)
5. Mackworth, A. K. (1977). Consistency in networks of relations. *Artificial Intelligence*, 8(1), 99–118. (AC3)
6. Prosser, P. (1993). Hybrid algorithms for the constraint satisfaction problem. *Computational Intelligence*, 9(3), 268–299. (BackJumping)
7. Dechter, R. (2003). *Constraint Processing*. Morgan Kaufmann. (Heurísticas de ordenación de variables)
8. Python Software Foundation. *Documentación de Python 3*: función time.perf_counter y variable de entorno PYTHONHASHSEED. https://docs.python.org/3/
9. Pygame. *Documentación oficial de pygame*. https://www.pygame.org/docs/
10. Anthropic (2026). *Claude Opus 5.5* [modelo de lenguaje]. https://claude.ai